



Senai – Santos Dumont
Técnico de Redes de Computadores

Alessandro Roberto dos Reis Santos
Bruno Souza dos Santos
Giovani Hardt Filho

Implementação de um Ambiente Seguro em Redes de Computadores

Professor Orientador: Prof. Paulo Eduardo Galvão Alves

São José dos Campos - SP

2019

Alessandro Roberto dos Reis Santos
Bruno Souza dos Santos
Giovani Hardt Filho

Implementação de um Ambiente Seguro em Redes de Computadores

Trabalho de Conclusão de Curso, apresentado às disciplinas de Projeto I e II, do curso presencial de Técnico de Redes de Computadores, da Escola SENAI “Santos Dumont” da cidade de São José dos Campos - SP, como requisito parcial para a obtenção do título de Técnico de Redes de Computadores.

Orientador: Prof. Paulo Eduardo Galvão Alves

São José dos Campos - SP

2019

Dedicatória

Dedico este trabalho à minha família, em específico à minha adorável e querida esposa Alessandra de Assis Fonseca Santos, que contribuiu efetivamente com o seu apoio, com o seu amor e todo o seu carinho para que eu desenvolvesse e aplicasse meus conhecimentos adquiridos ao longo do curso neste estudo e trabalho.

Alessandro Roberto dos Reis Santos

Dedicatória

Dedico este trabalho a todos os meus familiares que me apoiaram nessa longa jornada, em específico a minha mãe Claudia Cristina Souza dos Santos por sempre me aconselhar e me ajudar, e ao professor Airton Cesar Zombardi por me ajudar a estar sempre focado no objetivo proposto deste projeto que julgo ser tão importante para minha carreira profissional.

Bruno Souza dos Santos

Dedicatória

Dedico este trabalho à minha mãe, Elisangela Aparecida do Prado, que contribuiu pacientemente para que fosse possível a elaboração do trabalho, e agradeço também à minha tia Diana do Prado Calabrez por ter me dado a oportunidade de poder cursar este curso de Redes de Computadores.

Giovani Hardt Filho

AGRADECIMENTOS

A Deus, por todo o seu amor em nos abençoar e nos proporcionar saúde, inteligência e sabedoria para que conseguíssemos trilhar nossos caminhos nesta nova e intensa jornada.

Ao ex-diretor da escola SENAI “Santos Dumont”, sr. Carlos Ortunho Serra por ter nos proporcionado e ter apresentado uma excelente administração do curso técnico de redes de computadores enquanto responsável pela gestão da escola.

Ao ex-coordenador pedagógico dos cursos técnicos da escola SENAI “Santos Dumont”, sr. Ricardo Ladeira, pela ética e pela destreza apresentadas no apoio pedagógico e na colaboração à administração do nosso curso.

Aos professores Kleber Gelli e Paulo Galvão que ao longo do curso, dedicaram-se com extremo compromisso, empenho, empatia, caráter e ética no fomento e na transferência de seus conhecimentos técnicos para conosco alunos, sempre nos motivando e nos orientando, demonstrando não apenas o excelente ensino pedagógico e profissionalizante, mas acompanhado de um carinho e respeito imenso pelo aluno e ser humano em si, qualidades estas que nos faz refletir neste momento de gratidão, respeito e admiração pelos mesmos.

A todos os nossos colegas de turma e sala, por demonstrarem competências comportamentais no que infere ao nosso excelente relacionamento interpessoal, destacando a imensa empatia e coleguismo dos mesmos para conosco.

A todos aqueles, que de alguma forma, seja ela direta ou indiretamente, colaboraram para o desenvolvimento desta pesquisa ajudando-nos a atingirmos o escopo e a conclusão efetiva do presente trabalho e projeto proposto.

“Você é livre para fazer suas escolhas,
mas é prisioneiro das consequências.”

Pablo Neruda

RESUMO

Para que uma rede de computadores consiga desempenhar efetivamente seu papel, suas respectivas funções e ao mesmo tempo atingir seu intento de forma focada e satisfatória, no quesito e no que diz respeito ao atendimento eficaz ao usuário final, dando-lhe inclusive uma sensação concisa e percebida de segurança em um ambiente virtual, julga-se então ser de suma e de fundamental importância a relevância da realização de um estudo minucioso e detalhado, bem como o acompanhamento do desempenho da segurança e principalmente da saúde das máquinas em um ambiente de rede virtual e que, para que isso ocorresse, o grupo desempenhou fortemente suas competências técnicas e prontamente foi realizado um estudo acerca do uso de ferramentas de segurança e monitoramento de redes, as quais foram abordadas durante todo o ciclo deste trabalho, em uma análise realizada minuciosamente em um ambiente simulado de redes.

Inicialmente e embasado nos estudos realizados em sala de aula e em conformidade com o *brainstorming* realizado de acordo com o consenso do grupo, foram definidos os critérios, as métricas e todos os *try-outs* executados em exaustão para a avaliação dos *softwares* em um ambiente simulado de redes.

Tais análises têm o objetivo de demonstrar com eficácia o uso e a atribuição das ferramentas de segurança e detectoras, enfatizando-as e inserindo-as em um cenário previamente definido pelo grupo, abordando especificamente o monitoramento de falhas, a detecção, a identificação precisa de intrusão, o gerenciamento do desempenho de toda a rede e também a simulação de uma armadilha virtual destinada a um possível invasor, sendo todas estas tarefas atribuídas a quatro ferramentas de utilizações gratuitas e ou código aberto por assim dizer, sendo elas: Pfsense – Snort – Honeypot – Zabbix, ferramentas as quais serão responsáveis em demonstrar como abordaremos alguns critérios de avaliação, em específico no que tange à instalação das ferramentas, à configuração de *hosts* e serviços, além de apresentarmos através das mesmas, a capacidade de gerar informações claras e objetivas, bem como toda a eficiência e eficácia que abrange uma gestão correta no quesito segurança em uma rede de computadores.

Palavras chave: Implementação; Segurança; Redes; Computadores.

ABSTRACT

In order for a computer network to be able to effectively play its role, its respective functions and at the same time achieve its purpose in a focused and satisfying manner, in terms of effective end-user care, even giving it a concise and perceived safety in a virtual environment, it is therefore considered of paramount importance to carry out a thorough and detailed study, as well as to monitor the safety performance and especially the health of the machines in a virtual network environment. and that, for this to happen, the group exercised its technical skills strongly and promptly conducted a study on the use of network monitoring and security tools, which were addressed throughout the cycle of this work, in an analysis conducted in detail in a simulated network environment.

Initially and based on classroom studies and in accordance with the brainstorming performed in accordance with the group's consensus, the criteria, metrics and all try-outs performed for the evaluation of software in a simulated environment were defined. of networks.

Such analyzes are intended to effectively demonstrate the use and assignment of security tools and detectors, emphasizing them and placing them in a scenario previously defined by the group, specifically addressing fault monitoring, detection, accurate identification of intrusion, managing the performance of the entire network and also simulating a virtual trap for a potential attacker, all of which are assigned to four free and open source tools to put it this way: Pfsense – Snort – Honeypot – Zabbix, tools which will be responsible for demonstrating how we will address some evaluation criteria, specifically regarding the installation of tools, the configuration of hosts and services, and presenting through them the ability to generate clear and objective information. as well as all the efficiency and effectiveness that covers correct management in the security on a computer network.

Keywords: *Implementation; Safety; Networks; Computers.*

LISTA DE FIGURAS

Figura 1	Ilustração Princípios da Segurança	22
Figura 2	Operação do <i>DHCP</i>	36
Figura 3	Resolução <i>DNS</i>	37
Figura 4	Encaminhamento de Porta	39
Figura 5	Balanceamento de carga	40
Figura 6	Tipos de intruso	47
Figura 7	Localização dos Sistemas de Detecção de Intrusão	49
Figura 8	Funções dos <i>NIDS</i>	52
Figura 9	Detecção de intrusão baseado em assinatura	55
Figura 10	Detecção de intrusão baseado em anomalias	57
Figura 11	Modelo de Gerenciamento de Redes de Computadores	65
Figura 12	Árvore <i>MIB</i>	66
Figura 13	Esquema simplificado de configuração do <i>Zabbix</i>	69
Figura 14	Arquitetura <i>Zabbix</i>	70
Figura 15	Arquitetura <i>Zabbix</i> (Com Sistema <i>Firewall</i>)	71
Figura 16	Modelo de utilização do <i>proxy Zabbix</i>	74
Figura 17	Interface <i>Web</i> do servidor <i>Zabbix</i>	75
Figura 18	Topologia <i>Honeypots</i>	80
Figura 19	Instalação <i>Pfsense</i>	98
Figura 20	Interface <i>Pfsense</i> no servidor	99
Figura 21	Interface <i>Pfsense Web</i>	99
Figura 22	Interface <i>Pfsense Web (Firewall)</i>	100
Figura 23	Interface <i>Pfsense Web (System)</i>	101
Figura 24	Interface <i>Pfsense Web (Snort)</i>	101
Figura 25	Interface <i>Pfsense Web (Services)</i>	102
Figura 26	Configuração <i>Snort</i>	103
Figura 27	Configuração <i>Snort</i>	103
Figura 28	Configuração <i>Snort</i> (Portas)	104
Figura 29	Configuração <i>Snort</i>	104
Figura 30	Ativação do <i>Snort</i>	105
Figura 31	Configuração <i>Snort LAN</i>	105
Figura 32	Regras de Detecção	106
Figura 33	Aplicações Detectadas	106
Figura 34	Configuração <i>Mysql-server</i>	107
Figura 35	<i>Database Zabbix</i> do <i>Mysql</i>	109
Figura 36	Interface <i>Web Zabbix</i>	110
Figura 37	Interface <i>Web Zabbix</i>	110
Figura 38	Endereço <i>IP</i> do servidor	111
Figura 39	Servidor <i>Zabbix</i>	111
Figura 40	<i>Host Zabbix Server</i>	112
Figura 41	Configuração > <i>hosts</i>	112
Figura 42	OS Monitorado	113
Figura 43	Modificação de <i>hosts</i>	113
Figura 44	Ativação de <i>Triggers</i>	114
Figura 45	Ativação de <i>Triggers</i>	114
Figura 46	Configuração	115
Figura 47	<i>CPU</i> Gráficos	115

Figura 48	Gráficos adicionados	116
Figura 49	<i>Trigger</i>	116
Figura 50	Nova <i>Trigger</i>	117
Figura 51	Configuração de <i>e-mail</i> do administrador	118
Figura 52	<i>Trigger</i> desejada	118
Figura 53	Detalhes da operação	118
Figura 54	<i>Honeypots</i>	120
Figura 55	<i>IP</i> no <i>honeypot</i>	120
Figura 56	Topologia do ambiente de redes simulado e seguro	122

LISTA DE GRÁFICOS

Gráfico 1	Uso da <i>internet</i>	83
Gráfico 2	Conhecimento sobre os riscos de uso da <i>internet</i>	83
Gráfico 3	Uso de computador na residência	84
Gráfico 4	Preocupação com a segurança em uma rede de computadores	84
Gráfico 5	Conhecimento técnico da rede na residência	85
Gráfico 6	Uso de ferramentas para a segurança do dispositivo	85
Gráfico 7	Problemas de instrução na rede com vírus e roubo de dados	86
Gráfico 8	Investimento em um sistema de segurança da rede corporativa	86
Gráfico 9	Viabilidade do projeto	87
Gráfico 10	Armazenamento de arquivos	87
Gráfico 11	<i>Internet</i> integrada na empresa	88
Gráfico 12	Custo da implementação	88

LISTA DE TABELAS

Tabela 1	Estatística de incidentes reportados	25
Tabela 2	Padrão de endereçamento na <i>Internet (IPv4)</i>	38
Tabela 3	Comparação de <i>honeypot</i> de baixa, média e alta interatividade	78
Tabela 4	Cronograma de Gerenciamento	81
Tabela 5	Configuração das máquinas a se utilizar	82

SUMÁRIO

1 INTRODUÇÃO	17
1.1 OBJETIVO	18
1.1.1 Objetivos gerais:	18
1.1.2 Objetivos específicos:	19
1.2 JUSTIFICATIVA	19
2 FUNDAMENTAÇÃO TEÓRICA	20
2.1 SEGURANÇA DA INFORMAÇÃO	20
2.1.1 Atacantes, alvos e motivações	23
2.1.1.1 Atacantes	23
2.1.1.2 Alvos	24
2.1.1.3 Motivações	25
2.1.2 Ferramentas e tipos de ataques	25
2.1.3 <i>DoS (Denial of Service)</i>	26
2.1.3.1 Formas de ataques <i>DoS</i>	27
2.1.3.2 Tipos de ataques <i>DoS</i>	28
2.1.4 Engenharia Social	29
2.1.5 <i>Exploit</i>	29
2.1.6 <i>Phishing</i>	29
2.1.7 <i>Backdoor</i>	30
2.1.8 <i>Sniffer</i>	30
2.1.9 <i>Spoofing</i>	30
2.1.10 <i>Brute Force</i>	31
2.1.11 <i>Bot</i>	31
2.1.12 <i>Malware</i>	31
2.1.13 Vulnerabilidades	31
2.2 CONCEITUAÇÃO DE FIREWALL	32
2.2.1 Tipos de <i>firewall</i>	34
2.2.2 Arquiteturas de implantação de <i>firewall</i>	35
2.2.3 <i>DHCP (Dynamic Host Configuration Protocol)</i>	36
2.2.4 <i>DNS (Domain Name System)</i>	37
2.2.5 <i>NAT (Network Address Translation)</i>	37
2.2.6 <i>Port Forwarding</i> (Encaminhamento de Porta)	38
2.2.7 <i>Load Balancing</i> (Balanceamento de Carga)	39
2.2.8 Apresentação do <i>firewall PfSense</i>	40
2.3 DETECÇÃO DE INTRUSÃO	41
2.3.1 Sistemas de detecção de intrusão	41
2.3.2 Arquitetura	42
2.3.3 Estratégias de monitoramento	42
2.3.4 Disposição de sistema de monitoramento	44
2.3.5 <i>SDI</i> com monitoramento centralizado	44
2.3.6 <i>SDI</i> com monitoramento distribuído	44
2.3.7 <i>SDI</i> com monitoramento híbrido	45
2.3.8 Sistemas de detecção de intrusão distribuídos	45
2.3.9 Conceito de detecção de intrusão em redes de computadores	46
2.3.10 Intrusão em rede	46
2.3.11 Intrusos	47
2.3.12 Falsos positivos e falsos negativos	48

2.3.13	Análise e comportamento do fluxo de dados na rede	48
2.3.14	Localização de um <i>IDS</i> na rede	49
2.3.15	Tipos de detecção de intrusão segundo a arquitetura	50
2.3.16	Detectores de intrusão em redes de computadores	51
2.3.17	Componentes do <i>NIDS</i>	52
2.3.18	Classificação dos métodos de detecção	53
2.3.19	<i>SDI</i> baseado em assinaturas	54
2.3.20	<i>SDI</i> baseado em anomalias	56
2.3.21	Ferramenta de detecção de intrusão	58
2.3.21.1	Apresentação do <i>Snort</i>	58
2.3.21.2	História do <i>Snort</i>	58
2.3.21.3	Ambiente e mecanismos de detecção	59
2.3.21.4	Regras	60
2.3.21.5	Requisitos e instalação do <i>Snort</i>	62
2.4	GERENCIAMENTO DE REDES	63
2.4.1	Gerenciamento de segurança	67
2.4.2	Gerenciamento de desempenho	68
2.4.3	Ferramenta de gerenciamento	69
2.4.3.1	Apresentação do <i>Zabbix</i>	69
2.4.3.2	Características principais do <i>Zabbix</i>	69
2.4.3.3	Gerente <i>Zabbix</i>	72
2.4.3.4	Agente <i>Zabbix</i>	73
2.4.3.5	Tipos de verificação	73
2.4.3.6	<i>Proxy Zabbix</i>	73
2.4.3.7	Interface <i>web</i>	74
2.4.3.8	Gestão do desempenho	75
2.5	HONEYPOTS	75
2.5.1	Abrangência, vantagens e desvantagens	77
2.5.2	Classificação por meio de níveis de interatividade	78
2.5.3	Apresentação do <i>HoneyD</i>	79
3	MÉTODO E PROCEDIMENTOS	80
3.1	CRONOGRAMA	80
3.2	CUSTOS	81
3.3	TIPO DE PESQUISA	82
3.4	CARÁTER DA PESQUISA	82
3.5	MÉTODO DE ABORDAGEM	82
3.6	DADOS DA PESQUISA DE CAMPO	83
4	DESENVOLVIMENTO	89
4.1	IMPLEMENTAÇÃO	89
4.2	PENTESTING	93
4.3	PORTMAPPING COM NMAP	94
4.4	ATAQUE DoS EXTERNO E INTERNO	94
4.5	INVASÃO POR SSH VIA BRUTE-FORCE	95
4.6	INVASÃO VIA CONEXÃO TCP REVERSA	96
4.7	INSTALAÇÃO DOS SERVIÇOS	97
4.7.1	Instalação do <i>Pfsense</i>	97
4.7.2	Instalação do <i>Snort</i>	100
4.7.3	Instalação do <i>Zabbix</i>	106
4.7.4	Instalação do <i>HoneyD</i>	119
4.8	A TOPOLOGIA DO AMBIENTE DE REDES SIMULADO E SEGURO	122

4.9 A APRESENTAÇÃO DO PROJETO CONCLUÍDO.....	123
4.9.1 A definição dos critérios	123
4.9.2 As ferramentas disponíveis	123
4.9.3 A proposta da apresentação	125
4.9.4 Projeto e vídeo apresentado em âmbito escolar	126
5 RESULTADOS E DISCUSSÃO	127
6 CONCLUSÃO	129
7 RECOMENDAÇÕES	130
REFERÊNCIAS	132

1 INTRODUÇÃO

Segurança da informação é o processo de proteger dados de acesso, o uso, a modificação, o comportamento e ou a divulgação não autorizada e que, com o aumento crescente do uso de dispositivos eletrônicos em nossas vidas pessoais assim como nas empresas, a possibilidade de brechas de segurança e seus maiores impactos são crescentes nos dias atuais.

O roubo de identidade pessoal, a informação de cartão de crédito e outros dados importantes usando nomes de usuários e senhas *hackeadas* se tornaram comum e que, além disso, o roubo de dados confidenciais de empresas pode levar a prejuízos financeiros imensuráveis para um acionista e empreendedores, sejam eles de empresa de pequeno, de médio ou de grande porte.

Na mesma frequência que se aumenta a confiança das organizações em informações providas por sistemas computacionais, sendo indiscutível o ganho que organizações têm com o crescente uso de computadores, infelizmente tem se observado também uma frequência crescente no surgimento de vulnerabilidades nos diversos sistemas operativos e disponíveis no mercado.

A implementação de diretrizes para tentar diminuir tais transtornos também tenta evoluir de acordo com as necessidades de se alcançar resultados satisfatórios em relação à segurança da informação, sendo indiscutível o ganho que organizações terão com um crescente uso de computadores em relação ao acesso a redes de computadores, *internet* e seus diversos recursos. Porém, é extremamente insensato entrar neste novo "território" tecnológico desprovido de regulamentação adequada sem entender os riscos, sem formular uma política de segurança adequada e sem definir procedimentos para proteger informações importantes de sua organização.

Nos dias atuais, empresas dependem cada vez mais dos sistemas de informação e da *internet* para fazer e executar seus negócios, não podendo se dar ao luxo de sofrer interrupções ou sanções em suas operações, pois um mero incidente de segurança poderia impactar direta e negativamente suas receitas, afetando ao mesmo tempo a confiança de seus clientes e o relacionamento com sua rede de parceiros e fornecedores.

A incidência de segurança está diretamente relacionada com prejuízos financeiros, sejam eles devidos à parada de um sistema por conta de um vírus, ao furto de uma informação confidencial ou à perda de uma informação importante. Estima-se que em um passado não muito distante, *worms* e vírus que atingiram grandes proporções de propagação, como por exemplo, *My Doom*, *Slammer*, *Nimda*, tenham ocasionado prejuízos da ordem de bilhões de dólares no mundo todo.

Em última instância, um incidente pode impedir, direta ou indiretamente, a organização de cumprir sua missão e principalmente de gerar valor para o acionista e ou ao empreendedor. Essa perspectiva traz a segurança da informação para um novo patamar, não apenas relacionada com a esfera da tecnologia e das ferramentas necessárias para proteger a informação, mas também como um dos pilares de suporte à estratégia de negócio de uma corporação. A gestão da segurança assume, então, um novo significado, pois passa a levar em consideração os elementos estratégicos de uma organização e evolui para a extensão da prática de gestão de riscos do negócio.

Na contramão, com um único pensamento, em sanar e ou diminuir tais vulnerabilidades, sugere-se então fazer o uso dos atributos técnicos que um sistema de segurança oferece aos sistemas operativos, delegando ao mesmo a função única e exclusiva de proteção à rede e aos dados em um ambiente local ou corporativo, servindo até mesmo como parâmetro de incentivo à implementação de um sistema eficaz de segurança em uma infraestrutura local em redes de computadores.

1.1 OBJETIVO

1.1.1 Objetivos gerais:

Prover e simular um ambiente local, destacando e utilizando um sistema de segurança integrado e eficaz através de *softwares opensource* disponíveis e atuais.

1.1.2 Objetivos específicos:

- Implementar um *firewall* customizado (*PfSense*);
- Implementar um *IDS/IPS* (*Snort*);
- Implementar um serviço de monitoramento de dados e pacotes (*Zabbix*);
- Implementar um *Honeypot* (*HoneyD*);
- Apresentar e simular uma rede vulnerável e uma rede segura e protegida.

1.2 JUSTIFICATIVA

Crê-se na prerrogativa de que, uma vez que se possui acesso à *internet* ou até mesmo à uma *intranet*, a empresa apresenta vulnerabilidades e exposições a riscos externos e internos, sendo muitas vezes devido até mesmo ao seu mau uso atrelados à falta de conhecimentos e treinamentos adequados de seus colaboradores no que tange em específico, o assunto segurança em redes de computadores, gerando-se então consequências e problemas devastadores à sua rede e muitas vezes comprometendo até mesmo o escopo de seu próprio negócio, seja na alteração de desempenho em suas funções ou meramente na deficiência operacional de sua rede.

A exemplo de deficiências em segurança de rede, há a e exposição a riscos, tais como infecções por *malware*, ataques de *spammers*, *crackers* que podem ocasionar abertura de brechas de segurança, permitindo acessos internos e externos, bem como diversas anomalias já identificadas, demonstrando-se então haver por parte das empresas, uma imensa preocupação com a segurança de seus dados e de suas informações, estas muitas vezes sigilosas e atreladas ao *know-how* do seu negócio.

A evolução e atualização constante destas ameaças forçam os responsáveis de tecnologia da informação buscarem soluções ágeis, proativas e eficazes para preencherem estas lacunas com o crescente déficit de segurança virtual. Mesmo já comprovada, a segurança dos serviços de distribuições diversas exige cuidado e uma perícia apuradíssima em toda a sua configuração durante a implementação de um sistema de segurança.

O proposto pelo grupo é de que será realizado uma integração dos serviços/*softwares*, integrando-os e configurando-os adequadamente de acordo com as ferramentas a se utilizar, elencando-as de forma focada em um sistema de

segurança robusto e totalmente integrado à rede, demonstrando de antemão alguns tipos de ataques cibernéticos e comprovando na prática que existem falhas e vulnerabilidades em uma determinada rede.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 SEGURANÇA DA INFORMAÇÃO

Com a dependência do negócio aos sistemas de informação e o surgimento de novas tecnologias e formas de trabalho, como o comércio eletrônico, as redes virtuais privadas e os funcionários móveis, as empresas começaram a despertar para a necessidade de segurança, uma vez que se tornaram vulneráveis a um número maior de ameaças.

As redes de computadores, e conseqüentemente a *internet* mudaram as formas como se usam sistemas de informação. As possibilidades e oportunidades de utilização são muito mais amplas que em sistemas fechados, assim como os riscos à privacidade e integridade da informação. Portanto, é muito importante que mecanismos de segurança de sistemas de informação sejam projetados de maneira a prevenir acessos não autorizados aos recursos e dados destes sistemas (Laureano, 2004).

A segurança da informação é a proteção dos sistemas de informação contra a negação de serviço a usuários autorizados, assim como contra a intrusão, e a modificação não-autorizada de dados ou informações, armazenados, em processamento ou em trânsito, abrangendo a segurança dos recursos humanos, da documentação e do material, das áreas e instalações das comunicações e computacional, assim como as destinadas a prevenir, detectar, deter e documentar eventuais ameaças a seu desenvolvimento (NBR 17999, 2003; Dias, 2000; Wadlow, 2000; Krause e Tipton, 1999).

Segurança é a base para dar às empresas a possibilidade e a liberdade necessária para a criação de novas oportunidades de negócio. É evidente que os negócios estão cada vez mais dependentes das tecnologias e estas precisam estar de tal forma a proporcionar confidencialidade, integridade e disponibilidade – que conforme (NBR 17999, 2003; Krause e Tipton, 1999;

Albuquerque e Ribeiro, 2002), são os princípios básicos para garantir a segurança da informação – das informações:

- Confidencialidade: A informação somente pode ser acessada por pessoas explicitamente autorizadas; É a proteção de sistemas de informação para impedir que pessoas não autorizadas tenham acesso ao mesmo. O aspecto mais importante deste item é garantir a identificação e autenticação das partes envolvidas;
- Disponibilidade: A informação ou sistema de computador deve estar disponível no momento em que a mesma for necessária;
- Integridade: A informação deve ser retornada em sua forma original no momento em que foi armazenada; É a proteção dos dados ou informações contra modificações intencionais ou acidentais não-autorizadas. É a proteção dos dados ou informações contra modificações intencionais ou acidentais não-autorizadas.

O item integridade não pode ser confundido com confiabilidade do conteúdo (seu significado) da informação. Uma informação pode ser imprecisa, mas deve permanecer íntegra (não sofrer alterações por pessoas não autorizadas).

A segurança visa também aumentar a produtividade dos usuários através de um ambiente mais organizado, proporcionando maior controle sobre os recursos de informática, viabilizando até o uso de aplicações de missão crítica.

A combinação em proporções apropriadas dos itens confidencialidade, disponibilidade e integridade facilitam o suporte para que as empresas alcancem os seus objetivos, pois seus sistemas de informação serão mais confiáveis.

Outros autores (Dias, 2000; Wadlow, 2000; Shirey, 2000; Krause e Tipton, 1999; Albuquerque e Ribeiro, 2002; Sêmola, 2003; Sandhu e Samarati, 1994) defendem que para uma informação ser considerada segura, o sistema que o administra ainda deve respeitar:

- Autenticidade: Garante que a informação ou o usuário da mesma é autêntico; Atesta com exatidão, a origem do dado ou informação;
- Não repúdio: Não é possível negar (no sentido de dizer que não foi feito) uma operação ou serviço que modificou ou criou uma informação; Não é possível negar o envio ou recepção de uma informação ou dado;

- Legalidade: Garante a legalidade (jurídica) da informação; Aderência de um sistema à legislação; Característica das informações que possuem valor legal dentro de um processo de comunicação, onde todos os ativos estão de acordo com as cláusulas contratuais pactuadas ou a legislação política institucional, nacional ou internacional vigentes;
- Privacidade: Foge do aspecto de confidencialidade, pois uma informação pode ser considerada confidencial, mas não privada. Uma informação privada deve ser vista / lida / alterada somente pelo seu dono. Garante ainda, que a informação não será disponibilizada para outras pessoas (neste é caso é atribuído o caráter de confidencialidade a informação); É a capacidade de um usuário realizar ações em um sistema sem que seja identificado;
- Auditoria: Rastreabilidade dos diversos passos que um negócio ou processo realizou ou que uma informação foi submetida, identificando os participantes, os locais e horários de cada etapa. Auditoria em *software* significa uma parte da aplicação, ou conjunto de funções do sistema, que viabiliza uma auditoria; Consiste no exame do histórico dos eventos dentro de um sistema para determinar quando e onde ocorreu uma violação de segurança.

Em (Stoneburner, 2001) é sugerido que a segurança somente é obtida através da relação e correta implementação de 4 princípios da segurança: confidencialidade, integridade, disponibilidade e auditoria. A próxima figura ilustra a relação dos princípios para a obtenção da segurança da informação.

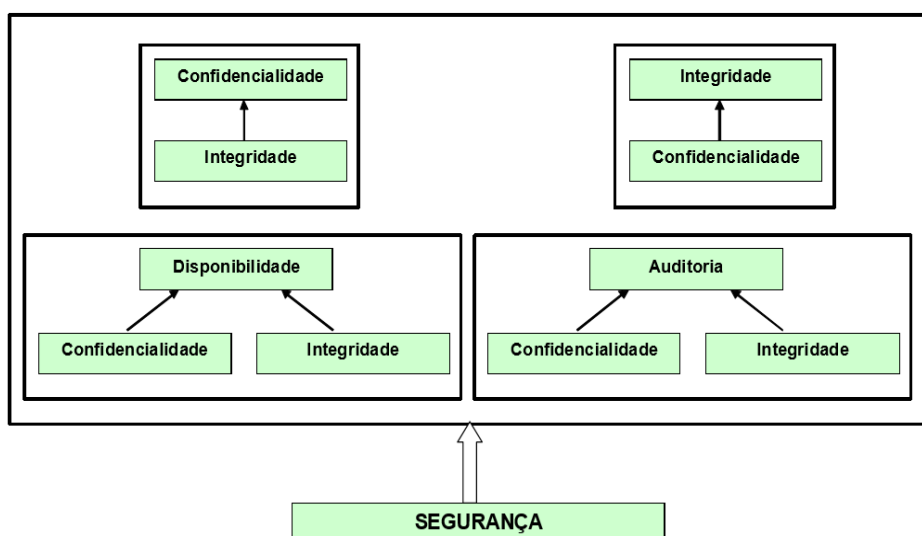


Figura 1: Ilustração Princípios da Segurança

Fonte: Autoria do grupo

A confidencialidade é dependente da integridade, pois se a integridade de um sistema for perdida, os mecanismos que controlam a confidencialidade não são mais confiáveis.

A integridade é dependente da confidencialidade, pois se alguma informação confidencial for perdida (senha de administrador do sistema, por exemplo) os mecanismos de integridade podem ser desativados.

Auditoria e disponibilidade são dependentes da integridade e confidencialidade, pois estes mecanismos garantem a auditoria do sistema (registros históricos) e a disponibilidade do sistema (nenhum serviço ou informação vital é alterado).

2.1.1 Atacantes, alvos e motivações

Com a informação cada vez mais valiosa tanto no meio da *internet* ou mesmo na *intranet* das empresas, os atacantes cada vez mais vêm evoluindo suas técnicas e disseminando métodos de ataques. Os ataques acontecem com alvos variados que são escolhidos de acordo com cada motivação dos atacantes. Os ataques podem levar a sérios prejuízos e até à estagnação de um negócio.

2.1.1.1 Atacantes

Os atacantes são usuários mal ou bem-intencionados que utilizam seu alto grau de conhecimento para realizar ataques a uma rede de computadores, visando comprometer a segurança da informação. Esses usuários, em sua maioria, são munidos de grande conhecimento de técnicas de programação, análise de sistemas e especialistas em segurança de redes.

As habilidades dos atacantes mudam à medida que as brechas de segurança são corrigidas, tornando assim os ataques cada vez mais modernos. Uma grande problemática são as ferramentas desenvolvidas pelos atacantes com grandes conhecimentos, automatizando procedimentos de ataques e exploradores de brechas por meio de ferramentas utilizadas por atacantes com nenhum ou pouco conhecimento.

O grupo de atacantes pode ser dividido em dois principais tipos, sendo que alguns realizam os ataques somente para descobrir e reportarem brechas de segurança em redes ou *softwares*, outros para comprometer a rede computacional, causando estragos inestimáveis. A seguir segue alguns tipos e definições de atacantes:

- *Script Kiddies*: Com o intuito de afetar o maior número de sistemas computacionais possíveis, esse tipo de atacantes não se importa se o que ele está afetando é uma rede corporativa ou um simples computador pessoal. Muitas vezes com pouco conhecimento, utilizam *scripts* e ferramentas prontas para realizar ataques em massa. Esse tipo de atacantes, também conhecido como *lammer*, é o mais comum na rede hoje, sendo a maioria dos ataques, *scans* e sondagens realizadas por eles;
- *Advanced Blackhats*: Mesmo em menor número são considerados os mais perigosos do grupo de atacantes. Procuram realizar ataques em sistemas de alto valor causando perda financeira ou de informações valiosas, como o terrorismo patrocinado por estados. Possuem uma habilidade computacional avançada, podendo atacar sistemas sem que o administrador saiba que está comprometido;
- *Advanced Whitehats*: Conhecidos como atacantes do bem, realizando ataques em prol do conhecimento. Os *Whitehats* são responsáveis por reportarem vulnerabilidades no sistema para grandes empresas desenvolvedoras de *softwares*. Em prol do conhecimento a todos, não realizam ataques para prejudicar corporações ou governos.

2.1.1.2 Alvos

Muitas pessoas têm uma ideia errada de que está fora dos alvos de qualquer possível atacante, devido achar que as informações contidas em seu computador não são importantes. Porém os atacantes não buscam somente informações, seu computador pessoal poderá ser invadido e o atacante utilizar seus recursos como conexão com a *internet*, disco rígido, memória e processamento para realizar ataques a terceiros.

Todo e qualquer dispositivo como *smartphones*, *notebooks*, *tablets* e computadores pessoais que estejam conectados em uma rede, podem receber um ataque.

2.1.1.3 Motivações

A motivação de se realizar um ataque ou se tornar um atacante possui diversos fatores como dinheiro, ego, diversão, ideologia, *status* e/ou inclusão em um grupo social. A obtenção desses fatores pode vir de várias maneiras como a invasão de uma rede corporativa, roubo de números de cartões de créditos ou possuir controle de milhares de computadores.

Tendo seu alvo e motivação escolhido, os atacantes gozam de diversos métodos e ferramentas para realizarem os ataques. Os métodos utilizados variam de acordo com a escolha feita anteriormente ao ataque, onde o atacante irá relacionar o melhor método e/ou ferramenta irá utilizar para atingir seu objetivo.

2.1.2 Ferramentas e tipos de ataques

Ao longo dos anos várias ferramentas e métodos de ataques têm sido desenvolvidos. Abaixo, uma lista das principais ferramentas e métodos. Desde o ano 1999, o CERT.br vem analisando as atividades na *internet* brasileira. Na tabela 1 são exibidos os ataques mais reportados durante o ano de 2012.

Tabela 1: Estatística de incidentes reportados
Totais mensais e Trimestral Classificados por Tipo de Ataque

Mês	Total	worm (%)	dos (%)	invasão (%)	web (%)	scan (%)	fraude (%)	outros (%)							
jan	27148	6830	25	7	0	603	2	2137	7	8478	31	4096	15	4997	18
fev	25266	4404	17	34	0	452	1	2211	8	8523	33	4183	16	5459	21
mar	34796	2380	6	22	0	559	1	3508	10	10616	30	5126	14	12585	36
abr	35342	3171	8	9	0	100	0	2111	5	9557	27	7923	22	12471	35
mai	40965	3242	7	11	0	413	1	3532	8	13339	32	8557	20	11871	28
jun	37806	2702	7	9	0	619	1	3124	8	16646	44	5653	14	9053	23
jul	42104	2922	6	17	0	1320	3	2899	6	20638	49	6580	15	7728	18
ago	60018	2867	4	40	0	659	1	1717	2	41741	69	5582	9	7412	12
set	53501	3157	5	9	0	1043	1	1359	2	35571	66	5730	10	6632	12
out	39326	3380	8	9	0	754	1	1186	3	23229	59	6442	16	4326	11
nov	40751	1858	4	125	0	716	1	1116	2	28065	68	5268	12	3603	8
dez	29006	1553	5	17	0	577	1	657	2	16095	55	4421	15	5686	19
Total	466029	38466	8	309	0	7815	1	25557	5	232498	49	69561	14	91823	19

Fonte: cert.br (2012)

- Worm: Notificações de atividades maliciosas relacionadas com o processo automatizado de propagação de códigos maliciosos na rede;
- DoS (Denial of Service): Notificações de ataques de negação de serviço, onde o atacante utiliza um computador ou um conjunto de computadores para tirar de operação um serviço, computador ou rede;
- Invasão: Um ataque bem-sucedido que resulte no acesso não autorizado a um computador ou rede;
- Web: Um caso particular de ataque visando especificamente o comprometimento de servidores *web* ou desfigurações de páginas na *internet*;
- Scan: Notificações de varreduras em redes de computadores, com o intuito de identificar quais computadores estão ativos e quais serviços estão sendo disponibilizados por eles. É amplamente utilizado por atacantes para identificar potenciais alvos, pois permite associar possíveis vulnerabilidades aos serviços habilitados em um computador;
- Fraude: Segundo Houaiss, é "qualquer ato ardiloso, enganoso, de má-fé, com intuito de lesar ou ludibriar outrem, ou de não cumprir determinado dever; logro". Esta categoria engloba as notificações de tentativas de fraudes, ou seja, de incidentes em que ocorre uma tentativa de obter vantagem;
- Outros: Notificações de incidentes que não se enquadram nas categorias anteriores.

2.1.3 DoS (Denial of Service)

Criado em meados da década de 90, o ataque *DoS (Denial Of Service)* têm se popularizado nos últimos anos, chamando atenção da mídia e dos profissionais de segurança. Ultimamente empregado como forma de protesto virtual a várias corporações e órgãos governamentais, o ataque *DoS* vem ganhando força, já que ferramentas podem ser facilmente encontradas em diversos fóruns ou *sites* especializados.

Grandes corporações, como *eBay*, *Yahoo*, *Amazon* e *CNN*, amargam perdas devido aos ataques *DoS*, cujo no ano de 2000, receberam grandes ataques do tipo *DoS* que resultou na perda de milhares de dólares. Em março de 2013, um ataque *DoS* direcionado a uma empresa especializada em *AntiSpam* foi noticiado

pela mídia como o maior ataque realizado na história, esse ataque chegou a causar lentidão em toda a *internet* mundial [JUNQUEIRA, 2013].

O ataque *DoS* tem por objetivo indisponibilizar ou causar lentidão a um serviço *web* enviando várias requisições ilegítimas. A realização é feita sem que uma invasão ou infecção ao servidor seja necessária, apenas utilizando algumas características de alguns protocolos, como o *TCP/IP (Transmission Control Protocol/Internet Protocol)*.

O efeito é obtido por meio de várias requisições realizadas pelo atacante a um servidor conectado à *internet* com requisições inúteis, fazendo com que acarrete em uma sobrecarga do servidor que por sua vez não consegue responder a todos, negando a disponibilidade do serviço ou aproveitando-se de falhas ou vulnerabilidades presentes na máquina vítima do ataque. Para que seja possível realizar um ataque *DoS* é necessário ter um computador com alto poder de processamento e uma boa banda de *internet* disponível ou diversos computadores com recursos variados que se concentram para enviar a uma mesma vítima. O ataque utilizando diversos computadores é muito mais perigoso, pelo ataque partir de diversas origens e ser potencializado, podendo ser controlado por um único atacante.

2.1.3.1 Formas de ataques *DoS*

As formas de ataques *DoS* são variadas e podem causar desde consumo total da banda da *internet* até consumo de processamento do servidor da vítima, chegando sempre ao mesmo objetivo, são elas:

- *SYN Flooding*: Uma forma bem comum de ataque *DoS* é com a utilização de pacotes *TCP/SYN* explorando a abertura de conexões *TCP* assim inundando o serviço. O ataque funciona de forma que o cliente envia um pacote *SYN* contendo parâmetros para que o servidor entenda como uma sequência de acesso e retorne um pacote *TCP SYN/ACK*, informando ao cliente que o pacote foi aceito. Por sua vez, o cliente envia um pacote *ACK* para completar a abertura de conexão (DUARTE, 2013). Utilizando a técnica de *IP Spoofing*, o *IP* do atacante é mascarado por um outro qualquer, dessa maneira, a vítima ao responder pelo ataque é direcionada para um endereço falso e fica

aguardando a uma resposta de uma requisição que não existe. Com essa forma de ataque, os recursos do servidor passam a ser utilizados, como memória e processamento, negando a novas requisições realizadas para o servidor;

- *Fraggle Attack*: Outra forma comum de ataque, que utiliza pacotes *UDP* para entupir o servidor de requisições inválidas. Ao contrário do ataque *SYN Flooding*, o *Fraggle Attack* não aguarda resposta do servidor, pois utiliza pacotes *UDP*. O ataque consiste em enviar diversos pacotes *UDP* ao endereço de *broadcast* da rede, com cabeçalho alterado levando o *IP* da vítima como a origem do pacote, para todas as portas do servidor;
- *Smurf Attack*: Esse método utiliza pacotes *ICMP* para saturar uma conexão da *internet* de baixa ou alta velocidade. Assim como o *Fraggle Attack*, são enviadas diversas requisições *ICMP Echo Request (ping)*, com o cabeçalho alterado, destinado ao *broadcast* da rede fazendo com que sejam gastos todos seus recursos para responder com *ICMP Echo Reply (ping)*, indisponibilizando os serviços contidos naquela rede.

2.1.3.2 Tipos de ataques *DoS*

As variantes de ataques *DoS* se diferenciam de acordo com a quantidade de computadores utilizados para realizarem os ataques. Geralmente os computadores utilizados para o ataque são infectados por ferramentas maliciosas sem que o proprietário perceba, denominadas de zumbis, onde o controle passa a ser do atacante, que pode direcionar o ataque com um grande poder para qualquer vítima. A seguir, é explicada a diferença de cada tipo de ataque:

- *DDoS*: É uma evolução do ataque *DoS* e tem o mesmo objetivo, só que em grandes dimensões. Para realizar o ataque, o atacante utiliza várias máquinas clientes infectadas com *malware (bots)* para fazer ataques *DoS* simultâneos. Por muitas vezes os *bots* são computadores pessoais, alvo preferido dos atacantes, devido à melhoria da velocidade da conexão com a *internet* residencial;
- *DRDOS*: É uma evolução do ataque *DDoS*, que além de utilizar máquinas infectadas para realizar ataques, alteram o cabeçalho do pacote para forjá-lo

com o endereço *IP* de resposta da vítima. Assim o atacante direciona o ataque para máquinas refletora, sem que saibam, respondem o ataque ao endereço de *IP* vítima, inundando sua conexão com requisições inválidas.

Os ataques *DoS* desafiam diretamente a disponibilidade dos servidores, à medida que novas técnicas para bloqueais para o ataque *DoS* e suas variantes são descobertas, novas formas de ataques vão surgindo, desafiando a segurança da informação e requerendo constantes pesquisa e aprendizado com os ataques.

2.1.4 Engenharia Social

É um dos ataques mais utilizados para se conseguir informações sigilosas e importantes, explorando as falhas de segurança dos humanos. Sem que necessite necessariamente de *internet*, uma rede ou se quer de um computador, o ataque utiliza a confiança da pessoa para conseguir seu objetivo. O ataque pode originar-se de um simples bate-papo na *internet* até um encontro em um café.

2.1.5 *Exploit*

O termo *exploit* é utilizado para se referir a pequenos códigos de programas desenvolvidos especialmente para explorar falhas introduzidas em aplicativos por erros involuntários de programação (ALMEIDA, 2013). Criado somente para explorar falhas específicas em *softwares* ou sistemas operacionais, geralmente há um *exploit* diferente para cada tipo de falha que pode ser preparado para atacar local ou remotamente.

Um exemplo de *exploit* bem comum é o *SQL Injection*, que se beneficia de um erro de programação onde os campos que recebem dados inseridos pelos usuários não são validados. O atacante por sua vez, insere códigos *SQL* nesses campos e executa a ação para ter acesso ao banco de dados utilizado pelo *software*.

2.1.6 *Phishing*

O método de *Phishing*, é uma fraude que utiliza a engenharia social para realizar roubos *online*. Este tipo de atividade tem se tornado muito comum na *internet*. O mesmo consiste em *e-mail* falsos, contendo códigos ou *software*

maliciosos, enviados em nomes de grandes corporações ou instituições governamentais com o objetivo de capturar dados de documentos pessoais, conta de banco, senhas, cartão de crédito entre outras informações importante.

2.1.7 *Backdoor*

O método de ataque *backdoor* contamina o computador por meio de um cavalo de Tróia que deixa propositalmente uma porta de rede aberta, assim sendo possível o acesso remoto. Esse tipo de vírus é comumente usado por ataques *DDoS*, onde o atacante infecta a máquina com a possibilidade de utilizar remotamente para um ataque. A infecção de vírus pode vir de várias formas, porém a mais utilizada é por meio de *e-mail* e utilizando páginas *web* falsas.

2.1.8 *Sniffer*

Método utilizado para interceptar e registrar todo o tráfego de dados em uma rede *ethernet* ou *wireless*. Assim que um *sniffer* é executado em uma rede, é realizada a captura do pacote e eventualmente seu conteúdo é decodificado e analisado de acordo com a *request for coment (RCF)* ou alguma outra especificação.

2.1.9 *Spoofing*

- *Arp Spoofing*: O ataque de *Arp Spoofing* consiste em o atacante se passar pela vítima redirecionando o tráfego de rede para sua máquina. Para que isso ocorra é enviada uma mensagem ao roteador da vítima informando que o endereço de *IP* da vítima está atrelado ao seu endereço físico (*MAC*).
- *IP Spoofing*: O ataque *IP Spoofing* trabalha no nível de pacotes onde seu cabeçalho é alterado para que o remetente não seja descoberto. O método consiste em realizar na troca do *IP* de origem do pacote para um falso remetente, como os roteadores não verificam esse endereço, o método se torna eficaz quando utilizado, principalmente por ataques *DoS*.

2.1.10 Brute Force

Um dos métodos mais antigos para se realizar um ataque, consiste em realizar o maior número de combinações possíveis para se obter o resultado esperado, como a descoberta de um usuário e senha. Ainda muito utilizado por atacantes o resultado é obtido por meio de *scripts* ou *softwares* que realizam a varredura de acordo com os parâmetros passados.

2.1.11 Bot

Realizando tarefas automáticas, como manter controle de canais de *IRC* e em sua maioria para realizar ataques *DDoS*, o *BOT* é um computador infectado com uma praga que permite ser acessado remotamente para execução de qualquer tarefa requerida pelo atacante, se tornando uma máquina “zumbi”.

2.1.12 Malware

Segundo o CERT.BR, Códigos maliciosos (*malware*) são programas especificamente desenvolvidos para executar ações danosas e atividades maliciosas em um computador. Algumas das diversas formas como os códigos maliciosos podem infectar ou comprometer um computador. Vírus, *Worm*, *Trojan* ou *Spyware* podem ser considerados *malwares*.

Muitos ataques relacionados a rede de computadores podem ser bloqueados de forma simples e rápida, com a utilização de tecnologias específicas. Uma delas presente tanto no meio corporativo quanto no do usuário doméstico, é chamado de *firewall*, que protege a rede de acordo com as configurações específicas realizadas pelo seu administrador.

2.1.13 Vulnerabilidades

A vulnerabilidade é o fator que, quando não devidamente detectado e corrigido, pode levar à intrusão e ao comprometimento do sistema. De acordo com Campos (2014, p.23), “os ativos de informação, que suportam os processos de negócio, possuem vulnerabilidades. É importante destacar que essas vulnerabilidades estão presentes nos próprios ativos, ou seja, que são inerentes a eles, e não de origem externa”.

2.2 CONCEITUAÇÃO DE FIREWALL

Com a evolução nos sistemas de comunicações, o acesso à informação se torna cada dia mais democrático e universal, e a *internet* tem papel fundamental na evolução do mercado corporativo atual. Com esse amplo acesso a informações, se tornou essencial o desenvolvimento de equipamentos com capacidade de prover a segurança das informações trafegadas pela rede. Esses equipamentos são responsáveis por uma série de competências, como por exemplo, o controle de acessos para evitar acessos nocivos ou não autorizados às informações.

A segurança da informação é regulamentada pelas normas *ISO/IEC 27000* e *ISO/IEC 27001* que consistem em definir um propósito para o desenvolvimento de um Sistema de Gestão e Segurança da Informação (SGSI) nas organizações, algo imprescindível tendo em conta a quantidade de informação produzida atualmente nas grandes corporações (*ISO/IEC 27000, 2013*).

Com esse novo ambiente de desenvolvimento de segurança, novos campos de estudo têm se destacado, como por exemplo, a segurança das redes. Essa área é marcada pela constante evolução, ou seja, é necessário o desenvolvimento de novas técnicas conforme novas formas de ataques são criadas.

Com base nesses argumentos, foram considerados alguns pontos importantes para estudo:

- Entendimento da forma como são constituídas as formas de invasão, que normalmente se dão através da exploração da implantação de novos sistemas corporativos, falhas na implantação de sistemas de segurança e novos sistemas de conectividade;
- A facilidade de acesso à *internet* possibilita a criação de novas formas de ataque, e por consequência a necessidade de desenvolvimento de novas formas de defesa aos mesmos. Pelo fato de um ataque precisar identificar somente uma falha para que possa servir ao seu propósito, a defesa tem de mitigar todas as formas de ataque e falhas no sistema de segurança. Com isso pode ser tomado como base que a defesa de um sistema de informação é muito mais complexa e trabalhosa que um ataque;

- Entendimento das mais variadas formas de ataque a um sistema facilita muito na criação de métodos de proteção aos mesmos. Os ataques a uma rede corporativa podem ter os mais diversos motivos, como por exemplo, interromper serviços da empresa, comprometendo a confiabilidade de dados e programas ou até mesmo a desestabilização de uma rede, com o objetivo de coletar informações que podem ser usadas no futuro para invasões com finalidades mais específicas, como o roubo de informações sensíveis e até mesmo para corromper sistemas vitais à empresa concorrente.

Com base neste contexto, foi desenvolvido uma das formas para mitigar esses ataques, o *firewall*, que realiza controle de acessos devidos a uma determinada rede. É possível interpretar um *firewall* fazendo uma analogia com a forma mais antiga de segurança medieval, criar um fosso ao redor de um castelo e forçar todos que quiserem entrar a passar por uma ponte levadiça, nessa analogia, o *firewall* seria a ponte levadiça, a única porta de entrada de uma rede (TANENBAUM, 2003).

O conceito de *firewall* começou a ser utilizado no final da década de 80, quando somente roteadores separavam pequenas redes corporativas. Desta forma, as redes poderiam instalar seus aplicativos da forma como lhes fosse conveniente sem que as demais redes fossem prejudicadas por lentidões.

Os primeiros *firewalls* a trabalharem com segurança de redes surgiram no início dos anos 90. Consistiam de mecanismos que com pequenos conjuntos de regras como, por exemplo: Alguém da rede A pode fazer acesso à rede B, porém a rede C não pode realizar acessos à rede A e B. Eram mecanismos bastante efetivos, porém extremamente limitados. A segunda geração de *firewalls* utilizava filtros, pacotes e aplicativos, além de trazer uma interface de gerenciamento de regras, um grande salto evolutivo.

Em 1994, a *Check Point* lançou o produto chamado *Firewall-1*, introduzindo uma amigável interface gráfica de gerenciamento, que continha cores, *mouse* e ambiente gráfico *X11* simplificando a instalação e a administração dos *firewalls*.

Atualmente existem várias soluções de *firewalls* muito mais modernas, as principais empresas do ramo são: *Cisco*, *Juniper*, *Check Point* entre outras.

Firewall é uma solução de segurança baseada em *hardware* ou *software*, que a partir de um conjunto de regras ou instruções, analisa o tráfego da

rede para diferenciar operações válidas ou inválidas dentro de uma rede corporativa. Um *firewall* analisa o tráfego de rede entre a *internet* e a rede privada ou entre redes privadas. Com base nessas definições, é possível compreender que o *firewall* é mais que uma simples barreira de proteção contra-ataques, ele pode ser utilizado para proteção dentro de uma rede controlando o tráfego de dados a servidores específicos.

Possuindo diversos recursos, o *firewall* se tornou uma ferramenta essencial para uma rede em geral. Não só defendendo, mas criando certas políticas de segurança e controle de conteúdo como *Network Address Translation (NAT)* / *Port Address Translation (PAT)*, estabelecimento de *Virtual Private Network (VPN)*, entre outros.

Segundo *Chris Roeckl* (ROECKL, 2004), o *firewall* filtra o tráfego trocado entre as redes, reforçando a política de controle de acesso de cada rede. Assegura que apenas o tráfego autorizado passa para dentro e para fora de cada rede ligada. Para evitar o comprometimento, o próprio *firewall* deve ser endurecido contra-ataque. Para permitir a formulação de políticas de segurança e verificação, um *firewall* também deve fornecer monitoramento e registros.

2.2.1 Tipos de *firewall*

Os três principais tipos de *firewall* são:

- *Firewall em nível de pacote*: O filtro de pacotes tem como objetivo permitir ou não a passagem de pacotes pela rede baseando-se em regras pré-definidas. Normalmente estes estão situados em roteadores, representando o ponto de acesso entre duas redes, permitindo que serviços controlem o tráfego, mantendo a rede protegida.

Este é o *firewall* mais implementado atualmente, pois é a proteção básica da rede, ao deixar portas de comunicação nocivas abertas e permitir o tráfego livre de pacotes, a rede fica suscetível a ataques (Camy, 2003).

- *Serviços Proxy*: Os servidores de serviço *proxy* são especializados em aplicações ou programas servidores que executam um *firewall*. Os *proxies* pegam a solicitação e requisição dos usuários para o serviço da *internet*, verificam se as solicitações serão aceitas dentro do conjunto de regras

preestabelecidas e em seguida passam ou não a solicitação adiante para o serviço específico solicitado (Stato Filho, 2009).

Estes servidores estão entre o usuário da rede interna e a *internet* e atuam como o próprio nome diz, como um mediador. É comum utilizarem transparência nesses servidores, como o nome sugere, ele fica transparente para o usuário, sendo imperceptível, porém atuando como filtro de pacotes.

- Circuit-Level Gateway: Este tipo de *firewall* cria um circuito entre o cliente e o servidor e não interpreta o protocolo de aplicação. Atua monitorando o *handshaking* (troca de informações para estabelecimento de comunicação) entre pacotes, objetivando determinar se a sessão é legítima (Stato Filho, 2009).

O principal objetivo de um *firewall* é fazer com que todas as informações trafegadas entre duas redes diferentes passem por ele. Para que isso aconteça, é necessário que haja um estudo sobre a arquitetura da rede que se deseja proteger.

2.2.2 Arquiteturas de implantação de *firewall*

As principais arquiteturas de implantação de *firewalls* são:

- Dual-homed host: Nessa arquitetura o equipamento deve possuir duas interfaces de rede, uma interface que se liga a *LAN*, que representa a rede interna, e uma interface *WAN*, que representa a rede externa, se tornando uma espécie de porta única de saída e entrada da rede. De acordo com os principais pesquisadores, esta arquitetura é ideal para redes com pequeno tráfego de informações para a *internet* e cuja importância não seja vital;
- Screened host: Nesta arquitetura as conexões podem ser abertas da rede interna para a *internet*, bem como das redes externas para a rede interna de forma exclusivamente controlada pelos *bastion hosts*. A filtragem de pacotes acontece no *firewall* que permite apenas poucos tipos de conexões, como por exemplo, consultas de *DNS* (*Domain Name System*). O *bastion host* deve manter um alto nível de segurança pelo fato dele ser o ponto de falha dessa arquitetura. Esta arquitetura é apropriada para redes com poucas conexões vindas da *internet* e quando a rede sendo protegida tem um nível de segurança relativamente alto.

- Screened Subnet: Nessa arquitetura, também conhecida como arquitetura de sub-rede com triagem, é adicionada uma nova rede de perímetro que isola os *bastion hosts*, máquinas vulneráveis na rede. Também são encontrados roteadores de triagem com várias placas de rede. Neste caso, o *bastion host* fica confinado em uma área conhecida como Zona Desmilitarizada (*DMZ*), aumentando o nível de segurança, uma vez que, para ter acesso à rede interna o atacante deverá passar por mais de um processo de filtragem. O *firewall* externo deve permitir que os usuários externos tenham acesso somente à área *DMZ* e o *firewall* interno deve permitir requisições apenas aos usuários da rede interna (Stato Filho, 2009, p.37).

Existem mais possibilidades de arquiteturas e uma flexibilidade no modo de configuração, devem ser avaliados os requisitos de proteção e de orçamento para poder atender as necessidades de proteção da rede em questão.

2.2.3 DHCP (Dynamic Host Configuration Protocol)

O *DHCP* é um servidor de endereços *IP* que ao receber uma solicitação de um dispositivo de rede por um *IP* atribui a este um endereçamento. Cada máquina que é conectada na rede transmite um pacote de *DHCP DISCOVER* para o agente de retransmissão *DHCP* da rede interceptar, ao então o agente envia este pacote em unidifusão ao servidor *DHCP*, possuindo apenas o endereço *IP* do servidor.

O *DHCP* distribui endereços de rede que são atribuídos fixamente ou dinamicamente para os *hosts* e dispositivos de rede (TANENBAUM, 2003, p. 349). A figura 2 mostra este processo *DHCP*.

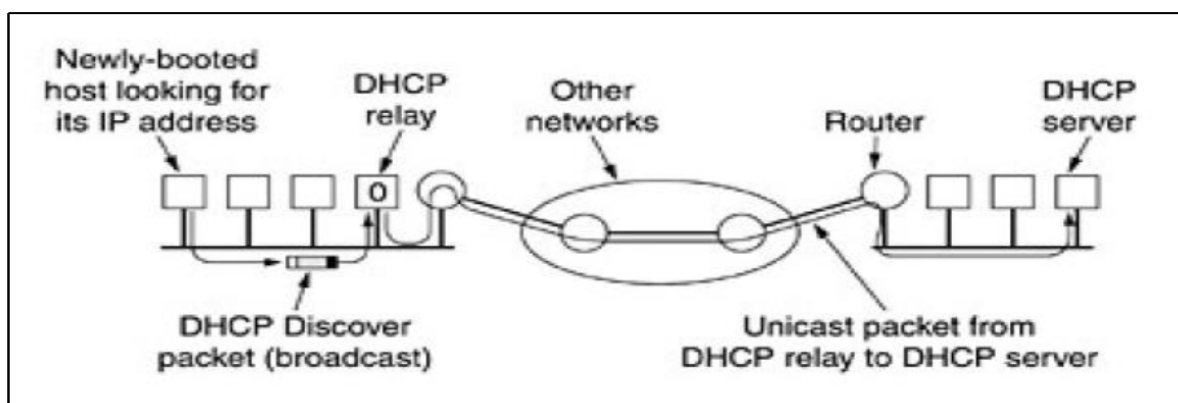


Figura 2: Operação do DHCP
Fonte: TANENBAUM (2003)

2.2.4 DNS (Domain Name System)

O *DNS* consiste na criação de um processo de atribuição de nomes baseados no domínio e em um sistema de bancos de dados distribuídos que implementam esse esquema de nomenclatura organizando e mantendo as informações. É utilizado para mapear nomes de *hosts* e destinos de correios eletrônicos em endereços *IP*, tendo também outras utilizações (TANENBAUM, 2003, p. 439).

O processo do *DNS* consiste em procurar o endereço *IP* correspondente ao nome de domínio do *site* especificado pela requisição (ex.: o endereço *IP* do *site* www.google.com pode ser resolvido pelo *DNS* como 173.194.118.51). A figura 3 traz um exemplo do esquema de resolução *DNS*.

```
Microsoft Windows [versão 10.0.18362.418]
(c) 2019 Microsoft Corporation. Todos os direitos reservados.

C:\WINDOWS\system32>ping www.google.com.br

Disparando www.google.com.br [172.217.30.67] com 32 bytes de dados:
Resposta de 172.217.30.67: bytes=32 tempo=5ms TTL=57
Resposta de 172.217.30.67: bytes=32 tempo=4ms TTL=57
Resposta de 172.217.30.67: bytes=32 tempo=6ms TTL=57
Resposta de 172.217.30.67: bytes=32 tempo=5ms TTL=57

Estatísticas do Ping para 172.217.30.67:
    Pacotes: Enviados = 4, Recebidos = 4, Perdidos = 0 (0% de
    perda),
Aproximar um número redondo de vezes em milissegundos:
    Mínimo = 4ms, Máximo = 6ms, Média = 5ms

C:\WINDOWS\system32>
```

Figura 3: Resolução *DNS*

Fonte: Autoria do grupo

2.2.5 NAT (Network Address Translation)

O *NAT* tem como ideia básica a utilização de um único endereço *IP* para determinada empresa trafegar na *internet* (*IP* roteável), possibilitando assim cada computador da rede interna ter um endereço *IP* exclusivo, para roteamento do tráfego interno. Quando um pacote de dados sai da rede interna para trafegar na *internet* é efetuada uma conversão de endereçamento para o endereço único da empresa para acesso à *internet*. Para que esse sistema fosse possível foram separadas três classes 21 de endereços *IP* que podem ser utilizadas para tráfego interno, porém não podem ser utilizadas para o tráfego na *internet* (TANENBAUM, 2003). A tabela 2 apresenta os endereços privativos e suas classes.

Tabela 2: Padrão de endereçamento na *Internet* (IPv4)

		Endereço IP			
Classe A	Início	1	0	0	0
	Fim	126	255	255	254
		rede	host	host	host
	Máscara	255	0	0	0
Classe B	Início	128	0	0	0
	Fim	191	255	255	254
		rede	rede	host	host
	Máscara	255	255	0	0
Classe C	Início	192	0	0	0
	Fim	223	255	255	254
		rede	rede	rede	host
	Máscara	255	255	255	0

Fonte: Teleco (2007)

Outra função do *NAT* muito utilizada pelas empresas provedoras de acesso à *internet* serve como uma alternativa para escassez de endereços *IP* que são destinados para os usuários de *ADSL*. A empresa atribui a seus clientes endereços 10.x.y.z, e então quando os clientes saem da rede do provedor para acessar a *internet* os pacotes passam pelo processo de *NAT* que efetua a conversão dos endereços internos para endereços *IP* roteáveis, e na volta desses pacotes eles sofrem o processo inverso (TANENBAUM, 2003, p. 343-344).

2.2.6 *Port Forwarding* (Encaminhamento de Porta)

O encaminhamento de portas é uma funcionalidade que consiste na associação de números de portas de comunicação a dispositivos da rede local onde estes dispositivos atendem as requisições nestas portas de serviços vindas da *internet*. Quando feitas, essas requisições são transferidas para um endereço *IP* da rede local. Desta forma o dispositivo que é responsável por receber as requisições para essas determinadas portas deve permanecer com um endereço *IP* estático (Intelbras, 2013). A figura 4 ilustra o processo de encaminhamento de porta vindo de um computador externo à rede local.

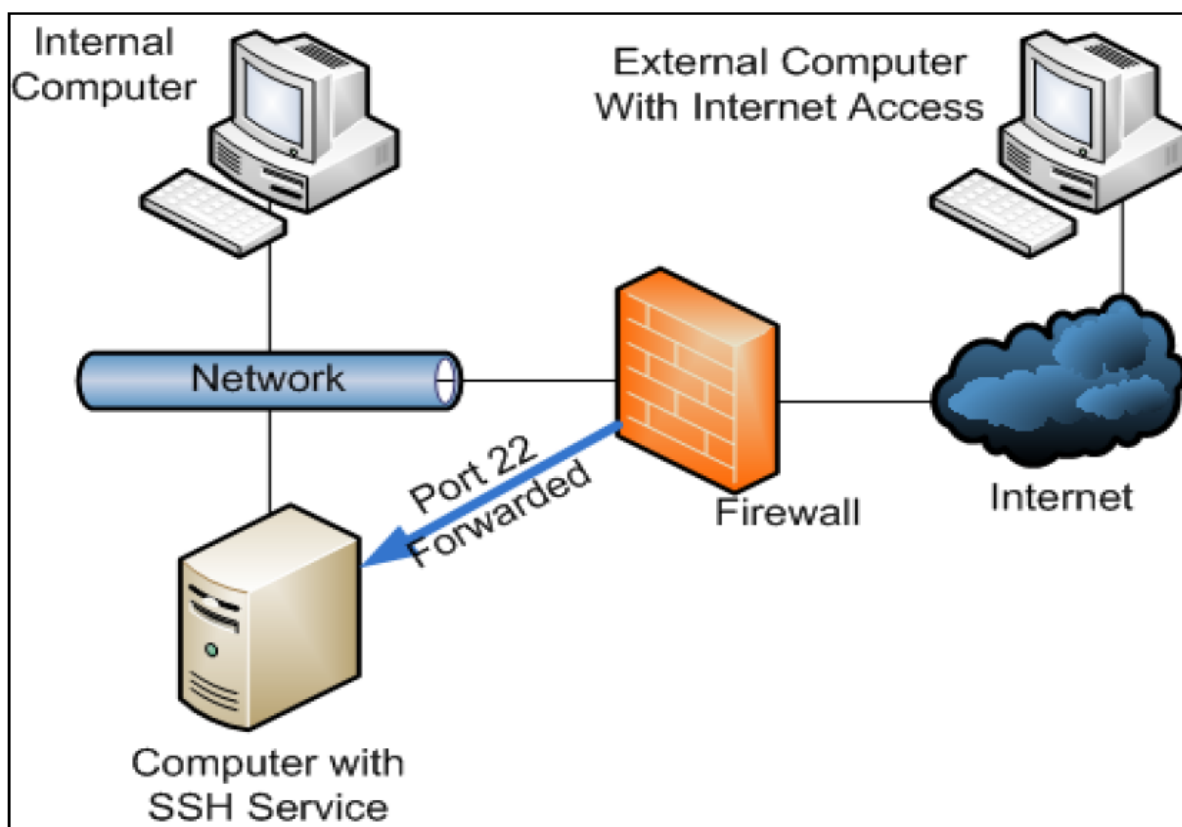


Figura 4: Encaminhamento de Porta

Fonte: LeGauss (2012)

2.2.7 Load Balancing (Balanceamento de Carga)

O balanceamento de carga torna os servidores altamente disponíveis e adaptáveis com a utilização de dois computadores disponibilizando seus recursos em conjunto. Para quem utiliza esses servidores o *cluster* (aglomerado de computadores) é transparente, ou seja, o usuário não percebe que está alternando entre duas máquinas diferentes. O balanceamento de carga pode oferecer um serviço ininterrupto, já que mesmo com uma máquina falhando a outra máquina pode continuar oferecendo o serviço (TechNet, 2014). A figura 5 mostra como o balanceamento de carga pode ser feito.

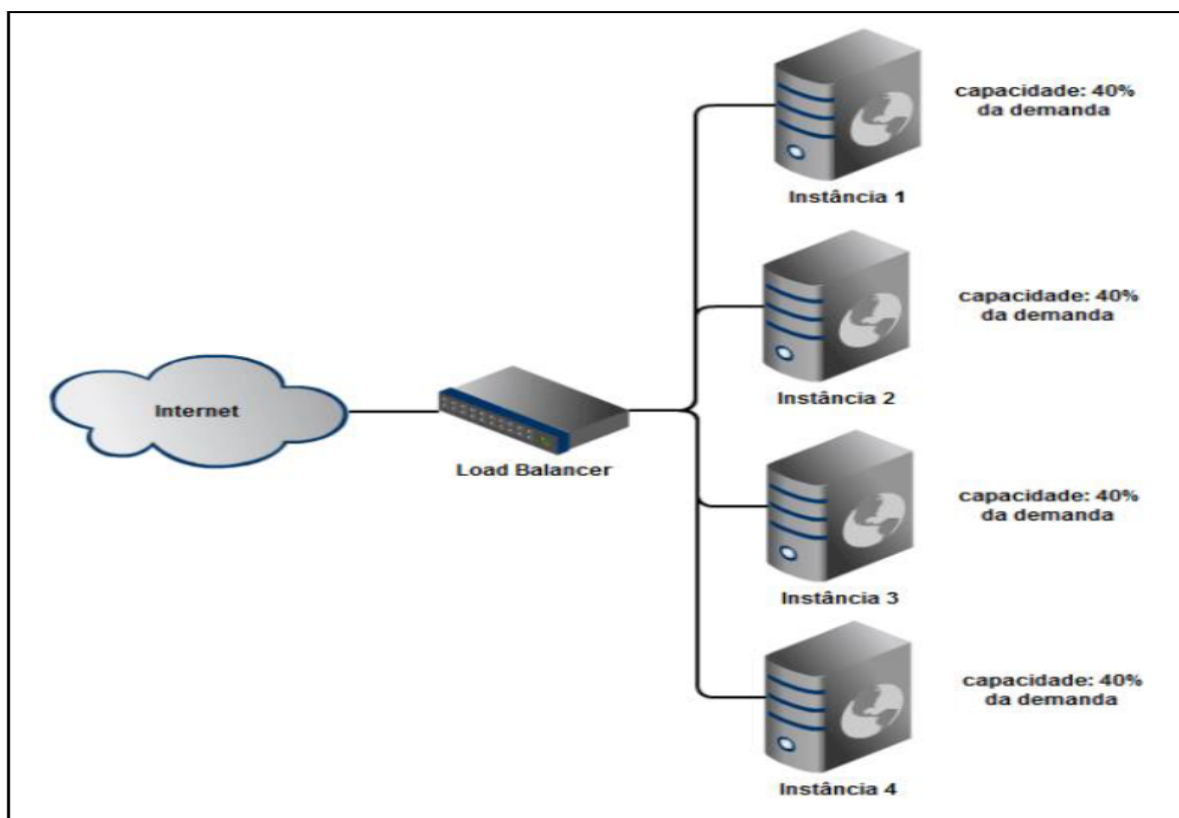


Figura 5: Balanceamento de carga

Fonte: Planeta Tecnologia (2012)

2.2.8 Apresentação do *firewall Pfsense*

O *Pfsense* é um *software* livre customizado da distribuição do *FreeBSD*, sendo adaptado para uso como *firewall* e roteador, que é inteiramente gerenciado via interface *web*. Além de ser um poderoso *firewall*, e uma plataforma de roteamento, ele possui uma variada lista de recursos que podem ser adicionadas através de *downloads* de pacotes permitindo assim a adição de funcionalidades de acordo com a necessidade do usuário. O projeto do *Pfsense* iniciou-se em 2004 se diferenciando de outro projeto o *m0n0wall*, por ser um projeto para ser instalado completamente em um pc (*PFSENSE*, 2014).

Recursos do *Pfsense*:

- *Firewall*;
- *State Table* (Tabela de Estados);
- *NAT*;
- Alta Disponibilidade;
- *Load Balancing* (Balanceamento de carga);
- *VPN*;

- *PPPoE Server*;
- *Reporting e Monitoring* (Relatório e Monitoramento);
- *Dynamic DNS* (DNS Dinâmico);
- *Captive Portal*;
- *DHCP Server and Relay*;

2.3 DETECÇÃO DE INTRUSÃO

2.3.1 Sistemas de detecção de intrusão

A detecção de intrusão é baseada na suposição de que o comportamento do intruso difere daquele de um usuário legítimo de maneira que podem ser quantificadas. Naturalmente, não podemos esperar que haverá uma distinção clara, exata, entre um ataque de um intruso e o uso normal dos recursos por um usuário autorizado. Em vez disso, devemos esperar que haja uma sobreposição (STALLINGS, 08).

Para se conseguir detectar uma intrusão em uma rede de computadores ou em um sistema de computador, deve ser feita uma análise, de forma manual ou automatizada, nos fatos ocorridos nestes meios.

Os sistemas de detecção de intrusão (*SDI*) tem como função o monitoramento e análise de eventos, de forma automática, que venham a ocorrer com o objetivo de informar o acontecimento de uma intrusão ou agir de forma proativa para que a intrusão não tenha sucesso para o invasor (BACE, 00).

Segundo Stalling (STALLINGS, 08) esse interesse por esse tipo de sistema é motivado por uma série de considerações, incluindo as seguintes:

- Se uma intrusão for detectada com rapidez suficiente, o intruso poderá ser identificado e expulso do sistema antes que seja feito qualquer dano ou qualquer dado seja comprometido. Mesmo que a detecção não seja suficientemente oportuna para impedir o intruso, quanto mais cedo a intrusão for detectada, menor serão os danos e mais rapidamente a recuperação poderá ser obtida;
- Um sistema de detecção de intrusão eficaz pode servir como um elemento desencorajador, atuando para impedir intrusões;

- A detecção de intrusão permite a coleta de informações sobre as técnicas de intrusão, o que pode ser usado para fortalecer a estrutura de prevenção de intrusão.

2.3.2 Arquitetura

Bace (BACE, 00) define que, de forma geral, um *SDI* é composto por três componentes básicos:

- Uma fonte de Informações de eventos ocorridos dentro do meio de atuação do *SDI*;
- Um componente que busca por sinais de intrusão nestes eventos;
- Um componente que tem como responsabilidade de reagir quando uma intrusão é detectada pelo componente anterior.

A arquitetura de um *SDI* precisa ser segura, pois ele pode ser alvo de um ataque. O invasor que explorar uma possível vulnerabilidade do *SDI* tem uma enorme vantagem, pois nenhum sistema de detecção estaria atuando sobre o seu ataque. Uma forma de proteção seria separar a informação que será analisada do sistema alvo do monitoramento, pois se essa separação não existir, o invasor altera estes dados, tornando o ataque imperceptível.

2.3.3 Estratégias de monitoramento

A atividade de monitoramento de um *SDI* depende do alvo monitorado pelo sistema que, segundo Bace (BACE, 2001), podem ser de três tipos: monitoramento em redes, monitoramento de computadores e monitoramento baseado em aplicações.

O monitoramento em redes de computadores analisa os pacotes da rede que trafegam em um segmento de rede. Também pode ser feito através de um *switch*, que detecta sinais de uma intrusão em algum dispositivo da rede ou de seus segmentos. É realizado distribuindo-se, nos diversos computadores da rede, agentes que analisam o tráfego de rede que passam por eles e tentam identificar pacotes de rede que contenham dados que identifiquem que uma invasão está acontecendo.

Devido a facilidade de distribuir os agentes monitores pela rede, torna-se vantajoso utilizar este tipo de monitoramento em grandes redes de computadores. Do ponto de vista do atacante, o monitoramento da rede torna-se imperceptível, uma vez que a distribuição dos agentes monitores em diversos pontos da rede aumenta a dificuldade em identificar um monitoramento nesta rede.

Por outro lado, em momentos que a rede esteja com tráfego intenso, pode não ser possível realizar a análise de todos os pacotes. Com isso, pode ocorrer que justamente dos pacotes que identificam a tentativa de intrusão não sejam monitorados. Outra desvantagem é a impossibilidade de um agente analisar pacotes que tenham dados criptografados. Atualmente, devido à necessidade de segurança dos dados que trafegam nas redes, este tipo de pacote é comum de ser encontrado.

Outro tipo de estratégia, segundo Bace (BACE, 2001), é o monitoramento de computadores. Este tipo de monitoramento analisa eventos que ocorrem em um ou mais sistemas de um computador. Na maioria das vezes, dados do sistema operacional são analisados, como por exemplo, *logs* do sistema de informações disponibilizados pelo *kernel*.

Os sistemas baseados neste tipo de monitoramento estão mais vulneráveis a ataques como, por exemplo, ataques de negação de serviço. Uma maior facilidade é gerada ao intruso por possuírem um ponto único de monitoramento. Outra desvantagem é a grande quantidade de informação gerada por um sistema operacional, pois em computadores que geram *logs* frequentemente, essas informações podem ser difíceis de serem analisadas, dificultando a identificação de possíveis sinais de ataque. Além disso, deve ser levado em consideração a diminuição de desempenho do computador monitorado, devido ao fato do monitoramento.

A utilização deste tipo de monitoramento tem suas vantagens. Ele possibilita a detecção ataques que não conseguem ser detectados através de uma rede de computadores, quando outros sistemas que rodam no computador possuem brechas que são exploradas por intrusos e dados de informação destes sistemas são gerados pelo sistema operacional da máquina.

O terceiro tipo de estratégia é um *SDI* baseado em aplicações. Normalmente, esse tipo de sistema monitora dados disponibilizados por aplicações que, comumente, consistem em *logs* transações realizadas através da aplicação.

Seu intuito é localizar e identificar usuários que excedem suas permissões de acesso aos dados do sistema ou ainda acesso não autorizado a dados do sistema.

Uma vez que este tipo acontece através do monitoramento da interação entre o usuário e a aplicação, uma grande vantagem que se pode obter é a identificação de acessos não autorizados.

Em contrapartida, uma desvantagem é que os arquivos de *log*, que contém as informações que são monitoradas pelo *SDI*, estão desprotegidos pelo sistema operacional. Esta falta de proteção compromete a eficácia do sistema, pois possibilita que estes dados sejam apagados ou adulterados.

2.3.4 Disposição de sistema de monitoramento

Os componentes de um *SDI* podem estar dispostos, segundo Bace (BACE, 2001), de três formas: centralizados, distribuídos ou organizados de forma hierárquica. Cada uma destas formas apresenta vantagens e desvantagens, que são explicadas a seguir.

2.3.5 *SDI* com monitoramento centralizado

Neste tipo de arquitetura, os dados monitorados pelos sensores são enviados para o console do *SDI*, cuja função é identificar as intrusões e tomar decisões sobre os alarmes que devem ser gerados no momento em que uma intrusão for identificada.

Este tipo de sistema é de fácil instalação e configuração. Em comparação a um sistema distribuído que necessita de um maior tempo de planejamento, do ponto de vista do desenvolvimento este sistema é mais simples. Entretanto, este tipo de *SDI* se torna ineficiente quando diversidade e tamanho das arquiteturas de redes existentes no ambiente é grande.

2.3.6 *SDI* com monitoramento distribuído

Nesta abordagem, os módulos comunicam-se através de mensagens, cooperando para que todo o sistema funcione corretamente e sejam minimizadas as possibilidades de falhas.

Nesta arquitetura, os dados não são enviados para um ponto central. Os sensores de monitoramento estão distribuídos em diversos pontos da rede obtendo dados, analisando possíveis intrusões e se necessário, reportam em um determinado formato (protocolo) o acontecimento de uma intrusão.

Um ponto negativo que contrasta com uma implementação centralizada é a necessidade de implementação elevada para garantir a segurança da troca de informações entre os módulos.

2.3.7 SDI com monitoramento híbrido

Em um SDI que possui este tipo de monitoramento, existe uma hierarquia onde os sensores se reportam ao uma console específica e estas reportam os dados recebidos dos diversos sensores a uma console principal.

Esta arquitetura possui o mesmo problema existente na arquitetura centralizada: Existem pontos que se estiverem vulneráveis a falhas ou a ataques, podem causa a indisponibilidade do sistema. A vantagem desta arquitetura é herdada das duas anteriores. Pode ser citada, principalmente, a facilidade de implementação. Além disso, algumas características provenientes da arquitetura distribuída e a tolerância a falhas nos módulos distribuídos continuam existindo.

2.3.8 Sistemas de detecção de intrusão distribuídos

Até recentemente, o trabalho sobre sistemas de detecção de intrusão focalizava as estruturas isoladas em um único sistema. Uma organização típica, porém, precisa defender um conjunto distribuído de *hosts* que compõem uma LAN ou inter-rede. Embora seja possível montar uma defesa usando sistemas de detecção de intrusão isolados em cada *host*, uma defesa mais eficaz pode ser obtida pela coordenação e cooperação entre sistemas de detecção de intrusão na rede (STALLINGS, 08).

Porras (PORRAS, 1992) aponta os seguintes problemas principais no projeto do sistema de detecção de intrusão distribuído:

- Um sistema de detecção de intrusão distribuído pode ter de lidar com diferentes formatos de auditoria. Em um ambiente heterogêneo, sistemas diferentes empregarão diversos sistemas de coleta de auditoria nativos e, se

estiverem usando a detecção de intrusão, podem empregar formatos variados para registros de auditoria relacionados à segurança;

- Um ou mais nós na rede servirão como pontos de coleta a análise dos dados dos sistemas na rede. Assim, dados brutos de auditoria ou dados resumidos devem ser transmitidos pela rede. Portanto, existe um requisito para garantir a integridade e a confidencialidade desses dados. A integridade é exigida para impedir que um intruso mascare suas atividades alterando as informações de auditoria transmitida. A confidencialidade é exigida porque as informações de auditoria transmitidas podem ser valiosas;

Pode ser usada uma arquitetura centralizada ou descentralizada. Com uma arquitetura centralizada, existe um ponto de coleta e análise central único de todos os dados de auditoria. Isso facilita a tarefa de correlacionar os relatórios que chegam, mas cria um gargalo em potencial e um ponto único de falha. Com a arquitetura descentralizada, existe mais de um centro de análise, mas estes precisam coordenar suas atividades a troca de informações.

2.3.9 Conceito de detecção de intrusão em redes de computadores

O conceito de detecção de intrusão (*Intrusion Detection System - IDS*) pode ser referenciado a uma tentativa com sucesso ou não de acesso, manipulação das informações e/ou tornar um sistema invadido não confiável. Um ataque é caracterizado como uma tentativa de modificar, manipular, ou perturbar o funcionamento de um sistema, bem-sucedida ou não (WANG, 2009).

2.3.10 Intrusão em rede

Segundo Silva e Sampaio (2006) intrusão pode ser definida como um conjunto de ações desencadeadas pelo intruso que compromete a estrutura básica da segurança da informação de um sistema: integridade, confidencialidade e disponibilidade. Devemos ter uma diferenciação entre “Ataque” e “Intrusão”, pois parecem ser a mesma coisa, mas tem algumas particularidades: ataque refere-se à tentativa de perturbação, já intrusão é um ataque realizado que obteve sucesso (foi bem-sucedido), pois invadiu a rede.

2.3.11 Intrusos

De forma simplificada pode-se dizer que intruso é quando um usuário tenta invadir um Sistema ou fazer mau uso do mesmo (LAUFER, 2003).

Classificam-se os intrusos em dois tipos:

- Intrusos Externos: são usuários que não possuem acesso físico a rede ou sistemas que estão atacando;
- Intrusos Internos: são usuários autorizados a usar os recursos disponíveis pelo sistema, estes possuem acesso às máquinas e seus recursos, são identificados como usuários legítimos do sistema, que com seus privilégios para tirarem algum proveito, podendo ser de duas formas: mascarados (passam como usuários legítimos do sistema) e clandestinos (aqueles que têm o poder de direcionar os dados de controle para si próprios), tendo como principal diferença o seu perfil de uso.

Os intrusos clandestinos são mais difíceis a serem detectados, eles aproveitam das condições privilegiadas que possuem para conseguir vantagens não autorizadas, podendo assim direcionar dados para outros caminhos (LAUFER, 2003).

Uma representação de ameaça em um sistema pode ser vista na figura 6, onde os recursos protegidos são vistos como anéis de controle e anéis de usuários.

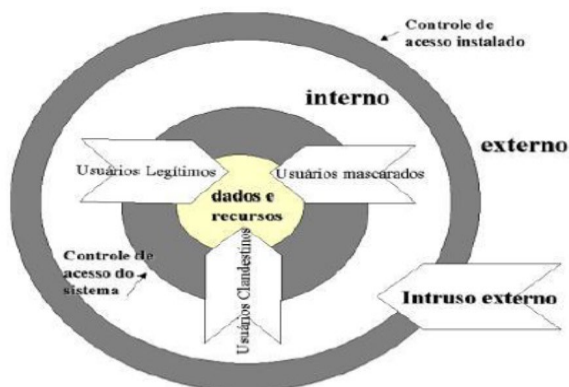


Figura 6: Tipos de intruso

Fonte: LAUFER (2013)

Na figura pode-se verificar que quando o controle de acesso possuir uma falha, os intrusos externos passam pelo mesmo e buscam acesso ao sistema. Já os intrusos internos quando encontram uma falha ou vulnerabilidade no controle

de acesso ao sistema invadem os dados e os recursos se passando por usuários legítimos.

2.3.12 Falsos positivos e falsos negativos

Na atualidade um dos problemas de *IDS*, levando em conta a questão de eficiência (precisão da detecção) que contabiliza através do número de erros de detecção que ocorrem, podemos verificar os seguintes critérios:

- Falsos positivos (FP): ocorrem quando pacotes são identificados pelas ferramentas de segurança como tentativas de ataque, quando na verdade trata-se de ações legítimas e não ataques. Pode ocorrer um número grande de falsos positivos que irá até mesmo atrapalhar a análise de um arquivo de registro de eventos, onde os ataques verdadeiros podem passar despercebidos (CRUZ, 2000);
- Falsos negativos (FN): ocorrem quando não são identificadas/detectadas as tentativas autênticas de ataques. No caso de uma ferramenta de detecção de intrusão usar análise de assinaturas, o requisito seria que as mesmas sejam atualizadas constantemente para verificar que um *log* de ataque que não tenha suas regras, passe despercebido (NED, 1999).

2.3.13 Análise e comportamento do fluxo de dados na rede

Apenas com um monitoramento mais adequado do tráfego de rede é possível determinar certos tipos de ataque. Para isso, existem várias maneiras de obter as informações sobre o tráfego de rede, sendo uma delas através de consultas *SNMP* (*SNMP-based*) dirigida aos dispositivos conectados à rede. Este tipo de consulta retorna informações a respeito do nível do tráfego, e não fornecem dados suficientes para uma análise de segurança mais detalhada dos sistemas monitorados (BERTHOLDO; ANDREOLI; TAROUCO, 2003).

Um fluxo pode ser identificado unicamente pelas seguintes informações, conforme Amoroso (1999):

- Endereço *IP* de origem e de destino;
- Porta de origem e destino (camada de transporte);
- Tipo de protocolo (camada de rede);

- Tipo de serviço (cabeçalho *TCP*);
- Interface de entrada do roteador.

Uma vez os dados gerados no *gateway* e armazenados em um servidor, eles são processados por ferramentas, gerando informações em modo gráfico e permitindo consultar os dados armazenados na base de dados de fluxos, através de um conjunto de ferramentas adicionais.

Os dispositivos conectados à rede são capazes de verificar o tráfego, identificando uma sequência de pacotes (mensagens) que são trocadas (AZEVEDO, 2012), assim esse tráfego pode ser definido como:

- Normal: tráfego legítimo, ou seja, não existe a ocorrência de ataques ou anomalias;
- Anômalo: presença de ataques ou anomalias como interrupção de segmentos da rede.

2.3.14 Localização de um *IDS* na rede

Em alguns casos a análise feita pelos *NIDS* abrange segmentos da rede mais sensíveis, dependendo da política de segurança, devido a este fato normalmente esses sistemas ficam após os *firewalls* e roteadores, conforme é apresentado por Rehman (2013) na figura 7, o posicionamento mais comum para um Sistema de Detecção de Intrusão:

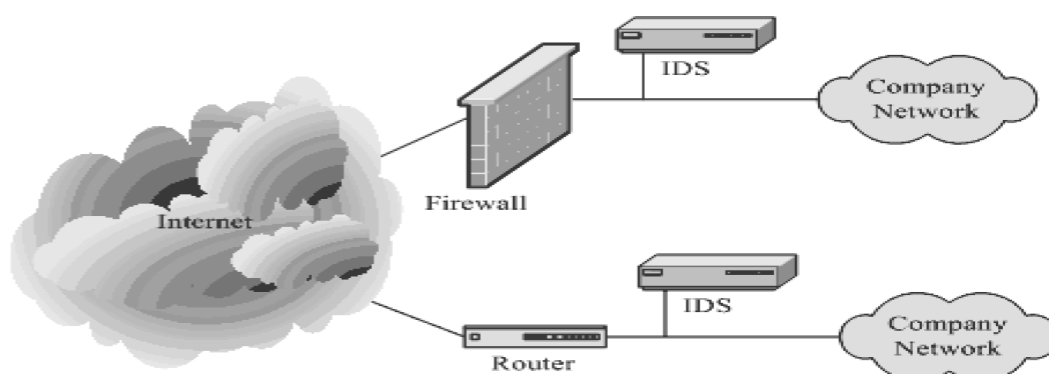


Figura 7: Localização dos Sistemas de Detecção de Intrusão
Fonte: REHMAN (2013)

O posicionamento dos *IDS* depende de uma série de fatores, esses fazendo referência à topologia da rede e as atividades de intrusão que se deseja monitorar.

Os ataques têm a sua origem em qualquer ponto da rede, tanto em redes locais ou outras. Muitas vezes, pode não ser suficiente a colocação dos equipamentos de monitoramento apenas nos pontos de entrada na rede (CARDANA, 2006).

2.3.15 Tipos de detecção de intrusão segundo a arquitetura

A combinação de diferentes tipos de *IDS* é importante para que a organização esteja adequadamente protegida contra ameaças, principalmente dos ataques realizados internamente e dos ataques vindos da *internet* (NAKAMURA, 2007), podendo ser classificados em detecção de intrusão baseado em *host* e rede.

Os Sistemas de Detecção de Intrusão baseados em *Host* (*HIDS – Host Based Intrusion Detection System*) realizam o monitoramento das atividades do sistema e dos programas que rodam no mesmo, com base em informações de arquivos de *logs* ou de agentes de auditoria.

Os *HIDS* são capazes de monitorar acessos e alterações em importantes arquivos do sistema, modificações nos privilégios dos usuários, processos do sistema, programas que estão sendo executado, uso da *CPU* e detecção de *port scanning*. Essas ferramentas são usadas para detectar atividades maliciosas em um único computador (KIZZA, 2005).

A detecção de Intrusão baseados em *host* conforme Kizza (2005) apresenta algumas vantagens:

- Capacidade de verificar sucesso ou falha de um ataque rápido analisando os *logs* do evento, apresentando informações mais precisas e menos falsos positivos;
- Detecção em tempo quase real, e os alertas são enviados ao administrador rapidamente;
- Não necessita de *hardware* adicional para sua instalação assim tendo um custo reduzido;
- Acessa informações antes e após a encriptação de dados;

- Capaz de analisar atividades em baixo nível, como acesso as permissões dos arquivos e tentativas de mudanças de privilégios.

O sistema de detecção de Intrusão baseados em Rede (*Network-Based Intrusion Detection System - NIDS*) monitora o tráfego do segmento da rede, geralmente com a interface de rede atuando em modo “promísco”.

A detecção é realizada com a captura e análise dos cabeçalhos e conteúdo dos pacotes, que são comparados com assinaturas ou padrões conhecidos.

2.3.16 Detectores de intrusão em redes de computadores

Os Sistemas de Detecção de intrusão em Redes de computadores (*NIDS, Network Intrusion Detection System*) são utilizados para monitoramento do tráfego de dados de uma rede ou de um segmento de rede. A análise é realizada em dados coletados da rede, ou em base de dados disponíveis ao *NIDS* (AZEVEDO, 2012).

Segundo Nakamura (2007),

NIDS são componentes essenciais em um ambiente cooperativo. Sua capacidade de detectar diversos ataques e intrusões auxilia na proteção do ambiente, e sua localização é um dos pontos a serem definidos com cuidado (NAKAMURA 2007, p. 264).

Conforme Kizza (2005) as principais vantagens dos *NIDS* são:

- Detecção e resposta em tempo real: Estando em pontos estratégicos da rede, consegue detectar intrusões rapidamente e notificar ao administrador;
- Capacidade de detectar ataques que os *HIDS* não pegam: Monitora no nível de transporte da arquitetura da rede. Nesse nível analisa pacotes não apenas por endereços, mas também em números de porta;
- Dificuldade de remover evidências: Os *NIDS* ficam em uma máquina dedicada e protegida, o que dificulta bastante a remoção de evidências pelo atacante;
- Baixo custo: Segundo Wang (2009), é necessário apenas sondas em alguns pontos estratégicos da rede. A monitoração passiva é outro ponto positivo, pois não há tráfego de rede normal resistente a intrusão;

Os *NIDS* possuem também algumas desvantagens como: não é possível analisar protocolos criptografados, há dificuldade de identificar ataques

fragmentados, possui pontos cegos, ou seja, na maioria das vezes os *NIDS* são colocados nas bordas da rede com isso alguns segmentos não são vistos para análise.

2.3.17 Componentes do *NIDS*

De acordo com Nakamura (2007), um sistema de detecção de intrusão funciona de acordo com uma série de funções que, trabalhando de modo integrado, são capazes de detectar, analisar e responder as atividades suspeitas. A figura 8 apresenta as funções e distribuição de um *NIDS* abaixo.

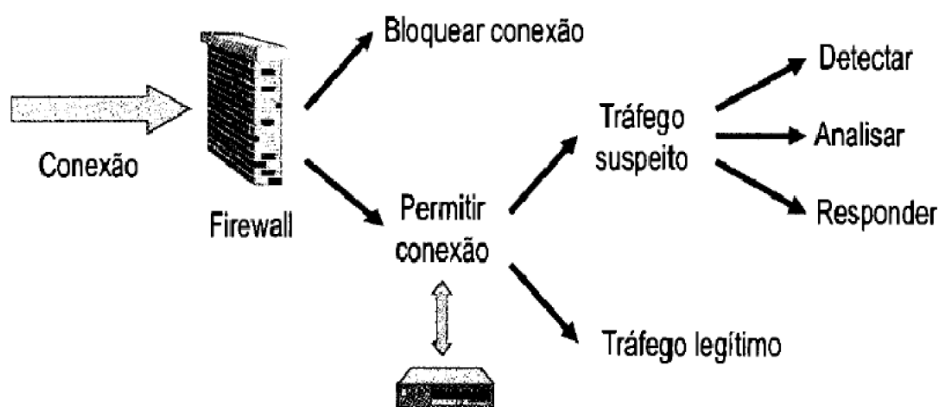


Figura 8: Funções dos *NIDS*
Fonte: NAKAMURA (2007)

Como se verifica na figura, após o *firewall* liberar as conexões, os sistemas de detecção coletam, analisam, armazenam essas informações e respondem às atividades suspeitas, classificando o tráfego como suspeito ou legítimo.

Para que o sistema tenha um bom desempenho na detecção de intrusão necessita-se um conjunto de ferramentas básicas como um coletor (sonda), analisador, banco de dados (*Mysql*), atuador e monitor, conforme especificado abaixo:

- **Coletor:** O coletor é responsável por capturar descritores do tráfego de rede, e normalmente está conectada em algum ponto de interconexão da rede, como por exemplo: roteador de borda, *bridge*, *firewall*, etc. O desempenho do coletor depende dos equipamentos da rede usados para a coleta, principalmente em redes de grande tráfego. Alguns *firewalls* também atuam como coletor,

armazenando informações para os *NIDS* (NORTHCUTT; NOVAK, 2002 apud PERLIN, 2010);

- Analizador: É um componente responsável por verificar os dados coletados buscando por eventos que indiquem uma intrusão ocorrida ou que estejam ocorrendo, tendo diferentes abordagens para análise dos dados, como a baseada em assinaturas ou anomalias descritas no decorrer do trabalho;
- Banco de dados: É onde ficam guardadas as informações dos *NIDS* e os eventos suspeitos, para posteriormente busca dessas informações e análise mais detalhada;
- Notificador: É o sistema responsável pelo envio de alertas ao administrador. Os alertas frequentes, com falsos positivos são prejudiciais e deixam o sistema com pouca confiança;
- Atuador: É uma ferramenta que possui a capacidade de execução de ações automatizadas quando uma anomalia é detectada pelo *IDS*. Na maioria das vezes o monitor ou terminal de comando tem o objetivo de fazer ligação entre o administrador do sistema e o sistema de detecção, sendo o monitor usado para configurar, verificar o funcionamento do *IDS* e até mesmo envio de alertas.

2.3.18 Classificação dos métodos de detecção

Os sistemas de detecção de intrusão se utilizam de algumas metodologias para a realização da análise dos dados e, consequente detecção de incidentes.

Assim, sendo classificados quanto ao método de análise ou detecção dos dados normalmente de duas formas: baseados em assinaturas e baseados em anomalias. A maioria dos sistemas de detecção existente se utiliza de ambas as metodologias, de forma separada ou integrada, para uma detecção híbrida e mais precisa de possíveis incidentes (SCARFONE; MELL, 2007).

2.3.19 SDI baseado em assinaturas

Um sistema de detecção de intrusão baseado em assinaturas compara os dados coletados em um ambiente com uma base de dados de ataques conhecidos ou regras pré-definidas por um sistema de detecção. Quando os eventos analisados são compatíveis com alguma assinatura da base de dados um alarme é disparado.

Segundo Roesch (1999), a detecção baseada em assinatura identifica ataques através da análise de assinaturas previamente estabelecidas sobre o comportamento padrão de determinado tipo de ataque, que são geradas por especialistas.

Com o surgimento de novas formas de ataques ou variações são necessárias atualizações constantes na base de ataques. Porém, mesmo com essa base atualizada, os *NIDS* têm dificuldades de detectar ataques desconhecidos, ataques mutantes ou ataques camuflados. De acordo com cada sistema, ataques são modelados como padrões de eventos específicos e quando a ocorrência de algum ou parte desses padrões é observada considera-se que há um ataque em curso (DEMIRAY, 2005).

A metodologia baseada em assinaturas é eficiente na detecção de ameaças com comportamento constante e já conhecidas, entretanto, é ineficaz na detecção de ameaças ainda desconhecidas ou na detecção de ameaças já conhecidas e que se utilizam de técnicas de evasão e, conseqüentemente, tem o comportamento original alterado (DEMIRAY, 2005).

Os *NIDS* baseados em assinaturas são bastante precisos nas suas detecções, apresentando baixo número de falso positivo, mas devido à dificuldade de detectar de novos ataques, podem apresentar grande número de falsos negativos, onde os ataques não são detectados, sendo uma possível falha de segurança.

Existem diversos *NIDS* disponíveis para o uso, como o *Suricata* (*Open Information Security Foundation*), *Snort* (Roesch, 1999) (*software* livre), *NetRanger* (*Cisco*, 2011) (*software* proprietário). Estes *NIDS* são baseados em assinaturas, os quais inspecionam eventos atuais e comparam suas regras (assinaturas) com dados da rede (WANG, 2009).

Na figura 9 verifica-se que os dados da auditoria são comparados pelo perfil do sistema (regras estabelecidas) e quando houver uma combinação dos dados da rede com as regras estabelecidas há uma detecção de ataques e envio de alertas.

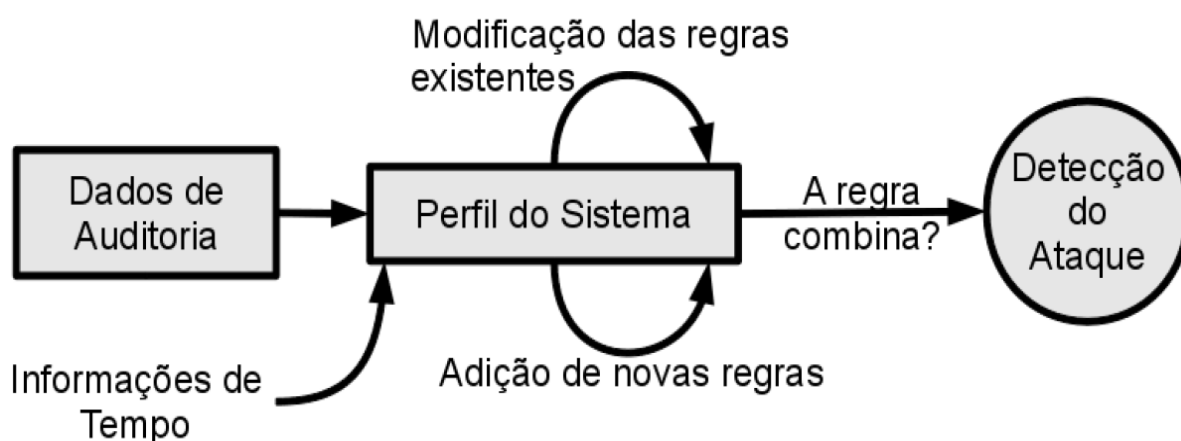


Figura 9: Detecção de intrusão baseado em assinatura

Fonte: SUNDARAM (1996)

Os diferentes tipos de assinaturas utilizados nesses sistemas podem ser (WANG, 2009):

- Assinaturas de redes: Informações dos pacotes que podem afetar a execução normal da rede consistem em assinaturas de cabeçalhos ou de campo de dados (*payload*), os quais verificam ações do usuário, enquanto assinaturas de cabeçalhos verificam pacotes maliciosos que são capazes de identificar, como, por exemplo, ataques de difusão (*broadcast*);
- Assinaturas de hosts: Utilizam informações de comportamentos que podem afetar na execução normal do sistema. Um exemplo seria três tentativas seguidas de senha errada de determinado usuário. Essas assinaturas podem ser:
 - a) Evento-único: Um único comando que é um comportamento suspeito;
 - b) Multi-eventos: Formada por grupos de várias assinaturas de evento único;
 - c) Multi-hosts: Formada por sequências de assinaturas de eventos-únicos originados de diferentes *hosts*;
 - d) Compostos: Faz a união de assinaturas de rede com assinaturas de *hosts* para identificar certos tipos de ataques que não podem ser identificados somente por um tipo de assinatura.

2.3.20 SDI baseado em anomalias

Nos *NIDS* baseado em anomalias é analisado o comportamento do tráfego de rede, sendo classificada como anomalia, a ocorrência de um evento que não segue o comportamento esperado, ou semelhante ao suportado pela rede (CHANDOLA; BANERJEE; KUMAR, 2009).

Anomalia é um evento que causa uma alteração em relação ao perfil padrão do sistema (KRUEGEL, 2003). De um modo amplamente analisado uma anomalia na rede pode ocorrer devido a um ataque, falha de equipamentos, problemas de configuração, sobrecarga da mesma ou uso inadequado de algum serviço ou recurso, então a possibilidade com *NIDS* baseados em anomalias na detecção dessas falhas é grande, se tornando um sistema complexo.

Em função de uma observação prévia das características de um sistema durante um determinado período de tempo torna-se possível definir o perfil de comportamento de seus diversos componentes (rede, usuários, estações, aplicações). Assim, esta metodologia de detecção se baseia na observação de eventos que ocorrem no sistema e sua comparação com um perfil já definido para uma possível identificação de desvios do comportamento normal, ou seja, identificação de anomalias no comportamento do sistema (DEMIRAY, 2005; LINDA, VOLLMER; MILOS, 2009).

A maior vantagem deste tipo de abordagem é a possibilidade de se detectar técnicas de intrusão ainda desconhecidas, pois embora não se conheça o seu comportamento, esse deve se distinguir naturalmente do comportamento esperado e de acordo com o perfil existente.

Além da necessidade de uma base de dados muito extensa, as principais desvantagens dessa metodologia é a impossibilidade de se aprender totalmente o perfil conforme o sistema se torna complexo e o fato que o comportamento dos componentes do sistema monitorado pode mudar com o tempo em consequência da mudança natural de comportamento dos usuários, causando uma taxa elevada de falsos alarmes (LINDA, VOLLMER; MILOS, 2009).

Os sistemas baseados em anomalias baseiam-se na supervisão de comportamentos anômalos do sistema, assumindo que as atividades anormais podem ser tentativas de invasões. Essa técnica constrói primeiro um modelo, geralmente estatístico, capaz de descrever as atividades normais dos usuários, dos

hosts, ou o tráfego normal da rede, marcando qualquer comportamento que significadamente desvia de um modelo como um ataque.

A figura 10 mostra quando houver um comportamento diferenciado de um novo perfil diante do que era considerado normal anteriormente é detectado um ataque.



Figura 10: Detecção de intrusão baseado em anomalias

Fonte: SUNDARAM (1996)

Esse tipo de *IDS* apresenta erros devido à identificação de atividades normais de um usuário como atividades anormais, ou quando deixa de identificar atividades anormais como invasão por parecerem muito com a atividade costumeira de um usuário. Podem-se encontrar os comportamentos (KUMAR e SELVAKUMAR, 2009):

- Intrusivo, mas não anômalo: também chamados de falso negativos. Neste caso, ocorre uma falha na detecção, pois a invasão não provoca atividade anormal, não sendo detectada pelo sistema;
- Não intrusivo, mas anômalo: também chamados de falsos positivos. Neste caso, ocorre uma falha na detecção, que indica erroneamente a anormalidade como uma invasão;
- Não intrusivo e não anômalo: também chamados de verdadeiros negativos (comportamento normal), neste caso a atividade não é anômala e não é evidenciada como invasão;
- Intrusivo e anômalo: também chamados de verdadeiros positivos. Neste caso, ocorre um acerto, pois a atividade é anômala e evidenciada como invasão.

Das diferentes abordagens de algoritmos utilizados, somente duas tem sucesso nas aplicações da última década: modelos estatísticos e baseados em redes neurais (PIETRO, MANCINI, 2008).

Os sistemas orientados a conexão poderiam até utilizar o campo de dados (*payload*) dos pacotes, porém o tempo utilizado para essa tarefa seria grande, o que diminuiria o desempenho do sistema. Na prática, tipicamente obtém-se o número de *bytes* enviados e recebidos, a duração da conexão e o protocolo da quarta camada do modelo de referência *TCP/IP* que foi utilizado (PIETRO, 2008).

Quanto aos tipos de dados utilizados, têm-se os sistemas que utilizam o cabeçalho e os que utilizam o *payload* do pacote. O primeiro considera somente os cabeçalhos dos pacotes, utilizando os cabeçalhos da terceira camada e, se existir, da quarta camada do modelo de transferência *TCP/IP*. Já os baseados no *payload* analisam os dados somente da quarta camada do modelo de transferência *TCP/IP* (aplicação). Existem também sistemas híbridos, que misturam informações coletadas dos cabeçalhos de pacotes e, se houver dos dados do *payload* da quarta camada do *TCP/IP* (PIETRO, 2008).

2.3.21 Ferramenta de detecção de intrusão

2.3.21.1 Apresentação do *Snort*

O *Snort* é um sistema de detecção e prevenção de intrusos de código fonte aberto, baseado em rede (*NIDS*). Seu criador foi Martin Roesch, que disponibilizou sua primeira versão de testes no ano de 1999, se tornando bastante popular pela flexibilidade nas configurações de regras e atualizações comparadas com outras ferramentas disponíveis (*SNORT*, 2013).

2.3.21.2 História do *Snort*

Esse *NIDS* foi desenvolvido na linguagem C, baseando-se na biblioteca de programação *Libpcap*, que faz a captura dos pacotes de rede. Inicialmente seu criador Roesch queria apenas uma ferramenta *Sniffing* melhor para o sistema operacional *Linux* que as existentes como o *tcpdump* (ferramenta nativa no *Unix* que captura pacotes).

Depois de ter seus objetivos concluídos na captura de pacotes, Roesch começou a desenvolver o *Snort* com função de *IDS*, pois não possuíam *softwares* livres com essas funções. Em 1999 consegue habilitar o *Snort* para comparar o tráfego de rede com as regras que ele definiu, sendo testada pela Comunidade Científica *Open Source*, passando por diversas atualizações tornando-se a mesma adaptável a vários ambientes (multiplataforma). Hoje a versão mais atual do *Snort* é a 2.9.x que pode ser instalada em vários sistemas operacionais, os quais desempenhando suas funções normalmente (*SNORT*, 2010).

O logotipo desse sistema de detecção de intrusão é um “porco”, faz essa comparação com o animal, pois fareja e captura todos os pacotes que estão passando na rede, comparando com seu conjunto de regras (banco de dados) existente.

A ferramenta *Snort* é bastante utilizada para detecção de intrusão, possuindo um desempenho bom e uma linguagem flexível para descrição de ataques. Baseia-se na aplicação de regras de filtragem em cada pacote, a fim de procurar assinaturas de ataques conhecidos. Uma limitação dessa ferramenta se refere ao fato de apenas procurar assinaturas de ataques em cada pacote, ou em um fluxo, sem conseguir detectar ataques que necessitem de pacotes de protocolos diversos para serem caracterizados.

2.3.21.3 Ambiente e mecanismos de detecção

O *Snort* pode ser instalado tanto no sistema operacional *Unix* e suas derivações quanto no *Windows*. Necessita da instalação da biblioteca *Libcap* no *Linux* e *Winpcap* no sistema operacional *Windows*, pois para o *Snort* funcionar ele necessita dessa biblioteca, podendo deixar a interface de rede em modo promíscuo, ou seja, capturando pacotes destinados a ela, bem como os pacotes destinados a outros *hosts* da rede.

Um componente importante para o *Snort* são os mecanismos de detecção, todos os dados provenientes dos pré-processadores são verificados através de um conjunto de regras (assinaturas), estas baseadas em um texto que normalmente em uma subdiretoria onde está instalado o *Snort*, constituída por duas partes: o cabeçalho e as opções. Exemplo: ficheiro “*rules*” é onde ficam todas as regras referentes a ataques *DoS*.

Existem cinco regras definindo o cabeçalho:

- Activation: Alerta e chama regra do tipo dinâmica “*dynamic*”;
- Dynamic: Permanece inativa até ser ativado por uma regra *activate*, registrando o tráfego;
- Alert: Gera um alerta usando um método selecionado e então registra os pacotes e os dados;
- Pass: Ignora os pacotes;
- Log: Registra e não alerta.

Para essas regras existe uma lista de protocolos suportados pelo *Snort*: protocolo *TCP* (*SNMP*, *HTTP*, *FTP*), *UDP* (*DNS*, *SNMP*, *DHCP*, *RIP*), *ICMP* (*Traceroute*, *ping*), *IP* (*ICMP*);

2.3.21.4 Regras

Como os vírus, as intrusões em redes têm determinados tipos de assinatura, essas usadas para criar regras do *Snort*. Podem ser usados *honeypots* (potes de mel), para descobrir o que os intrusos estão a fazer e quais as técnicas a utilizar. Essas assinaturas podem estar presentes em partes do cabeçalho de um pacote ou mesmo nos dados.

As regras do *Snort* (*rules*) operam na camada de rede (*IP*) e protocolos da camada de transporte (*TCP/UDP*) também existindo métodos para detectar anomalias nos protocolos da camada de aplicação.

As versões mais recentes do *Snort* fazem análise na camada de aplicação além da camada de rede e de transporte, onde as regras são aplicadas em ordem em todos os pacotes independente do seu tipo e/ou tamanho.

Conforme Caruso (2005) as regras do *Snort* são escritas em formato texto em uma única linha, e constituem-se de duas sessões: sendo cabeçalho e opções ou quando em linhas extensas são quebradas pelo uso de caracteres de concatenação (“\”), essas regras ficam armazenadas no ficheiro “*snort.conf*”. Podem ser usadas de diversas formas como: para gerar uma mensagem de alerta ou para fazer um registro de um evento.

Para entendermos melhor as regras deixamos em destaque e enumeradas algumas partes dos *logs* gerados por um ataque *FTP* em uma máquina *Linux* de forma a explicar seu funcionamento:

Exemplo de um cabeçalho de *log*:

```
alert tcp $EXTERNAL_NET any -> $HOME_NET 21
```

alert - Este é o formato utilizado para a saída. Os formatos de saída possíveis são: *alert*, *log*, *pass*, *dynamic* e *activate*. O formato de saída é utilizado pelo *Snort* para classificar e separar as regras em cinco categorias principais (cinco cadeias de regras).

TCP - Esta parte da sintaxe define o protocolo em uso; neste caso, *TCP*. Este campo pode aceitar os valores: *TCP*, *UDP*, *IP* e *ICMP*. Para cada uma das cadeias de saída citadas no item anterior, o *Snort* cria 4 novas cadeias, uma para cada tipo de protocolo aceito. Existirá, por exemplo, uma árvore de regras *TCP* para cada uma das cadeias de formato de saída.

\$EXTERNAL_NET - Esta parte da sintaxe é o endereço *IP* de origem.

any. Esta é a porta de origem selecionada como qualquer porta de origem.

=> Esta seta indica a direção do fluxo de dados; neste caso, de *\$EXTERNAL_NET* vindo de qualquer porta para *\$HOME_NET* na porta 21.

\$HOME_NET - A variável *HOME_NET* foi utilizada para representar os endereços de destino abarcados pela regra.

21 - É a porta destino, indicando potenciais ataques à porta 21, que é a porta tipicamente utilizada para atividades *FTP*.

Opções das regras:

```
msg: "FTP EXPLOIT wu-ftpd 2.6.0 site exec format string overflow Linux";
flow:to_server, established; content:"|31c031db31c9b046cd8031c031db|";
reference:bugtraq,1187; reference:cve,CAN-2000-0573; reference:
arachnids,287; classtype: attempted_admin; sid:344; rev:4;
```

msg "FTP EXPLOIT wu-ftpd 2.6.0 site exec format string overflow Linux". Esta é a mensagem exibida pelo alerta.

flow: to_server, established. O *Snort* contém palavras chave que ligam a módulos (ou "*plugins*") de detecção na seção de opções das regras. A opção *flow* é um apontador para os módulos de detecção cliente/servidor. Os módulos cliente-servidor ligam-se ao pré-processador para verificar se o pacote é parte de uma sessão estabelecida.

content "|31 c0 31 bd 31 c9 b0 46 cd 80 31 c0 31 db|". Se o pacote pertence a uma sessão estabelecida, o *Snort* irá tomar o conteúdo indicado e

tentará compará-lo com pacote em análise usando um algoritmo de comparação de cadeias de caracteres.

Reference. Esta palavra chave permite incluir referências à informação de identificação de ataques provida por terceiros; por exemplo, *URLs* para "*Bugtraq*", "*McAfee*", e os códigos de fabricantes ou identificação dos vendedores.

Classtype: attempted_admin. Existe uma classificação dos ataques para permitir aos usuários entenderem rapidamente e priorizar cada ataque. Cada classificação tem uma prioridade, que permite ao usuário priorizar quais eventos ele busca por meio de um apenas um número: 1 para alto, 2 para médio e 3 para baixo, sendo essas classificações não usadas nas definições das regras.

Sid: 344. Este é o identificador único para a regra do *Snort*. Todas as regras do *Snort* têm um número único de identificação. Informação sobre a regra pode ser verificada via www.snort.org/snort-db. O *SID* é também utilizado por programas de relatórios para identificar as regras.

Rev: 4 Esta seção das opções refere-se ao número de versão para a regra. Quando as regras do *Snort* são propostas pela comunidade "*open-source*", as regras passam por um processo de revisão. Ao longo do tempo, este processo permite às regras serem refinadas e a evitar falsos positivos.

2.3.21.5 Requisitos e instalação do *Snort*

A instalação do *Snort* foi feita no *Linux Ubuntu* versão 12.04 em uma máquina virtualizada com os seguintes complementos: *ACIDBASE* (*Analysis Console for Intrusion Detection*) servindo como interface gráfica para visualização dos ataques, *Apache2*, *Mysql-server* e *PHP5*. Foi escolhido o *Ubuntu* devido suas funcionalidades e também por ser um *software* livre e gratuito. Essa máquina foi configurada com 2 GB de memória *RAM* e alocada 40 GB para armazenamentos das informações, após feitas as instalações atualizou-se o sistema.

Para trabalhar-se com o *ACIDBASE* no monitoramento de *logs* torna-se obrigatório o uso de um Sistema de Gerencia de Banco de Dados (SGBD), sendo o *Mysql* mais usado, também o uso de um servidor *web* como o *Apache* com suporte a *PHP*, devido os *scripts* do *ACIDBASE* serem escritos em *PHP*.

Comando para rodar arquivos sintéticos de *logs* da *DARPA* de 1999 para verificação dos ataques comparando com regras do *snort*:

```
Snort -r arquivo.tcpdump -c /etc/snort/snort.conf
```

Após feita as instalações devidas acima, verificamos as configurações das regras para identificar ataques, essas localizadas em um arquivo */etc/snort/snort.conf*. Os alertas gerados (*logs*) de ataques se localizavam no diretório */var/log/snort/alert*.

2.4 GERENCIAMENTO DE REDES

Embora o contexto do trabalho abranja apenas e unicamente a implementação de um sistema robusto de segurança, julga-se então ser impossível não fundamentar, discorrer e mitigar, ainda que brevemente sobre o assunto gerenciamento de redes, pois é parte correlata e integrante do ambiente simulado e inclusive de uma das propostas de implementação do trabalho.

Pois bem, no contexto mundial têm-se observado, de maneira geometricamente progressiva, a expansão do emprego das redes computacionais. Hoje, esses dispositivos não se restringem ao costumeiro *layout* homem-máquina. Diversos dispositivos se conectam e fazem parte de uma rede computacional provendo serviços ou se servindo dos mesmos. (KUROSE, 2010, p. 553)

Nesse cenário, existem profissionais constantemente preocupados em saber como estão esses serviços, seja em sua disponibilidade, na qualidade do que se fornece, sua segurança, entre outros aspectos. Esses profissionais preocupam-se, portanto, em fazer com que a rede mantenha-se cumprindo o papel para o qual ela foi configurada. Contudo, a limitação espacial do ser humano não alcança as dimensões de grandes redes, sendo necessário lançar mão de ferramentas que possam apoiar esse processo de administração dos serviços de rede. Por este motivo existem as ferramentas de gerenciamento de redes de computadores. (KUROSE, 2010, p. 553-554)

Dessa maneira, podemos definir gerenciamento de redes de computadores como as atividades de monitoração e controle do funcionamento das redes de computadores em função de informações coletadas nessa rede que mostram falhas e/ou situações anormais de funcionamento. Tais informações são oriundas da integração e coordenação de elementos de *hardware*, *software* e

humanos, que verificam em tempo real o desempenho e a qualidade dos serviços desta rede. O gerenciamento de redes de computadores pode ser dividido em 05 (cinco) áreas funcionais de acordo com o Modelo OSI, a saber: gerenciamento de falhas, configuração, contabilização, segurança e desempenho. (SANTOS, 2011, p. 9-10) (KUROSE, 2010, p. 556)

A infraestrutura de gerenciamento de redes de computadores envolve 03 (três) infraestruturas principais: a entidade gerenciadora, o dispositivo gerenciado, e o protocolo de gerenciamento de rede.

A entidade gerenciadora é o cérebro da aplicação responsável pelo gerenciamento de rede, normalmente executada do local definido como a central de operações de redes de computadores (*NOC ou Network Operations Center*). Todos os dados coletados da rede são direcionados para o NOC e lá esses dados são processados e analisados, apresentando os resultados úteis para o gerente da rede, e permitindo sua interação com os dispositivos gerenciados. (KUROSE, 2010, p. 557)

Os dispositivos gerenciados são os equipamentos de rede (incluindo seus *softwares* proprietários) que se encontram sob o controle de uma determinada rede através de seu NOC. Em cada dispositivo gerenciado existem os objetos gerenciados (por exemplo, no dispositivo placa de rede pode ser gerenciados os objetos protocolo de roteamento *RIP* e protocolo de roteamento *OSPF*). Cada objeto possui informações associadas que são coletadas dentro de uma Base de Informações de Gerenciamento (*Management Information Base – MIB*). Em cada dispositivo gerenciado também reside o agente de gerenciamento de rede, que nada mais é que um processo que funciona no dispositivo gerenciado que se comunica com a entidade gerenciadora, seja para fins de envio de dados, seja para receber comandos específicos de gerenciamento. A figura 11 ilustra um modelo de gerenciamento. (KUROSE, 2010, p. 557)

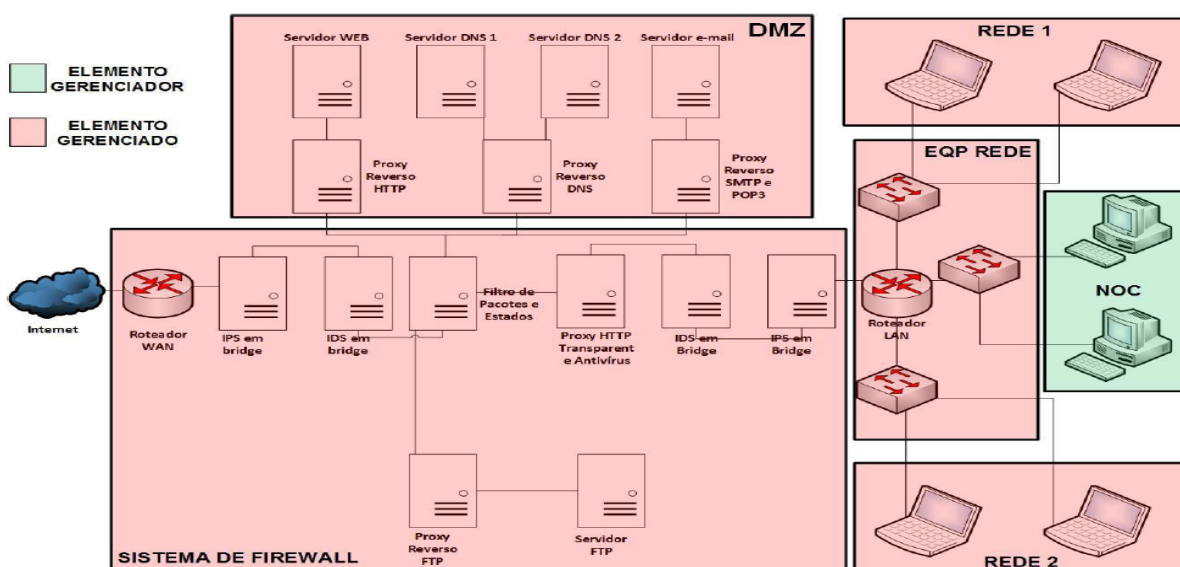


Figura 11: Modelo de Gerenciamento de Redes de Computadores
Fonte: BRASIL (2014)

O protocolo de gerenciamento de rede é o responsável por permitir a conversa entre a entidade gerenciadora e seu agente de gerenciamento de rede. Ressalta-se, aqui, que a simples existência do protocolo de gerenciamento de rede não gerencia a rede por si, mas fornece condições para que o responsável pela gerência da rede o faça. (KUROSE, 2010, p. 557)

Uma estrutura de gerenciamento padrão de redes é constituída por quatro partes que será descrito nos parágrafos seguintes: uma definição dos objetos de gerenciamento de rede (objetos *MIB*), uma linguagem de definição de dados, um protocolo e capacidades de segurança e de administração. (KUROSE, 2010, p. 557)

A linguagem de definição de dados empregada para gerenciamento de redes é a Estrutura de Informações de Gerenciamento (*Structure of Management Information – SMI*). O *SMI* é o responsável por definir as informações de gerenciamento de uma determinada entidade de rede gerenciada. Ele, portanto, estipula as regras gerais para nomear essas entidades, bem como tipifica cada entidade para torná-la particular sem, contudo, instanciar a entidade. (FOROUZAN; FEGAN, 2008, p. 579-584) (KUROSE, 2010, p. 557)

O *MIB* (figura 12) é o responsável por instanciar as entidades de rede mencionadas anteriormente, criando uma espécie de banco de dados virtual de informações que guarda os valores que refletem o estado atual das entidades de rede, e capaz de serem solicitados pela entidade gerenciadora de rede, ou

corrigidos pela mesma. Ele identifica de forma particular o elemento gerenciado permitindo singularidade dos objetos a par da grande diversidade dos mesmos. (FOROUZAN; FEGAN, 2008, p. 585-589) (KUROSE, 2010, p. 564-565)

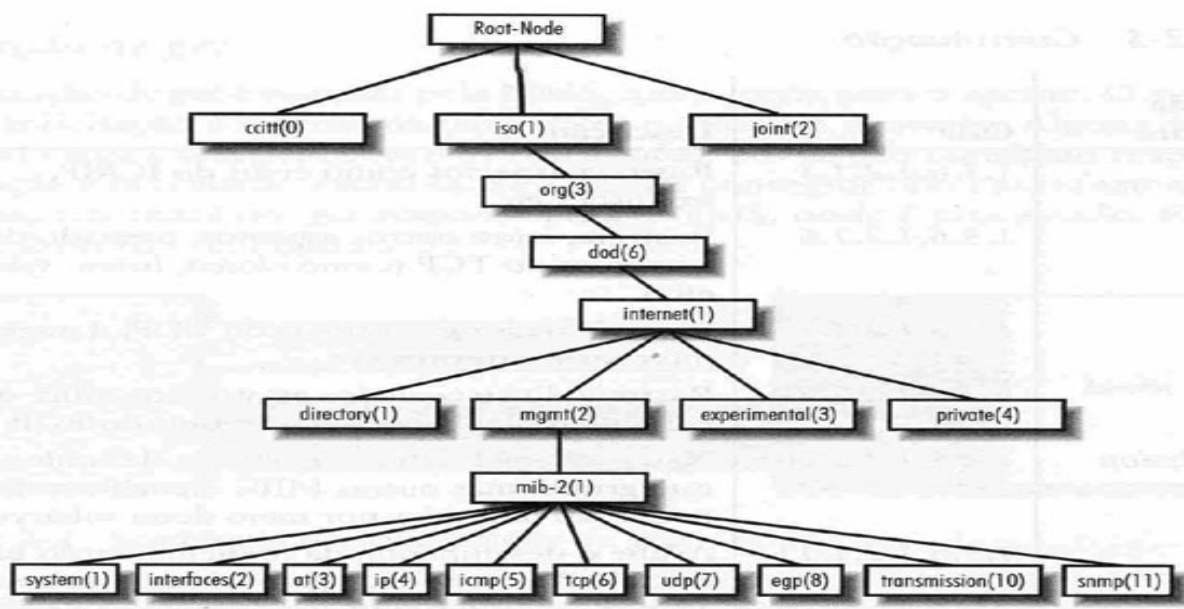


Figura 12: Árvore MIB

Fonte: SILVA (2004)

O protocolo de gerenciamento ora empregado é o *SNMP* (*Simple Network Management Protocol*), projetado em nível de aplicativo, de modo a poder monitorar dispositivos feitos por diferentes fabricantes e que se encontram instalados em diferentes redes físicas. Ele é o responsável por definir o formato das requisições e respostas que circulam entre a entidade gerenciadora e o agente de gerenciamento de rede e normalmente roda sobre *UDP* (não necessita de transporte confiável). Atualmente o protocolo *SNMP* encontra-se em sua terceira versão (*SNMPv3*), cuja principal inovação de interesse para o escopo desta pesquisa é a implementação de um modelo de segurança focado no trajeto da informação pela rede dentro das características de tráfego do *SNMP*. Tal modelo fornece os serviços de autenticação (protocolos *HMAC-MD5-96* e *HMAC-SHA-96*), privacidade (protocolo *DES* em modo *CBC*) e mecanismos de temporização para proteção contra atrasos e reenvios de mensagens. Acrescenta-se, ainda, que o *SNMPv3* foi desenhado dentro de uma arquitetura modular (diferentemente de seus antecessores), cujos módulos interagem provendo serviços uns aos outros, melhorando assim sua capacidade de administração. (COMER, 2007, p. 541)

(KUROSE, 2010, p. 565-568) (FOROUZAN; FEGAN, 2008, p. 575-576) (SANTOS, 2011, p. 68)

2.4.1 Gerenciamento de segurança

A modernidade trouxe consigo o uso cada vez mais intenso de tecnologias e serviços que se servem das potencialidades e versatilidades de uma rede de computadores. Contudo, isto também atraiu pessoas de má índole para se servir das deficiências encontradas nessas redes e, portanto, auferir algum tipo de vontade sobre a pessoa alheia. No mundo contemporâneo, o bem mais valioso existente, apesar de intangível, é a informação. (BARROS; GOMES; FREITAS, 2011, p. 15-16)

A informação, nesse caso, está materializada nos seus respectivos ativos, que nada mais são do que as estruturas físicas responsáveis por seu armazenamento, processamento e transmissão. Para que esses ativos tenham um grau de segurança aceitável, deve-se sustentá-los com os pilares da segurança da informação e das comunicações, a saber: disponibilidade (garantia de que a informação estará disponível a qualquer momento para quem tenha autorização de acesso), integridade (garantia de que a informação não foi modificada), confidencialidade (garantia de que somente pessoas autorizadas tenham acesso aos conteúdos segmentados por atribuições) e autenticidade (garantia de procedência da informação). (BARROS; GOMES; FREITAS, 2011, p. 17-18)

Inicialmente, cabe aqui a apresentação da diferença entre falhas e agressões. Falhas são defeitos eventuais que podem comprometer o funcionamento de um sistema sem, contudo, ter sido fruto de um dolo externo à rede que se está gerenciando. Agressões são as ações dolosas, normalmente realizadas por um agente externo, que têm o objetivo claro de atingir qualquer um dos pilares da segurança da informação e das comunicações. (PINHEIRO, 2002, p. 77)

As ações realizadas através das agressões aos sistemas (mais uma vez, o trabalho está vocacionado ao gerenciamento de segurança, e não no de falhas) possuem dois objetivos principais: ter acesso ilegítimo à uma determinada informação, ou se valer, de alguma forma, da capacidade de processamento do alvo, seja para empregá-la, ou mesmo para deteriorá-la. (PINHEIRO, 2002, p. 77)

O sistema de gerenciamento de redes em si também se constitui um alvo interessante para os propensos atacantes, que podem estar monitorando o tráfego de pacotes do protocolo *SNMP* com fins a entender como o sistema está funcionando (escuta passiva) ou mesmo com a finalidade de alterar essas informações (escuta ativa), e também se fazendo passar por uma entidade gerenciadora ou por um agente do sistema (mascaramento). (PINHEIRO, 2002, p. 79-81)

Os requisitos de proteção que atenderão a todo e qualquer sistema são baseados, em sua totalidade, nos pilares da segurança da informação e das comunicações já mencionados (disponibilidade, integridade, confidencialidade e autenticidade). A tecnologia empregada é indiferente quando se possui princípios claros a serem seguidos. Contudo, recomenda-se que para fins de gerenciamento de redes de computadores, a troca de mensagens entre a entidade gerenciadora e os agentes seja encapsulada por uma interface de segurança. (PINHEIRO, 2002, p. 81-85)

2.4.2 Gerenciamento de desempenho

O gerenciamento de desempenho ajuda o administrador de rede a verificar o consumo das capacidades da rede por seus usuários em função das tendências de uso dos componentes da rede mostrada em determinados períodos. Isto significa que este tipo de gerenciamento objetiva dar suporte às necessidades finais dos usuários dos serviços disponibilizados ao identificar que os recursos computacionais podem estar se esgotando, ou mesmo, que podem ser realocados. Tudo de maneira a otimizar o emprego do material.

Esse tipo de gerência é importante para o presente trabalho em razão das plataformas de comando e controle, que hoje tem sua base numa rede de computadores, possam-se manter atendendo as expectativas de seus usuários, retornando as informações necessárias no mais curto espaço de tempo. (PINHEIRO, 2002, p. 61-62)

2.4.3 Ferramenta de gerenciamento

2.4.3.1 Apresentação do Zabbix

O Zabbix é um sistema *Open Source* do tipo *NMS (Network Management System)*, possuindo assim sofisticação e funcionalidades avançadas que o torna indicado para vários cenários de gerenciamento, cujo esquema simplificado de configuração pode ser visto na figura 13. (SANTOS, 2011, p. 119)

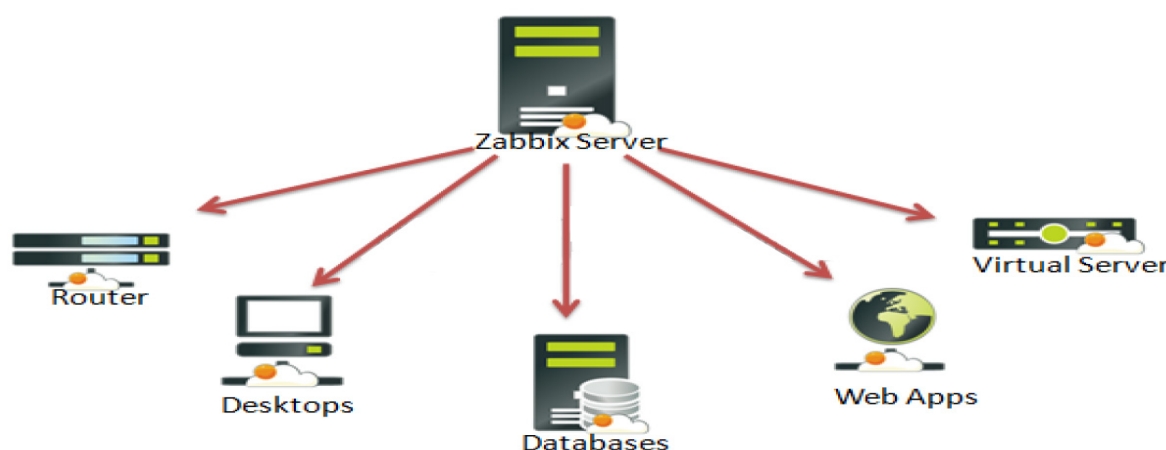


Figura 13: Esquema simplificado de configuração do Zabbix
Fonte: KUMAR (2016)

2.4.3.2 Características principais do Zabbix

- Interface *web*;
- Suporte aos sistemas *Linux*, *Solaris*, *HP-UX*, *AIX*, *FreeBSD*, *OpenBSD*, *NetBSD*, *MacOS X*, *Windows*;
- Agentes nativos para maioria dos Sistemas Operacionais;
- Capacidade de monitorar diretamente *SNMP (v1, v2, v3)* e dispositivos *IPMI*;
- Integração com banco de dados *Mysql*, *POSTGRES* e *Oracle*;
- Geração de gráficos;
- Envio de alertas por *e-mail* e *SMS*;
- Permite customização (Zabbix, 2015).

Composto pelo servidor Zabbix (como componente central do sistema, é responsável pela verificação remota dos serviços de rede por checagem simples ou por ação dos agentes), banco de dados (onde as informações coletadas e as

configurações são armazenados), interface *web* (meio pelo qual as informações são validadas e o aplicativo é configurado), agente (aplicação local da ferramenta no objeto gerenciado que acompanha ativamente o objeto com atualizações enviadas para o servidor), *proxy Zabbix* (opcional, coleta dados de desempenho e disponibilidade de um servidor *Zabbix*, com a vantagem de coletar milhares de informação por segundo), e *Java Gateway* (suporte ao monitoramento de aplicações *JMX* (*Java Management Extensions*)). (HORST; PIRES; 2015, p. 21-22)

Concernente à sua arquitetura, o cérebro do sistema (servidor *Zabbix*, interface *web* e banco de dados) podem ficar instalados em uma mesma máquina. Contudo, a instalação em máquinas diferentes é interessante para fins de desempenho no processamento e, também, para aproveitar estruturas pré-existentes no sistema a ser monitorado, como Servidor de Gerenciamento de Banco de Dados (SGDB) e Servidor *web*. Os objetos gerenciados não controlam o que está sendo monitorado, pois toda a configuração fica no servidor. O controle no objeto gerenciado são as permissões do sistema operacional, que normalmente são as permissões dos usuários e, portanto, as permissões do Agente *Zabbix*. Nessa arquitetura é interessante lembrar que dispositivos remotos também são gerenciáveis pelo *Zabbix*, sendo necessário que o *proxy Zabbix* seja instalado na rede controlada por esse *firewall*. (HORST; PIRES; DÉO, 2015, p. 23-24)

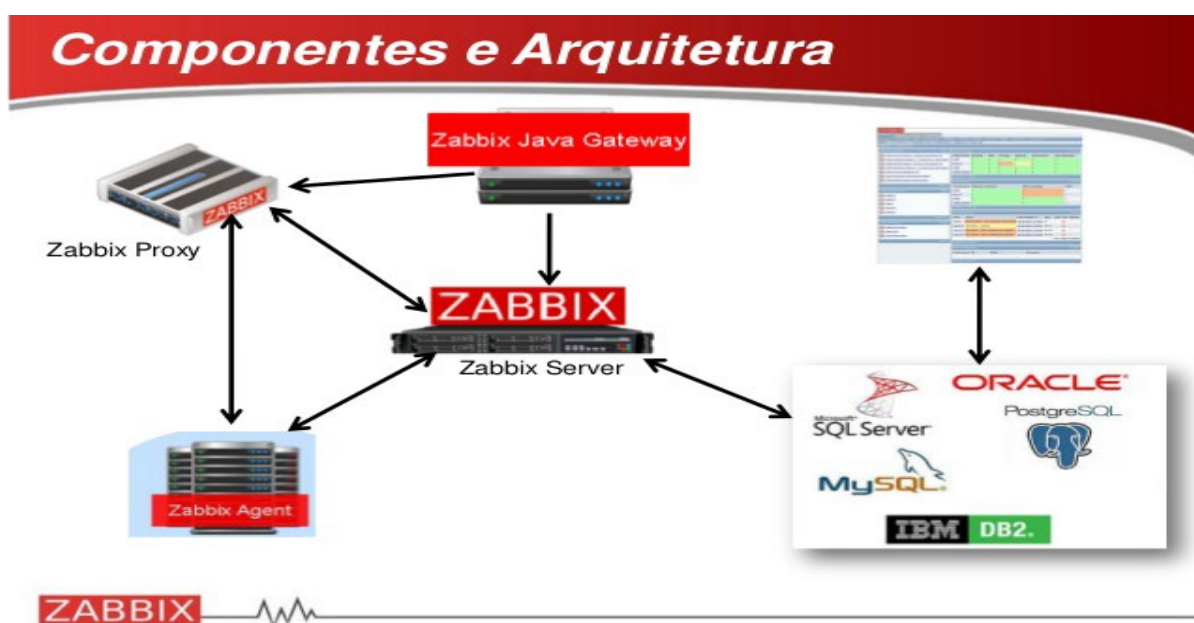


Figura 14: Arquitetura Zabbix
Fonte: PIRES (2015)



Figura 15: Arquitetura Zabbix (Com Sistema Firewall)
 Fonte: PIRES (2015)

A comunicação entre o agente e o servidor *Zabbix* é realizada através das portas 10050/TCP (Agente), 10051 (*Trapper*) e, quando o componente *Java Gateway* está em uso, a porta 10052/TCP. Para fins de monitoramento específico, o *Zabbix* ainda utiliza as portas dos protocolos *SNMP* (161/UDP) e *IPMI* (623/UDP). A comunicação entre o servidor *Zabbix* e seu respectivo agente também pode ser de forma passiva (servidor solicita ao agente um dado) ou de forma ativa (o agente informa ao servidor diretamente). O servidor também pode não utilizar o agente para coletar informações, indo, portanto, diretamente no objeto gerenciado, nas portas lógicas específicas em que funcionam os serviços e realizando sua consulta. É a chamada verificação simples que pode lançar mão dos protocolos *SNMP*, *SSH*, *TELNET* e *JMX*, uma vez que alguns sistemas podem, também, não suportar o agente *Zabbix*. Tais possibilidades tornam a ferramenta bastante completa. (HORST; PIRES; DÉO, 2015, p. 24) (JONCK; CATARINA, 2014, p. 169-170) (MOHR, 2012, p. 28-29)

A segurança no controle de acesso ao servidor *Zabbix* possui 03 (três) fases, a saber: validação do usuário, verificação do perfil e verificação de grupos. Para que isso seja possível são levantados 02 (dois) grupos de controle. O primeiro identifica o perfil de acesso e busca identificar quais módulos *Zabbix* um determinado usuário pode acessar, enquanto que o outro coloca os usuários em

grupo de *hosts*, onde são definidos quais *hosts* o usuário pode acessar. Os perfis de acesso são definidos nos grupos usuários, administrador e super administrador. Os módulos da ferramenta e os *hosts* disponíveis para acesso são, portanto, definidos sempre a nível de grupo, e nunca individualmente. Cabe ressaltar, oportunamente, que o *Zabbix* permite integração com o protocolo *HTTP* (*Apache*) e o padrão *LDAP* para fins de autenticação. Como incremento dessa segurança, devem ser adotadas as boas práticas de programação concernentes aos serviços comuns (Servidor *web*, banco de dados e sistema de *firewall*), além do tunelamento entre o Servidor *Zabbix*, seus agentes e seus *proxies*. (HORST; PIRES; DÉO, 2015, p. 134-145, 392-402)

Além dessas possibilidades, o *Zabbix* dispõe das funcionalidades a seguir, segundo Horst, Pires e Déo (2015, p. 20-21): autodescoberta de dispositivos de rede; autodescoberta de recursos do *host*; monitoramento distribuído com administração centralizada; monitoramento com ou sem o uso de agentes; suporte ao *SNMP*; autenticação segura do usuário; permissões flexíveis; auditoria; monitoramento de aplicações *web*, *java* e de ambientes virtualizados; envios de alerta por *e-mail*, *SMS*, entre outros.

2.4.3.3 Gerente *Zabbix*

É o componente central da solução e, em ambientes centralizados, os agentes enviam os dados coletados (sobre integridade, disponibilidade e estatísticos) para ele” (VLADISHEV, 2016, Cap. 1).

O gerente é responsável pelo gerenciamento e pela coleta e recebimento de dados, pelo cálculo do estado das *triggers*, e pelo envio de notificações aos usuários (VLADISHEV, 2016, Cap. 2).

“O gerente também pode executar por si só verificações remotas nos dispositivos monitorados, estas verificações ocorrem quando se utiliza itens do tipo “verificação simples”” (VLADISHEV, 2016, Cap. 2).

2.4.3.4 Agente Zabbix

O Agente *Zabbix* é o componente da solução de monitoramento que “é instalado no dispositivo alvo da monitoração. Possui capacidade de monitorar ativamente os recursos e aplicações locais (discos e partições, memória, estatísticas do processador, etc)” (VLADISHEV, 2016, Cap 2).

“O agente concentra as informações locais sobre o dispositivo monitorado para posterior envio ao servidor ou *proxy Zabbix* (dependendo da configuração)” (VLADISHEV, 2016, Cap 2).

Em caso de falhas ou mau funcionamento em algum componente do dispositivo monitorado, “o servidor *Zabbix* pode alertar ativamente os administradores do ambiente sobre o ocorrido” (VLADISHEV, 2016, Cap. 2).

2.4.3.5 Tipos de verificação

Existem duas formas utilizadas pelos agentes para realizar as verificações (VLADISHEV, 2016, Cap. 2):

- Verificação Ativa: “O agente precisa primeiro receber a lista de itens a monitorar e o intervalo entre coletas pretendido. Esta informação vem do servidor *Zabbix* através de requisições periódicas do agente”;
- Verificação Passiva: “O servidor ou o *proxy Zabbix* requisitam o dado toda vez que é necessário (uso de *CPU*, memória, disco, etc), o agente responde com o resultado do teste solicitado”.

2.4.3.6 Proxy Zabbix

O *proxy Zabbix* é um servidor *Zabbix* configurado para funcionar como “um processo que pode receber dados de um ou mais dispositivos monitorados e enviar ao servidor *Zabbix*”, assim sendo, para os agentes monitorados “o *proxy* passa a ser o servidor *Zabbix*”. Os dados de monitoramento coletados dos agentes são armazenados no *proxy* e posteriormente enviados ao servidor *Zabbix* ao qual ele pertence, sendo excluídos na sequência (VLADISHEV, 2016, Cap. 2).

“A utilização deste componente é opcional, mas normalmente é muito benéfica pois distribui a carga de monitoração normalmente atribuída ao *Zabbix server*” (VLADISHEV, 2016, Cap. 2).

“O *proxy Zabbix* é a solução ideal para a monitoração centralizada de localidades geograficamente dispersas e para redes gerenciadas remotamente” (VLADISHEV, 2016, Cap. 2).

O uso do *proxy Zabbix* requer um banco de dados local, que pode ser *SQLite*, *Mysql* ou *PostgreSQL*, que são suportados nativamente. Ainda assim, é possível utilizar outros bancos de dados, como o *Oracle* ou *IBM DB2*, porém, nestes casos os riscos e limitações devem ser analisados criteriosamente (VLADISHEV, 2016, Cap. 2).

Na figura 16 vemos um exemplo de utilização de um *proxy Zabbix* utilizado para monitorar um conjunto remoto de servidores.

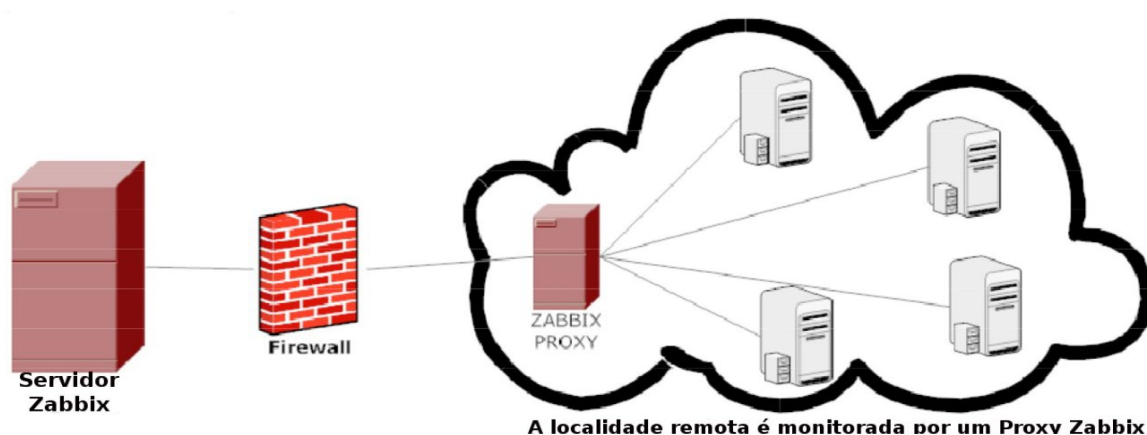


Figura 16: Modelo de utilização do *proxy Zabbix*

Fonte: *Zabbix Brasil* (2016)

2.4.3.7 Interface web

Levando em conta que os dispositivos e serviços a serem monitorados estão em um ambiente de rede, a aplicação *Zabbix* conta com uma interface web, acessível através de qualquer navegador, que foi idealizada e desenvolvida “para possibilitar um fácil acesso a partir de qualquer plataforma” e possibilitar o gerenciamento do *Zabbix* através desta interface” (VLADISHEV, 2016, Cap 17).

Na figura 17 pode-se ver a tela principal do servidor *Zabbix*, onde visualiza-se o '*Dashboard*' da aba '*Monitoramento*', página que mostra de forma prática e agrupada, um resumo das ocorrências de todo o ambiente monitorado.

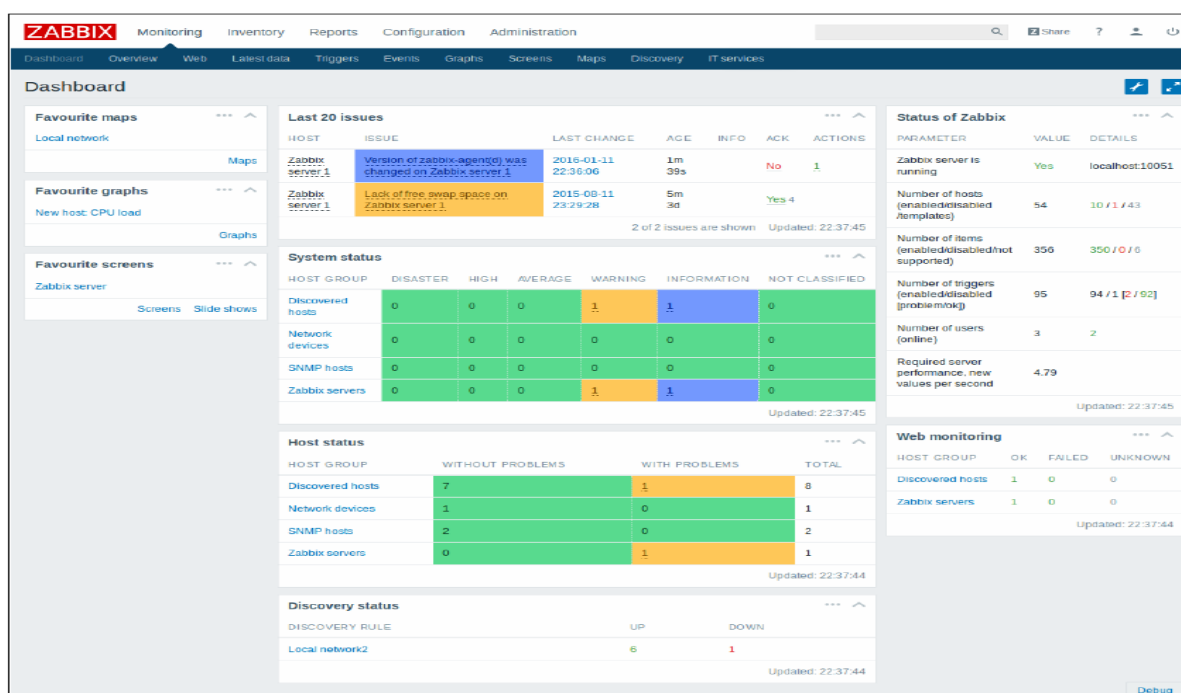


Figura 17: Interface Web do servidor Zabbix
Fonte: Zabbix Brasil (2016)

2.4.3.8 Gestão do desempenho

A gestão do desempenho está diretamente relacionada com a segurança, dado que os ataques realizados contra um sistema, comprometem diretamente o funcionamento do mesmo, principalmente os de negação de serviço. No entanto, é importante mencionar nesse momento que o *Zabbix* possui capacidade de monitorar muitos parâmetros do sistema, como carga do processador, atividade de disco rígido e disponibilidade de memória, e disponibilizá-los sob a forma de gráfico de tendências, mais eficientes até mesmo que os gerados por outros *softwares* como por exemplo pelo *NAGIOS*, por utilizar o pacote *RRDTool* (o mesmo do *CACTI*) no lugar do *MRTG*. (ROCHA; SERRADOURADA, 2008, p. 46)

2.5 HONEYPOTS

O recurso computacional de segurança dedicado a ser sondado, atacado ou comprometido é chamado de *honeypot* (HOEPERS *et al.*, 2012), pois simulam um ambiente real de produção, mas possuem mecanismos de contenção, de alerta e de coleta de informações dos atacantes que alivia a carga dos ataques direcionados ao ambiente real de produção, não o comprometendo.

Utilizando recursos para simular diversos serviços como, *FTP*, servidor de *e-mail*, *SSH*, dentre portas utilizadas por alguns programas, um *honeypot* tem por objetivo receber todo o tráfego malicioso direcionado a um ambiente de produção e impedir que esse tráfego chegue à *internet*, devolvendo somente respostas inofensivas.

O *honeypot* pode ser utilizado para pesquisa ou para produção. Quando utilizado para o propósito de pesquisa, o *honeypot* tem por objetivo coletar informações como novas ferramentas, *malwares* e táticas utilizadas pelos atacantes. Quando utilizado para o propósito de produção, o *honeypot* é usado para a segurança de outros ambientes reais e assim protegendo a organização do real poder de um ataque. Esse tipo de *honeypot* pode proteger a organização de três modos, sendo eles: prevenção, detecção e resposta (SPITZNER, 2003).

- **Prevenção:** Voltado para prevenção de ataques. O *honeypot* trabalha dificultando o acesso do atacante, fornecendo respostas mais lentas ou resposta pré-determinadas pelo administrador do *honeypot*, confundindo o atacante e fazendo com que perca interesse ao alvo.
- **Detecção:** Importante para a detecção de atividade maliciosa, para que os administradores do *honeypot* possam rapidamente tomar uma decisão sobre a ação a ser tomada, interrompendo ou minimizando possíveis danos ao ambiente de produção.
- **Resposta:** Para coletar informações necessárias, como ferramentas utilizadas pelo atacante, endereço *IP* e como ele agiu no sistema, com a finalidade de responder ao ataque.

Apesar do conceito ter sido criado há mais de uma década, só recentemente aplicações comerciais e artigos publicados sobre o conceito vem surgindo. Segundo Spitzner (SPITZNER, 2003), a história do *honeypot* pode ser resumida na seguinte lista:

- 1990/1991 – Surgem as primeiras publicações sobre o conceito de *honeypot*. Em especial os artigos publicados por Bill Cheswick (1990) e Clifford Stoll (1990) se destacam, onde em ambos os casos foi descrita o acompanhamento de uma invasão em um sistema de produção até que a identidade do invasor fosse obtida.

- 1997 – Criação da primeira solução de *honeypot* para a comunidade de segurança, a *Deception Toolkit (DTK)* desenvolvida por Fred Cohen.
- 1998 – É criado o primeiro *honeypot* comercial vendido ao público, chamado de *CyberCop Sting*, foi a primeira a introduzir o conceito de múltiplos sistemas virtuais em um único *honeypot*. Nesse momento também surgem ferramentas mais amigáveis que tornou o *honeypot* mais popular, como o *backofficer friendly*, que é uma ferramenta grátis, de uso simples e roda em *Windows*.
- 1999 – Formação do *honeynet project*, onde um grupo se dedicaria para estudar toda a comunidade de atacantes maliciosos para aprender sobre novas técnicas e métodos de invasão. Nesse momento é de grande evolução para a tecnologia *honeypot*.
- 2000/2001 – Empresas passam a usar *honeypot* para estudar toda a atividade de *worm* e detecção de novas ameaças.

Com o crescimento da *web*, o desenvolvimento do *honeypot* voltado para esse meio é intenso, possibilitando que uma aplicação completa para web seja simulada e dados com informações de ataques sejam capturados e analisados. Dessa maneira torna-se uma ferramenta importante, pois cada vez mais as aplicações locais, passam a se tornar públicas na *web*, como *sites* de compras, sistemas de busca e grandes sistemas *ERP*.

2.5.1 Abrangência, vantagens e desvantagens

Contendo em seu próprio conceito o real valor, um *honeypot* permite na implementação que todo o tráfego direcionado a ele, serem decorrências de ataques ou anomalias identificadas na rede, dessa maneira há uma diminuição de falsos positivos. Essa abrangência do *honeypot* confronta diretamente outras ferramentas de segurança, como o *IDS* e *firewall*, que trata o tráfego como um todo.

Outras de suas principais vantagens, estão na implementação e na captura dos dados. A implementação do *honeypot* é relativamente simples e econômica, pois não há necessidade de desenvolver algoritmos complexos, nem mesmo para trabalhar com dados criptografados e com *IPV6*, e não há necessidade de que o servidor tenha um *hardware* robusto, podendo ser virtualizado.

Quanto à captura dos dados, em teoria, os falsos negativos, não são capturados obtendo assim uma diminuição nos dados que são armazenados. Em

um servidor que receberia todos os dados teriam 5.000 alertas em um arquivo de 10 GB no *honeypot* teriam 30 alertas em um arquivo de 10 MB, simplificando a análise e dando uma resposta mais rápida ao ataque.

Uma das desvantagens de um *honeypot* está na sua visão limitada dos dados que podem ser analisados, já que somente atividades que são direcionadas a ele podem ser analisadas, não identificando, por exemplo, ataques com foco a um ponto específico da rede. É possível também que um atacante identifique o padrão de respostas de um *honeypot* e direcione o ataque para outro ponto da rede.

O risco é outro ponto de desvantagem para um *honeypot*, pois a partir do momento que se temo *honeypot* comprometido por um atacante, pode ser utilizado para afetar outros pontos da rede da organização. Toda vez que ao adicionar novos recursos ao *honeypot* utilizando um endereço IP, um risco é assumido e certas medidas de contenção devem ser tomadas para que eles sejam minimizados. O risco pode variar de acordo com cada nível de interatividade de *honeypot*.

2.5.2 Classificação por meio de níveis de interatividade

Tabela 3: Comparação de *honeypot* de baixa, média e alta interatividade

Característica	Baixa Interatividade	Média Interatividade	Alta Interatividade
Instalação	Fácil	Envolvido	Difícil
Configuração	Fácil	Envolvido	Difícil
Manutenção	Fácil	Envolvido	Difícil
Coleta de Informações	Limitada	Variável	Extensa
Risco	Baixo	Médio	Alta

Fonte: HOEPERS (2007)

No *honeypot* de baixa interatividade são instaladas ferramentas que simulam um ambiente real, como um *FTP* ou serviços *HTTP*, onde não é permitido que o atacante interaja com o servidor, sendo que, para isso, é necessário que o sistema operacional seja instalado e configurado de modo seguro. Como o atacante não consegue, em teoria, controle do sistema operacional e seu nível de

complexidade de implantação é menor, esse tipo se torna o mais recomendado para a utilização nas organizações, pois consegue somente detectar tipos já conhecidos de ataques.

Já um *honeypot* de alta interatividade permite que o atacante tenha controle total sobre o sistema operacional e os serviços nele contido, onde os mesmos não são apenas simulados, como em um *honeypot* de baixa interatividade. Este tipo é o mais arriscado e de maior complexidade de implantação, porém oferece uma gama maior na coleta dos dados. Este método de interatividade é mais utilizado para o propósito de pesquisa.

O *honeypot* de média de interatividade, assume características dos *honeypots* de baixa e alta, podendo ter mais interação que os de baixa, mas menor que os de alta. Mesmo tendo um nível de interatividade maior, os serviços não se equivalem a um sistema real, sendo que possam ser simulados, porém não somente respondendo a testes de conexão, mas acrescentando informações fazendo com que o atacante interaja mais com o ambiente. Com esse nível de interação é possível que o *honeypot* seja utilizado para o propósito de produção quanto o de pesquisa, pois reúne mais informações.

2.5.3 Apresentação do *HoneyD*

Desenvolvido e mantido desde abril 2002 por *Niels Provos* na Universidade de Michigan (PROVOS, 2002). O *HoneyD* é uma ferramenta para *honeypots* de baixa interatividade que como as demais ferramentas, simulam serviços virtuais como *FTP*, *HTTP* e *SSH* em uma rede, utilizando *scripts* com respostas pré-programadas. Inicialmente pode ser utilizado para produção, porém possui aplicações específicas para pesquisa podendo simular todas as portas disponíveis em uma rede.

O *honeypot* foi desenvolvido seguindo as práticas de *software* livre, por esse motivo, é uma ferramenta grátis e de código totalmente aberto, onde pode ser customizado de acordo com a necessidade. O que é muito importante para que seja adequado a cada realidade, diferentes de outros tipos de *honeypots*.

Em seu desenvolvimento o *HoneyD*, introduziu vários novos conceitos para *honeypots*. Um desses conceitos está na resposta de ataques, já que o *HoneyD* não responde somente a ataques direcionados a o *IP* dele, podendo

assumir *IPS* da rede que não estão sendo utilizados para outros serviços. Isso diferencia das demais ferramentas de *honeypots* disponíveis no mercado.

Outro conceito do *HoneyD* é que sua arquitetura também permite, além de emular diversos serviços, emular vários sistemas operacionais ao mesmo tempo. Além disso é possível emular sistemas operacionais a nível de pilha *IP*, podendo assim responder a scans a assinatura exata do sistema operacional emulado, se comportando com um sistema operacional legítimo.

Possuindo grande capacidade de processamento, o *HoneyD* pode assumir simultaneamente milhares endereços de *IP* e interagir ativamente com o atacante. O mais provável é que sua rede se torne um grande gargalo com a grande quantidade de *IPS* e não o *honeypot*, que tem capacidade de trabalhar com redes extremamente grandes.

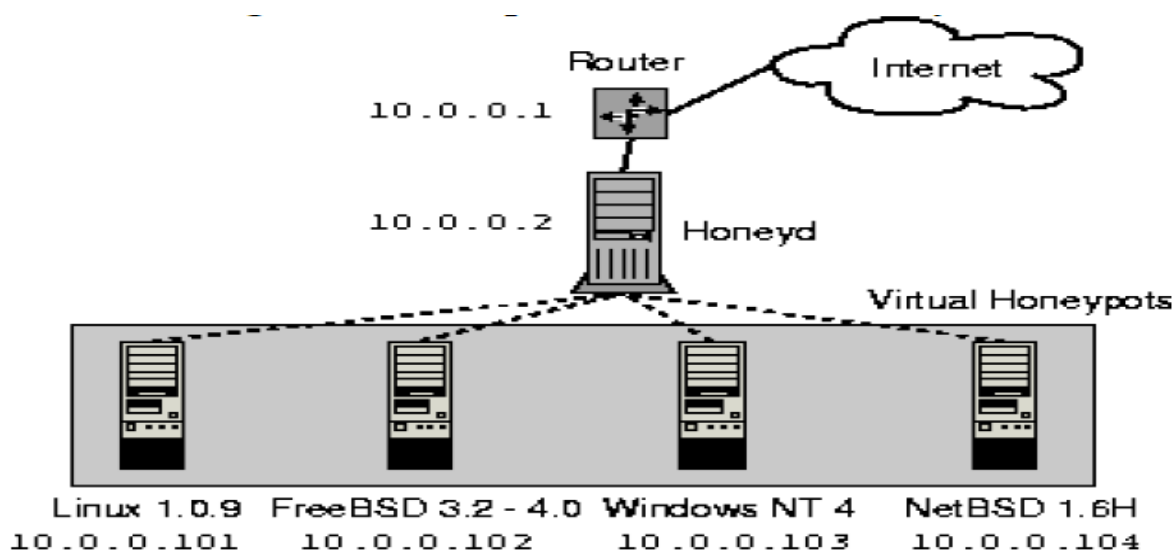


Figura 18: Topologia Honeypots

Fonte: PROVOS (2002)

3 MÉTODO E PROCEDIMENTOS

3.1 CRONOGRAMA

As atividades propostas pelos docentes e pelo grupo responsável pelo projeto serão desenvolvidas seguindo as seguintes etapas, conforme à tabela de cronograma abaixo:

Tabela 4: Cronograma de Gerenciamento

Cronograma Gerenciamento de um Ambiente Seguro em Redes de Computadores							
1 Etapa (6 meses)	Janeiro	Fevereiro	Março	Abril	Maio	Junho	Execução de atividades
Escolha de temas							Alessandro, Bruno, Geovani, Jackeline, Ailton
Análise dos temas							Alessandro, Bruno, Geovani, Jackeline, Ailton
Apresentação dos temas para o docente							Alessandro, Bruno, Geovani, Jackeline, Ailton
Levantamento de dados							Bruno, Geovani
Pesquisa de campo sobre viabilidade							Bruno, Geovani
Começo do documento							Alessandro, Bruno, Geovani, Jackeline
Pesquisas em livros							Alessandro, Bruno, Geovani
Pesquisa na internet							Alessandro, Bruno, Geovani, Jackeline
Elaboração da apresentação							Bruno
Apresentação							Alessandro, Bruno, Geovani, Jackeline, Ailton
Férias							
2 Etapa (6 meses)	Julho	Agosto	Setembro	Outubro	Novembro	Dezembro	Execução de atividades
Continuação do documento							Alessandro, Bruno, Geovani, Jackeline
Pesquisas							Alessandro, Bruno, Geovani, Jackeline
Elaboração parte prática							Geovani
Ataque cibernético a rede							Geovani, Alessandro
Análise do projeto							Alessandro, Bruno, Geovani, Jackeline, Paolo
Formatação do documento							Alessandro
Pesquisa sobre satisfação							Alessandro, Bruno, Geovani, Jackeline
Apresentação final							Alessandro, Bruno, Geovani, Jackeline
Conclusão do trabalho							Alessandro, Bruno, Geovani, Jackeline

1 Etapa	Dias aula de Projeto 1		2 Etapa	Dias aula de Projeto 2		Horas Trabalhadas	
	22/01/2019	16/04/2019		02/08/2019	27/09/2019	1 Etapa	120 Horas
	29/01/2019	17/04/2019		09/08/2019	04/10/2019	2 Etapa	64 Horas
	05/02/2019	23/04/2019		16/08/2019	11/10/2019		
	12/02/2019	24/04/2019		23/08/2019	18/10/2019		
	19/02/2019	30/04/2019		30/08/2019	25/10/2019		
	26/02/2019	07/05/2019		06/09/2019	01/11/2019		
	12/03/2019	08/05/2019		13/09/2019	08/11/2019		
	13/03/2019	14/05/2019		20/09/2019	22/11/2019		
	19/03/2019	15/05/2019					
	26/03/2019	22/05/2019					
	27/03/2019	23/05/2019					
	02/04/2019	28/05/2019					
	03/04/2019	29/05/2019					
	09/04/2019	03/06/2019					
	10/04/2019	04/06/2019					

3.2 CUSTOS

De acordo com projeto de implementação de um sistema seguro em redes de computadores, os custos do projeto só serão cobrados pela mão de obra dos técnicos envolvidos. Para a instalação em uma empresa é cobrador R\$200,00 (Duzentos reais) por hora corrida, para cada técnico envolvido. Para os procedimentos de configuração foram calculadas 18 horas. Se caso a empresa necessitar de mais recursos serão cobradas as horas a mais.

A instalação do projeto descrito tem a necessidade de um servidor de acordo com a tabela abaixo com uma infraestrutura de rede já pronta e ambas devem ser fornecidas pelo contratante.

Tabela 5: Configuração das máquinas a se utilizar

Processador	Intel® Xeon® E-2124 de 3,3GHz, cache de 8MB, 4 núcleos / 4 segmentos, com turbo (71W)
Memória RAM	16GB UDIMM DDR4 de 2666 MT/s
Hd	2TB SATA cabeado, 6 Gbps, 7200 RPM e 3,5"
Placas de rede adicionais	2 placas de rede

3.3 TIPO DE PESQUISA

A pesquisa será feita através de uma ferramenta chamada “*Google Forms*”, que possibilita gerar um *link* para ser mandado aos colaboradores da pesquisa onde se cadastrarão e responderão as perguntas envolvidas no projeto, pesquisa feita em prol da viabilidade e consistência de argumentos para a necessidade da implementação na rede.

3.4 CARÁTER DA PESQUISA

Ao utilizar o método qualitativo, o dado obtido através da pesquisa demonstrará a necessidade de uma implementação na rede de uma empresa corporativa.

Tendo como a principal função minimizar ao máximo as chances de ataques com sucesso de *hackers* ou até mesmo eliminá-las por completo.

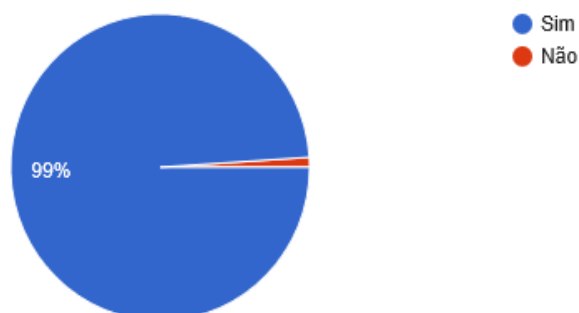
3.5 MÉTODO DE ABORDAGEM

A coleta de dados será feita com pessoas de empresas e pessoas que trabalham muito com a utilização da *internet*.

3.6 DADOS DA PESQUISA DE CAMPO

Você utiliza internet?

96 respostas



Pesquisa feita com alunos do senai envolvendo 96 integrantes como mostra os graficos abaixo:

Gráfico 1: Uso da *internet*
(Fonte: Autoria e pesquisa do grupo)

De acordo com o gráfico1 nota-se que 99% utilizam a *internet*.

Você sabe dos riscos que suas informações correm quando conectado à internet?

95 respostas

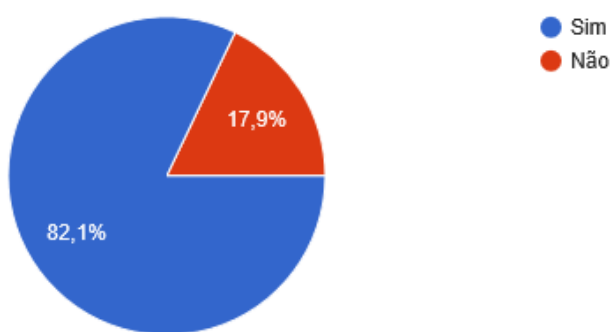


Gráfico 2: Conhecimento sobre os riscos de uso da *internet*
(Fonte: Autoria e pesquisa do grupo)

De acordo com o gráfico 2, pode-se notar que 82,1% sabe dos riscos e 17,9% não sabem dos riscos na *internet*.

Você tem computador na sua casa?

96 respostas

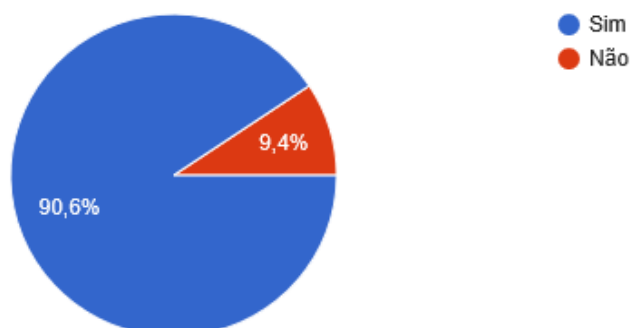


Gráfico 3: Uso de computador na residência
(Fonte: Autoria e pesquisa do grupo)

De acordo com o gráfico 3, pode-se notar que 90,6% contem computador na residência onde mora e 9,4% não tem.

Você gostaria de navegar na internet em um ambiente onde suas informações são seguras e confiáveis?

96 respostas

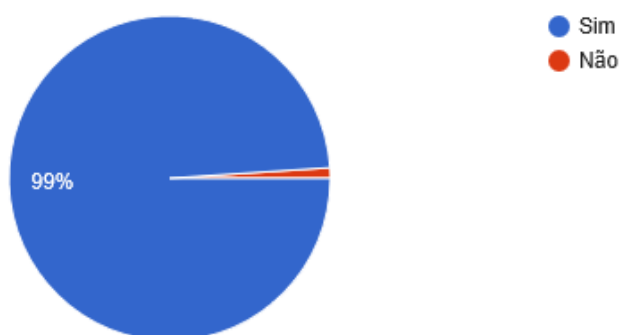


Gráfico 4: Preocupação com a segurança em uma rede de computadores
(Fonte: Autoria e pesquisa do grupo)

De acordo com o gráfico 4, pode-se notar que 99% querem ter segurança e 1% não querem ou não sabem os riscos que correm.

uma escala de 1 a 5, qual o seu conhecimento técnico sobre a segurança de sua residência?

postas

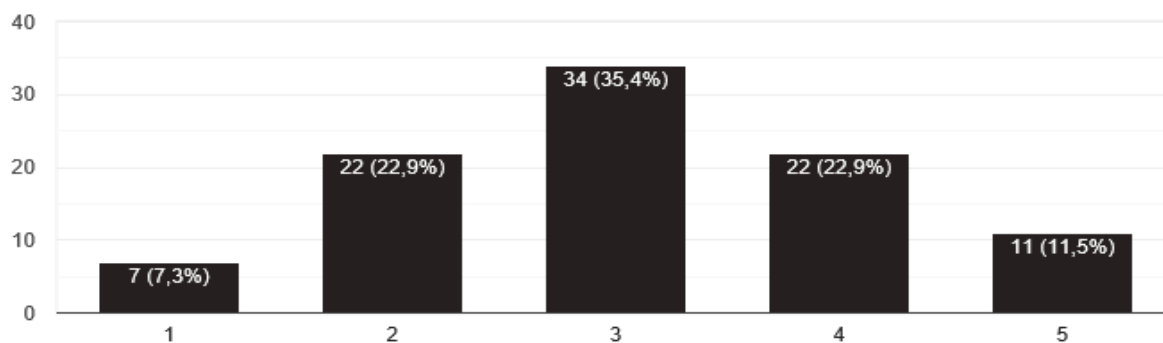


Gráfico 5: Conhecimento técnico da rede na residência
(Fonte: Autoria e pesquisa do grupo)

De acordo com o gráfico 5, a maioria dos entrevistados tem um conhecimento regular da rede que estabelece em sua residência.

Você utiliza alguma ferramenta para a segurança do seu dispositivo?

96 respostas

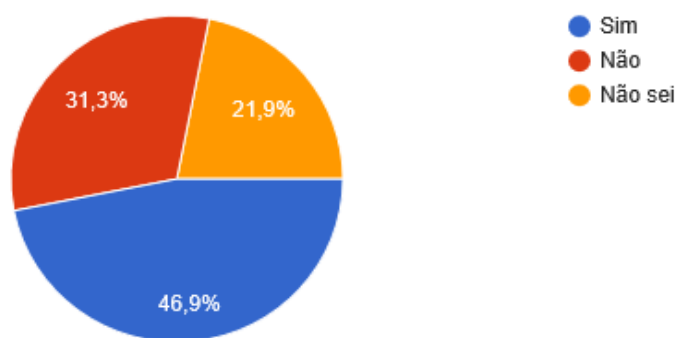


Gráfico 6: Uso de ferramentas para a segurança do dispositivo
(Fonte: Autoria e pesquisa do grupo)

De acordo com o gráfico 6, pode-se notar que 46,9% usam ferramentas para a segurança de informações, 31,3% não usam e 21,9 não sabem se usam.

Você já teve problemas na segurança da sua rede, como vírus, intrusos ou até mesmo roubo das suas informações?

96 respostas

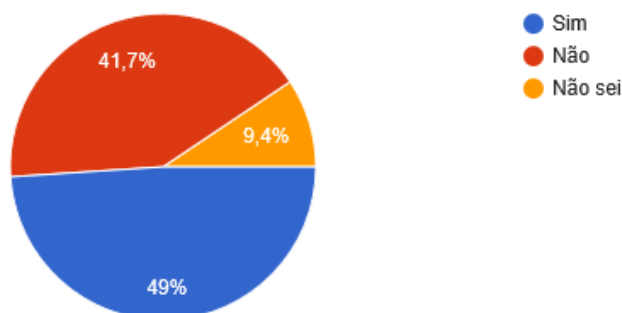


Gráfico 7: Problemas de instrução na rede com vírus e roubo de dados
(Fonte: Autoria e pesquisa do grupo)

De acordo com o gráfico 7, pode-se notar que 49% já teve problemas, 41,7% nunca tiveram e 9,4 não sabem se já teve.

Você como um empresário em potencial, com uma quantidade considerável de computadores na sua empresa, investiria em um serviço e projeto de segurança na sua rede de computadores?

96 respostas

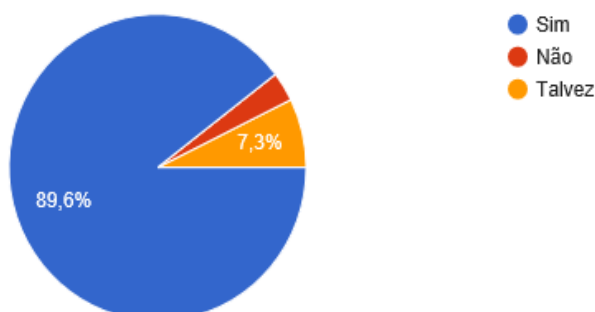


Gráfico 8: Investimento em um sistema de segurança da rede corporativa
(Fonte: Autoria e pesquisa do grupo)

De acordo com o gráfico 8, pode-se notar que 89,6% iriam investir, 3,1% não teriam esse interesse e 7,3 não sabem a necessidade.

Você acha viável a proposta do nosso projeto?

96 respostas

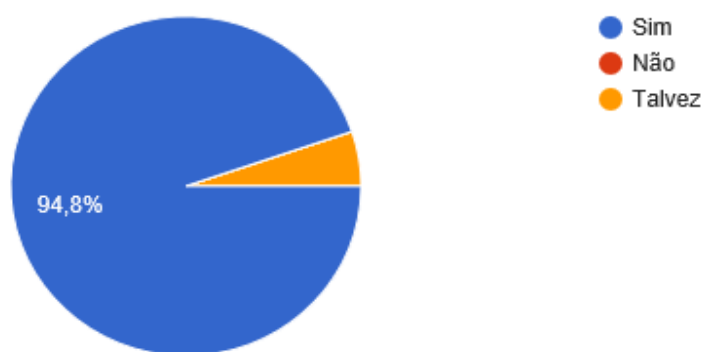


Gráfico 9: Viabilidade do projeto
(Fonte: Autoria e pesquisa do grupo)

De acordo com o gráfico 9, pode-se notar que 94,8% consideram viável o projeto e 5,2% talvez.

Pesquisa esta realizada com quatro empresas distintas que fazem o uso de dados frequentes.

A maioria das empresas possuem suas informações confidenciais armazenadas de maneira digital. Caso houvesse a perda, o comprometimento da integridade ou o roubo destas informações, quais seriam as consequências para sua empresa?

4 respostas

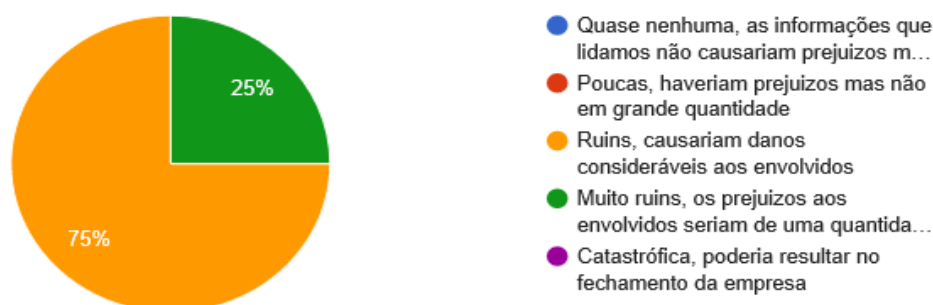


Gráfico 10: Armazenamento de arquivos
(Fonte: Autoria e pesquisa do grupo)

De acordo com o gráfico 10, pode-se notar que 75% tem os dados armazenados de maneira ruim e 25% muito ruins.

O quão integrada é a rede de computadores da sua empresa com as informações que ela lida?

4 respostas

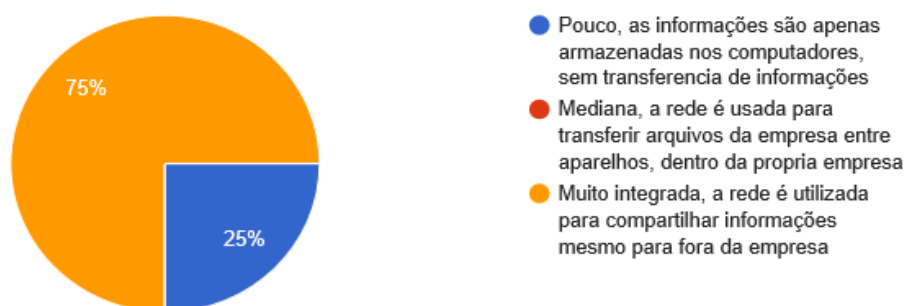


Gráfico 11: Internet integrada na empresa
(Fonte: Autoria e pesquisa do grupo)

De acordo com o gráfico 11, pode-se notar que 75% é muito integrada e 25% pouco integrada.

Você aceitaria a implementação de um ambiente com mais segurança para a rede de computadores da sua empresa, por um valor de até R\$ 15.000,00 (quinze mil reais)?

4 respostas

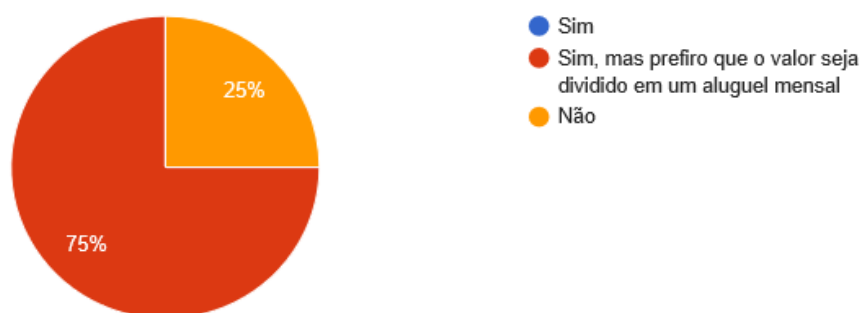


Gráfico 12: Custo da implementação
(Fonte: Autoria e pesquisa do grupo)

De acordo com o gráfico 12, pode-se notar que 75% pagaria esse valor pela segurança e 25% não pagariam.

Ao analisarmos as pesquisas, constatamos a viabilidade e a necessidade da implementação de um sistema mais robusto de segurança, evitando ataques e roubos de dados tanto no sistema corporativo ou na residência.

4 DESENVOLVIMENTO

4.1 IMPLEMENTAÇÃO

Ao longo do desenvolvimento do trabalho, após a definição de quais *softwares* gratuitos seriam utilizados no projeto pelo grupo, passado a fase de definições, partimos então para a fase de implementação do ambiente seguro, utilizando os respectivos sistemas:

- Pfsense: Um *firewall* customizado, baseado na licença *BSD* que possui capacidades equiparadas a outros *firewalls* comerciais;
- Snort: Um detector de intrusão do sistema, capaz de analisar e *sniffar* toda a rede, bem como o tráfego de dados em tempo real;
- Honeyd: Um sistema de *honeypots* de baixa-média interação, sendo um simulador eficiente de máquinas virtuais em uma rede de computadores, este é muito utilizado pela facilidade e por sua efetividade de implementação.
- Zabbix: Um sistema de monitoramento robusto de *hosts* que oferece e tem uma gama compatível com vários sistemas e equipamentos;

A princípio, os sistemas utilizados seriam implementados em um servidor de virtualização, o *XenServer*, instalado em uma máquina real e avulsa, mas devido à falta de recursos de *hardware* desta mesma máquina, os sistemas foram migrados para um ambiente virtual com o sistema *Windows 10* e utilizando o *virtualbox*, descartado assim pelo grupo, a utilização de um servidor físico e real.

A instalação foi realizada da mesma maneira nas duas alternativas, sendo a única diferença, esta atrelada à capacidade de *hardware* que a segunda máquina possui e que foi o suficiente e necessário para a realização dos testes.

Primeiramente foi realizado a instalação do *Pfsense*, devido ele possuir a capacidade de roteamento de *internet* para a rede interna, que seria necessário para os outros sistemas. A instalação do *Pfsense* não difere da instalação de um sistema operacional comum baseado em *BSD*, sendo ele instalado em uma máquina que possui duas interfaces de rede, a externa (*WAN*) e a interna (*LAN*). Toda a configuração do *Pfsense* foi executada através da interface

web que o *firewall* oferece, utilizando um navegador de *internet* em uma máquina *Windows 7* conectado na interface *LAN*.

Inicialmente o *PfSense* já vem com as configurações de *NAT* e *DNS* necessárias para que seja efetuado o roteamento da *internet*, tendo sido necessário apenas a configuração do serviço *DHCP* e as regras de tráfego de dados. O *DHCP* foi configurado para entregar endereços *IP* na faixa 200.200.200.2/24 até 200.200.200.254/24, sendo o endereço *IP* do próprio *Pfsense* atribuído a 200.200.200.1 na interface *LAN*, que servirá como *gateway* para rede. O endereço *IP* da interface *WAN* foi feito automaticamente via *DHCP*.

Uma grande parte do vazamento de informações seja em uma empresa ou até mesmo em uma residência, é a falta de criptografia das informações enviadas à *internet*, devido a alguns serviços e *sites* não utilizarem os protocolos de rede mais seguros, permitindo que, caso a informação seja interceptada, ela possa ser lida facilmente sem a necessidade de descriptografar os dados. Devido a essas vulnerabilidades, foram atribuídas à interface de rede *LAN*, várias regras que bloqueiam tentativas de acesso a protocolos que não utilizem criptografia, em suma a porta 80 do protocolo *HTTP*; os protocolos de *e-mail SMTP*, *POP3* e *IMAP* nas portas 25, 110 e 143 respectivamente; e os protocolos de acesso remoto e transferência de arquivos *Telnet* e *FTP*, nas portas 23 e 20/21.

O *Pfsense* por padrão, na instalação verificou-se que ele detém duas regras na interface de entrada *WAN*: uma regra que bloqueia conexões de redes privadas e outra que bloqueia conexões vindas de redes que não são reconhecidas pela IANA.

Devido à existência de possíveis tentativas de ataques externos através de portas e endereços que podem não ser cobertos pelas duas regras, elas foram substituídas por outras, uma regra que bloqueia toda tentativa de conexão através da porta *WAN*, e outra que caso necessário, libere apenas a porta 2222, que foi definida como porta padrão para conexão remota via *SSH*.

Após instalação do *Pfsense*, foi feito a instalação do pacote de serviços *Snort* através da própria interface de configuração do *Pfsense*. O *Snort* quando instalado desta maneira possui alguns recursos que permitem um melhor funcionamento dos dois sistemas atuando em conjunto.

O *Snort* como a maioria dos *IDS*, utiliza para realizar as detecções, pacotes de regras que são disponibilizadas de maneira gratuita e paga, que podem

ser baixadas e configuradas através da interface do *Pfsense*, além da capacidade de adição de regras customizadas. Nem todas as regras de todos os pacotes foram utilizadas durante os testes, devido já vir com muitas regras de detecções ativas, sendo muitas delas um falso positivo ocasionando em bloqueios não intencionais, o que se averiguou na ocasião que pode sim acabar ocorrendo. Para a decisão de regras ativas, foi utilizado o prefixo de seleção de regras ‘balanceado’, que equilibra conectividade e segurança para a rede.

Os pacotes de regras disponíveis para utilização, são:

- Regras Comunitárias Snort GPLv2: regras feitas pela comunidade, liberadas de maneira gratuita;
- Regras Snort VRT: regras feitas pela equipe de pesquisa de segurança do *Snort*, com atualizações bissemanais. Só é possível a utilização das mesmas, após a inscrição no *síte* do *Snort*, porem as regras tem 30 dias de atraso. Para utilizar as regras recentes de maneira imediata, é necessário também ser um inscrito pago do *Snort*.
- Regras de Ameaças Emergentes OPEN e PRO: pacotes de regras este, frequentemente atualizados para ameaças recém descobertas, sendo também necessário ser inscrito e pagar à desenvolvedora *CISCO*, para utilizar a versão *PRO*.

Além destas regras, a integração com o *Pfsense* habilita a capacidade do *Snort* de utilizar o recurso *OpenAppID*, que quando ativas, são capazes de identificar e catalogar diferentes aplicativos que utilizam a rede, como *Facebook*, *Amazon* e *Netflix*, podendo também fazer o bloqueio dos mesmos tanto pelo *Snort* e *Pfsense*.

Foram atribuídos ao *Snort*, duas interfaces de rede para monitoramento, a *WAN*, para ameaças externas e a *LAN* para ameaças internas. Na interface *WAN*, além de atribuídos as regras, foi habilitada a opção de bloqueio de endereços *IPs* que gerasse algum tipo de alerta, mas na interface *LAN*, como ainda existiam as chances de ocorrer alarmes falsos, para evitar o bloqueio de algum *host* internamente, esta opção de bloqueio foi previamente desativada.

Mesmo com uma rede protegida, ainda pode sim haver diversas falhas nos próprios sistemas que deveriam efetuar a defesa e muitas possíveis ameaças, mas que, podem não ser detectadas por sistemas externos. O *Zabbix* funcionando através do sistema de gerente e agente, que no caso do gerente, sua função é

apenas de solicitar informações aos agentes, gerar e enviar alertas, criar as métricas, gráficos, e a interface de monitoramento; O agente, seu papel foi o de coletar informações da máquina onde foi instalado e quando solicitado pelo gerente, enviando as informações, sendo todas estas armazenadas em um banco de dados no próprio gerente.

O gerente *Zabbix* foi instalado em uma máquina virtual *Linux Debian 8*, com o endereço de *IP* fixo de 200.200.200.20/24, e a interface de configuração foi acessada através do navegador de *internet* de uma máquina *Windows 7*. Os agentes foram instalados em todos os sistemas e máquinas da rede interna, incluindo o próprio sistema operacional *Debian*, onde o *Zabbix* também deveria estar incluso nas instalações, bem como a instalação na interface do potente *firewall Pfsense*.

Foram configurados no *Zabbix* o monitoramento padrão para cada *host* utilizando os itens oferecidos pelos *templates* que o próprio *Zabbix* oferece, dando principal atenção ao monitoramento do sistema *BSD*, onde os outros sistemas estavam instalados, com foco na utilização dos recursos de *hardware*, como *CPU* e memória, já que se caso houvesse alguma falha nestes sistemas, a rede interna poderia ser comprometida.

A principal característica da utilização do *Zabbix* no ambiente foi a integração que ele possui com os outros sistemas, podendo gerar alertas baseados em *logs* e comportamentos do sistema monitorado. Desta maneira foram configurados para monitoramento do *Pfsense/Snort*, a leitura dos *logs* gerados, permitindo até mesmo o envio de alertas ao administrador de segurança via *e-mail*.

Nos outros *hosts* foram criados um item que monitora os *logs* de sistema, visando verificar os *logs* por possíveis indicações de tentativas de ataque, como repetidas tentativas de *login* falhas, ou desativação de algum serviço importante, como o *firewall* local por exemplo.

Se caso alguma brecha fosse encontrada no sistema criado, um atacante poderia tentar algo na rede interna, ou utilizar algum computador que já comprometido, sendo controlado remotamente, poderia efetuar ataques em outros computadores. Sistemas como um *IDS*, que utilizam pacotes de regras para fazer sua atuação nas ameaças intrusivas, são vulneráveis às ameaças que não são documentadas ainda, como é o caso dos *zero-day-exploits*. E para estes fins foi escolhido a implementação de um *honeypot* na rede interna, utilizando o *HoneyD*.

O *HoneyD* foi instalado na mesma máquina *Debian 8* onde está instalado o gerente *Zabbix*, podendo assim, mais facilmente, ter seus *logs* monitorados pelo *Zabbix*, sendo capaz de gerar os alertas no caso de alguma tentativa de ataque.

Foram criados pelo *HoneyD*, diversas máquinas virtuais *honeypot* variadas, sendo elas 5 *Windows XP*, 5 *Linux 3.1*, ambas com seus endereços *IP* sendo variados dentro do escopo da rede 200.200.200.0/24. As máquinas *honeypot Windows* incluem nas suas configurações as ações de bloqueio de conexões do tipo *TCP* e *UDP*, e a liberação do acesso *ICMP*. Foram configurados também o tempo ativo da máquina de 1728650 segundos, a abertura da porta 80, 22, 23, 135, 139 e 445 via *TCP* e da porta 53 via *UDP*, portas que geralmente podem ser encontradas abertas em um sistema *Windows*. No caso dos *honeypots Linux*, as configurações de protocolos *TCP*, *UDP* e *ICMP* são as mesmas, porém as portas abertas são as portas 22, 8080 e 80.

Para poder visualizar os acontecimentos em tempo real, foi definido um arquivo onde o *HoneyD* iria criar os *logs* dos *honeypots*, na própria pasta do *HoneyD*. Este arquivo de *log* foi utilizado pelo *Zabbix* como item de monitoramento, gerando alertas em caso de qualquer tipo de conexão ou solicitação direta ao servidor *honeypot*.

4.2 PENTESTING

Para verificarmos e mensurarmos a capacidade defensiva e principalmente da segurança dos sistemas implementados, foi decidido que seriam realizados diversos testes de intrusão, utilizando diversas variáveis e através do sistema operacional *Kali Linux*, que dispõe de uma gama imensa de ferramentas para o *pentest*. Os testes de intrusão foram executados em um ambiente simulado e na seguinte ordem: Primeiramente sem os sistemas de defesa, executados com o intuito de mostrar a capacidade funcional e eficaz dos ataques, demonstrando como podem ser prejudiciais à saúde de um sistema e à uma rede desprovida de segurança; E novamente os mesmos ataques, mas desta vez com os sistemas de segurança ativos para verificarmos a eficiência e a eficácia de cada um deles. Durante a repetição dos testes, alguns tiveram que ser refeitos com um ou dois dos sistemas inativos, para assim podermos atestarmos a atuação de algum dos sistemas sem a interferência de outro.

Para os diferentes tipos de ataques, foram utilizadas várias ferramentas de invasão que advém do sistema operacional *Kali Linux*, mas juntamente com os testes de penetração, foram também utilizadas as ferramentas de defesas dos sistemas já implementados e que, a sequência dessas atividades está discurrida nos próximos subtítulos e parágrafos deste trabalho.

4.3 PORTMAPPING COM NMAP

Não sendo um ataque direto, mas uma maneira de utilizar uma ferramenta de mapeamento para fazer o reconhecimento e juntar informações sobre portas, serviços e vulnerabilidades do sistema, muito utilizado previamente a outras ferramentas de ataque.

- Utilização do ataque: Utilizado a ferramenta *Nmap* para fazer *Portmapping* no *firewall* de borda, para verificar possíveis vulnerabilidades contra o próprio servidor e obter assim o acesso interno;
- Resultados do ataque: Verificado portas e serviços do *firewall* com sucesso. Tais ataques poderiam ser utilizados contra estas portas para comprometer o sistema do servidor e expondo a rede interna a vários outros ataques;
- Defesa no projeto implementado: Utilizando as regras de detecção do *Snort* na interface *WAN*, o *Snort* com sucesso pôde detectar a tentativa de *Portscan* e bloquear o *IP* do atacante antes que alguma informação fosse conseguida. O *Portscan* também não foi capaz de diferenciar as máquinas reais dos *honeypots*, gerando também um alerta a cada tentativa.

4.4 ATAQUE DoS EXTERNO E INTERNO

Ataques de negação de serviço (*DoS - Denial of Service*) visam causar o atraso ou a indisponibilidade de algum serviço, muitas vezes optando por sobrecarregar o sistema que é o alvo do ataque.

A ferramenta *Synflood* utiliza-se do próprio funcionamento do protocolo *TCP* para derrubar o sistema na camada de transporte. No protocolo *TCP*, quando uma solicitação é feita, uma série de mensagens são trocadas entre o cliente e o servidor, conhecido como aperto de mão em três etapas (*three-way handshake*), onde o cliente envia uma solicitação *SYN* (sincronia), o servidor envia uma mensagem de *SYN-ACK* (sincronia reconhecia), e então o cliente envia uma última mensagem *ACK* (reconhecimento), para então estabelecer a conexão.

O *Synflood* envia solicitações *SYN* que são propositalmente falhas e incompletas, podendo não responder com o ultimo *ACK*. O servidor aguarda pela resposta por um tempo e tenta reestabelecer a conexão pela falta do último *ACK*, causando congestionamento no servidor. A ferramenta permite o envio de diversas solicitações como esta, podendo comprometer completamente o serviço que foi o alvo do ataque.

- Utilização do ataque: Utilizado o *Synflood* para efetuar um ataque *DoS* pela rede externa e rede interna contra o servidor;
- Resultado do ataque: Os serviços simplesmente pararam de funcionar em questão de poucos minutos de execução, exceto externamente;
- Defesa no projeto implementado: De maneira externa, todas tentativas de *DoS* foram bloqueadas pelo *firewall Pfsense*, sem nenhuma capacidade de conexão ou de execução do ataque sem utilização de outros recursos. Caso houvesse a tentativa do ataque em uma rede interna (usuário mal-intencionado e ou máquina da rede interna comprometida), o *Snort* seria capaz de detectar facilmente o ataque e em instantes, logo após a execução do ataque realizado internamente.

Foi observado que, mesmo e caso o sistema fosse comprometido, o monitoramento do *Zabbix* verifica a disponibilidade do sistema e a carga colocada na *CPU* por este ataque, podendo por si só alertar o administrador do sistema sobre o ataque intentado.

4.5 INVASÃO POR SSH VIA BRUTE-FORCE

Um ataque *Brute-force* consiste em uma tentativa de descoberta de senha por tentativas e erros, onde se utiliza uma lista de usuário e senhas que são testadas uma a uma pela ferramenta de ataque, até que possa ser conseguido o acesso ao sistema. Dependendo da quantidade de senhas e usuários possíveis, notou-se um tempo excessivo na execução do ataque, que de acordo com o dicionário utilizado, este ataque varia de alguns minutos e ou até mesmo vários dias na tentativa exaustiva de se conseguir a quebra de senhas.

Existem diversos sistemas que podem ser usados para gerar o arquivo com as possíveis senhas de usuário, e para execução do *Brute-force* que será discorrido a seguir, foram utilizados os pacotes *crunch* e *medusa* respectivamente para estes fins.

- Utilização do ataque: Utilizado um arquivo pequeno que continha a senha do usuário administrador, e depois utilizado um arquivo gerado pelo *crunch*. O pacote *medusa* então foi utilizado para fazer o *Brute-force* com cada arquivo;
- Resultado do ataque: O serviço/ferramenta *Medusa* foi capaz de conseguir acesso *SSH* no sistema alvo. Com o segundo arquivo a operação iria demorar no mínimo 2 dias, mas também teria sucesso já que a senha do usuário havia sido gerada e incluída na lista de senhas.
- Defesa no projeto implementado: Utilizando a ferramenta *Zabbix*, através de seu monitoramento, foi possível analisar os *logs* do sistema, e gerar os alertas baseados neles. Para que tal configuração fosse possível, foi necessário configurar corretamente o agente *Zabbix* para que o monitoramento 'ativo' fosse executado. Então, na interface *web* e após a configuração do item de monitoramento '*log*', especificou-se uma *trigger* para caso uma certa frase que indica falha no *login* fosse identificada em um curto período de tempo e que, o *Zabbix* pôde então gerar os alertas e tomar as ações para alertar o administrador de segurança do sistema.

4.6 INVASÃO VIA CONEXÃO TCP REVERSA

Um dos tipos de ataques mais perigosos e eficientes realizados na simulação, pois através das ferramentas disponibilizadas pelo *Metasploit* no sistema operacional *Kali Linux*, foi estabelecido uma conexão de nível privilegiado ao sistema atacado, com inúmeras possibilidades para o comprometimento do sistema da vítima, podendo recuperar o acesso remoto até mesmo após o sistema ser desligado e religado, bastando ao atacante, apenas ficar com seu *software* de escuta, aguardando a vítima se conectar à *internet*.

Quando obtido a conexão, o atacante possui quase que o controle total do sistema, como o acesso irrestrito aos arquivos, podendo realizar a modificação do sistema, capturando teclas digitadas com ferramentas como o *keylogging*, visualizar a tela do usuário em tempo real, roubar por completo todas as senhas do sistema, que aliás, mesmo com criptografia, podem ser facilmente descobertas com simples aplicações da *internet*.

Como o roteador e *firewall* por padrão não permitem a conexão provida da *internet* para a rede interna, foi então utilizado o que é denominado de

payload, que quando executado na máquina alvo, ele mesma cria uma conexão *TCP* com a máquina do atacante, fazendo a conexão ser da rede interna para a externa, burlando facilmente o sistema de proteção da rede.

- Utilização do ataque: Para *Linux*, o acesso inicial ao sistema foi executado por uma brecha no compilador de código C distribuído (*distcc*) para que ele executasse comandos no *shell* do sistema. Então, na máquina alvo foi baixado um *exploit* sem nome, que será executado na máquina alvo para estabelecer a conexão privilegiada junto ao atacante via *Netcat*. Para *Windows* foi criado o *payload* através da ferramenta *Msfvenom* no próprio *Metasploit*, que será executado na máquina alvo, tentando assim a conexão reversa e privilegiada ao sistema;
- Resultado do ataque: Em ambos os casos a conexão foi realizada de maneira privilegiada e eficiente, pois foi facilmente estabelecida a conexão reversa e como exemplo, várias senhas e arquivos, foram todos roubados de ambos sistemas, onde testamos também e com sucesso, várias outras aplicações de acesso e ações realizadas no sistema da vítima;
- Defesa no projeto implementado: O *Snort* foi o principal atuador para a detecção e bloqueio do ataque, já que este tipo em específico de ataque é realizado de uma maneira arquitetada para burlar o *firewall*. Houveram inúmeras detecções pelo *Snort* de acordo com os diferentes estágios do ataque, onde primeiramente ele bloqueou a tentativa de *download* por detectar o arquivo malicioso e que, após esta ação ele bloqueou a tentativa de criar-se a conexão reversa pelo *exploit*, mesmo quando este já dentro do sistema, e por fim, detectou uma conexão já ativa e bloqueou com sucesso a conexão, encerrando assim a possibilidade do ataque e invasão por todas as vias de acessos.

4.7 INSTALAÇÃO DOS SERVIÇOS

4.7.1 Instalação do Pfsense

O *Pfsense* foi instalado em uma única máquina virtual, sendo um sistema operacional próprio baseado na arquitetura *Free BSD*, utilizando a imagem baixada do próprio *site* do *Pfsense*. A arquitetura dele suporta apenas a arquitetura de sistemas *64 bits* e que, é também necessário haver duas placas de redes

instaladas e configuradas na máquina para o seu correto funcionamento, sendo a primeira em modo *Bridge/NAT* e a segunda em modo interno.

Após iniciado a máquina com o *CD* ou *pendrive* de instalação do *Pfsense*, aceitou-se os termos de uso, e então prosseguimos com a opção de instalar o *Pfsense* e que, conforme a figura 19, podemos alterar inclusive o idioma para português, bem como o teclado para '*brazilian accent keys*'.

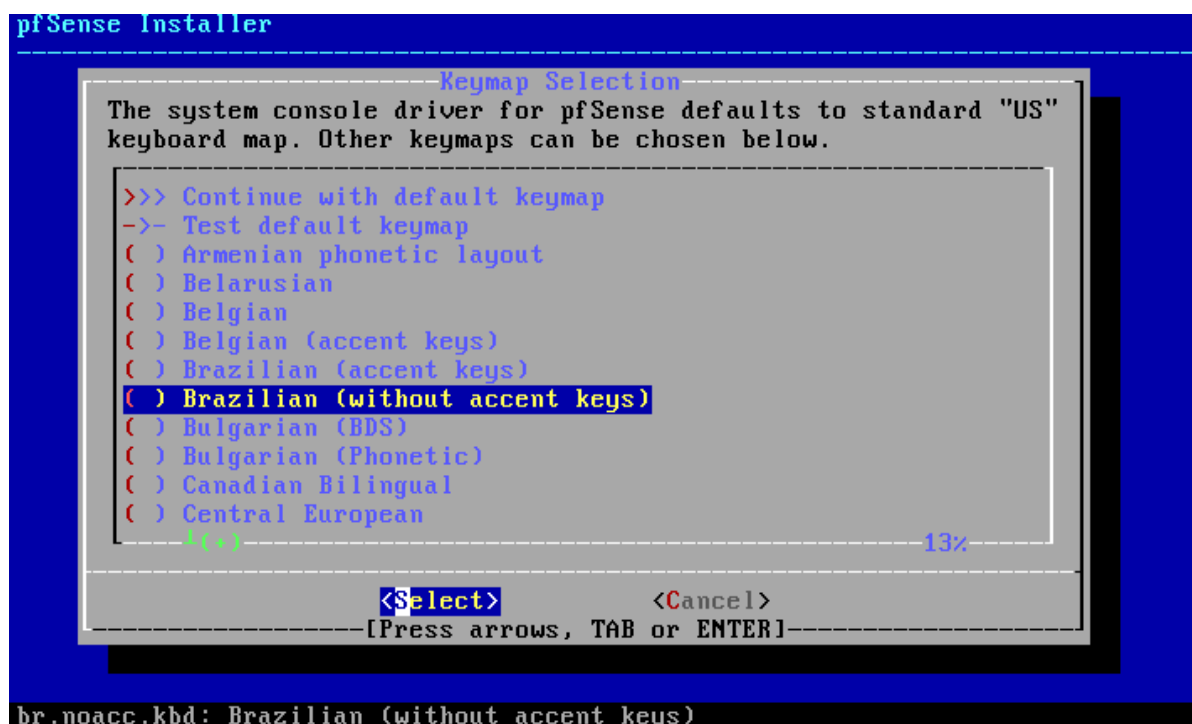


Figura 19: Instalação Pfsense

Fonte: Autoria do grupo (2019)

Prosseguindo com a instalação guiada, selecionamos a opção '*guided disk setup*'. A instalação e verificação básica então foi efetuada automaticamente, e após isso o setup do *Pfsense* perguntou se desejávamos abrir o *shell* para outras configurações antes de efetuar o *reboot*. Neste caso clicamos em não e após opção selecionada, retiramos o disco do sistema e clicamos para iniciar o *reboot*, para a instalação não se iniciar novamente.

Após reiniciado o sistema, conforme figuras 20 e 21, verificou-se que o número de *IP* da placa estava em modo de rede interna, modo este que será utilizado para acessar o setup e a interface gráfica de configuração do *Pfsense* através do próprio navegador do sistema, dando-nos inclusive opções para efetuar todas as regras necessárias de acordo com a necessidade de proteção da rede e que, o usuário e a senha padrão se arrasta a seguir logo após a figura 21.

```

Starting syslog...done.
Starting CRON... done.
pfSense 2.4.4-RELEASE amd64 Thu Sep 20 09:03:12 EDT 2018
Bootup complete

FreeBSD/amd64 (pfSense.localdomain) (ttyv0)

VirtualBox Virtual Machine - Netgate Device ID: a46d192dff0a04337e2e

*** Welcome to pfSense 2.4.4-RELEASE (amd64) on pfSense ***

WAN (wan)      -> em0      -> v4/DHCP4: 192.168.42.8/24
LAN (lan)      -> em1      -> v4: 192.168.1.1/24

0) Logout (SSH only)          9) pfTop
1) Assign Interfaces          10) Filter Logs
2) Set interface(s) IP address 11) Restart webConfigurator
3) Reset webConfigurator password 12) PHP shell + pfSense tools
4) Reset to factory defaults  13) Update from console
5) Reboot system              14) Enable Secure Shell (sshd)
6) Halt system                 15) Restore recent configuration
7) Ping host                   16) Restart PHP-FPM
8) Shell

Enter an option:

```

Figura 20: Interface Pfsense no servidor

Fonte: Autoria do grupo (2019)



[Login to pfSense](#)

The image shows the pfSense web interface login page. It has a dark blue background. At the top left is the pfSense logo. At the top right is a link 'Login to pfSense'. In the center, the text 'SIGN IN' is displayed. Below it, the username 'admin' is entered in a text field. Below the username field is a password field represented by a series of dots. To the right of the password field is an eye icon for toggling visibility. At the bottom center is a green button labeled 'SIGN IN'.

Figura 21: Interface Pfsense Web

Fonte: Autoria do grupo (2019)

O usuário e senha padrões utilizados foram:

Usuário: *admin*

Senha: *Pfsense*

Logo após o primeiro *login*, algumas configurações foram efetuadas, como os *IPs* das interfaces de rede, endereço *MAC* 'falso' para o sistema, entre outros.

Por padrão, a versão 2.4.4 do *Pfsense* já vem com *NAT* ativo, servidor *DHCP* nas interfaces *LAN*, *DNS*, e como regras padrões, liberado todo o acesso, exceto regras de bloqueio à tentativas de conexão na interface *WAN* por redes privadas pela *RFC 1918* e redes *bogon* (falsas) providas da *internet*, não assinaladas pela IANA.

Como está sendo utilizado como *firewall* de borda, existem algumas regras a mais que foram úteis, como: bloqueio de qualquer conexão de saída que utiliza portas sem criptografia, como a porta 80 (*HTTP*) e porta 110 (*POP3*), e nas regras de entrada, adicionamos uma exceção às conexões de acesso remoto.

Todas estas configurações de regras, conforme figura 22, foram executadas através das abas *Firewall>Rules*

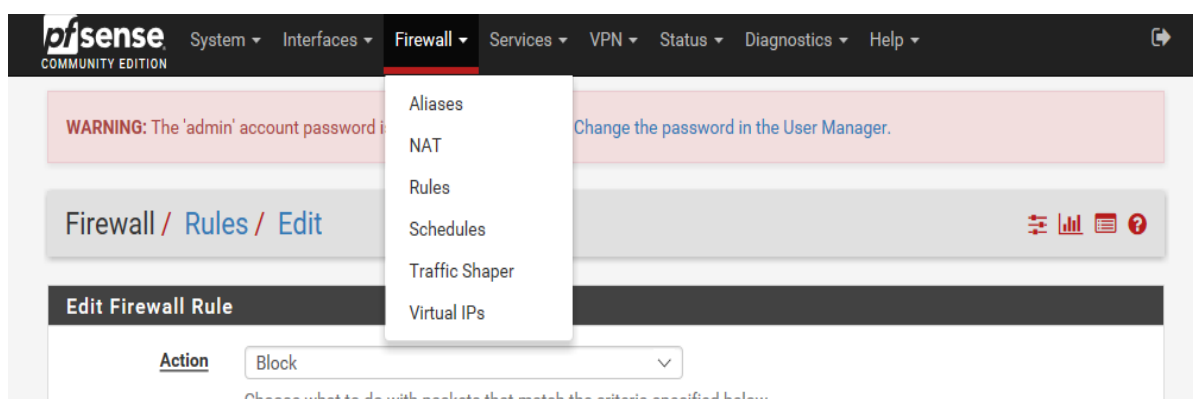


Figura 22: Interface *Pfsense* Web (*Firewall*)
Fonte: Autoria do grupo (2019)

Selecionamos a interface de rede que iríamos inserir todas as regras de entrada, estas que foram atribuídas à interface *WAN*, atrelada e responsável pela rede externa e que, observamos também que assim como qualquer *firewall*, além de possuímos as regras de entrada previamente configurada, deve-se haver também as regras de saída, as quais inserimos na interface *LAN*, que é a rede interna do nosso sistema.

Para o bloqueio específico de aplicativos, utilizamos e configuramos o recurso *OpenAppID* do *Snort*.

4.7.2 Instalação do *Snort*

O pacote *Snort* foi instalado em conjunto com o *Pfsense*, sendo ideal para o correto funcionamento da detecção de intrusão, porém este sempre atuando paralelamente com o *firewall* e que, utilizamos o recurso *OpenAppId*, que nos

permitiu realizarmos o bloqueio de aplicativos específicos, ação esta enfatizada e realizada pelo *Snort*.

O *Snort*, um pacote já inserido nas dependências do *Pfsense*, conforme figuras 23 e 24, foi instalado através da própria interface gráfica do *Pfsense* e já no navegador do sistema a se proteger, local este específico em: *System>Package Manager>Available Packages*

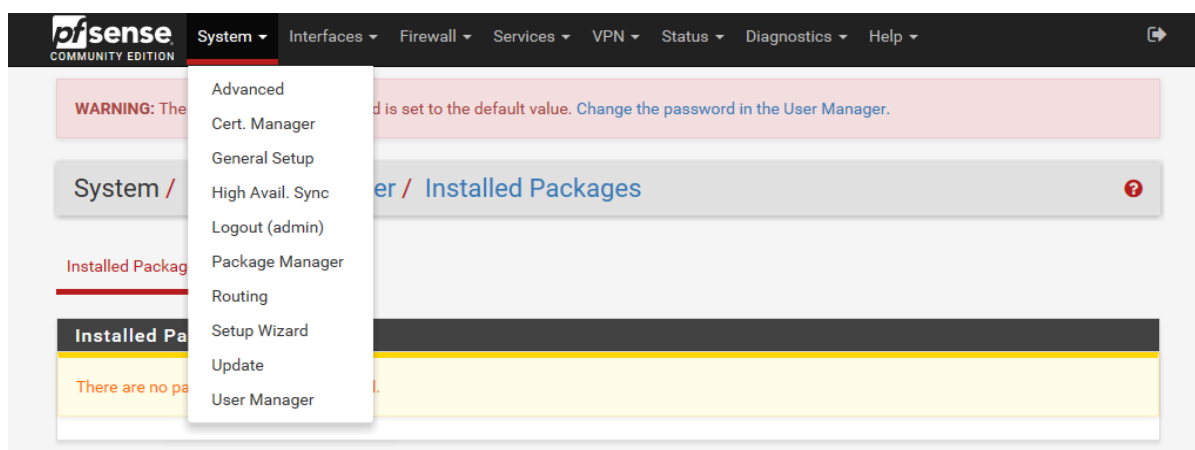


Figura 23: Interface *Pfsense Web (System)*

Fonte: Autoria do grupo (2019)

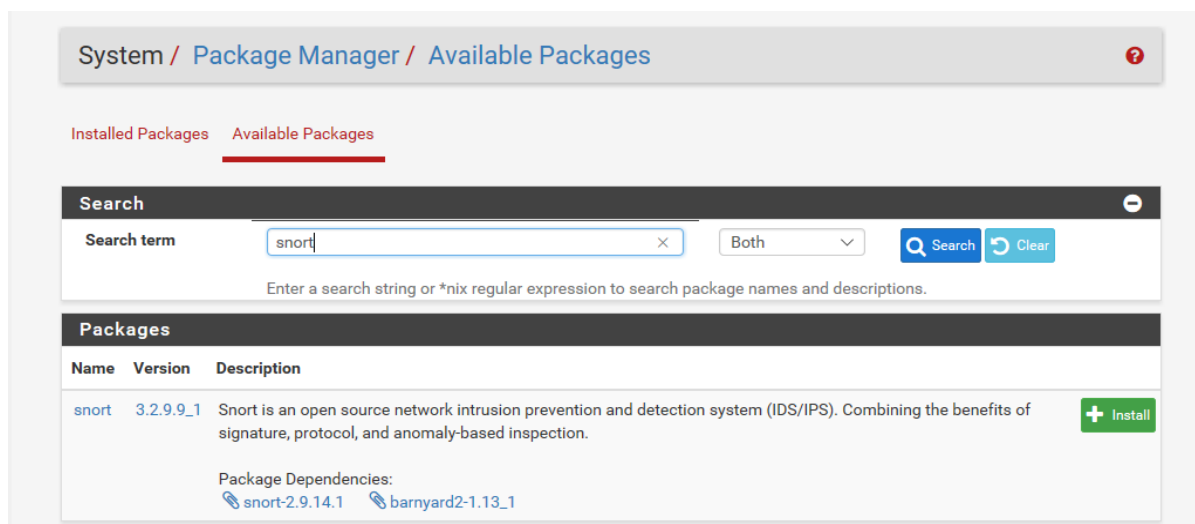


Figura 24: Interface *Pfsense Web (Snort)*

Fonte: Autoria do grupo (2019)

Após instalado, ainda na interface gráfica, em específico no navegador do sistema, conforme figura 25, todas as configurações do *Snort* foram previamente configuradas em: *Services>Snort*:



Figura 25: Interface *Pfsense Web (Services)*
Fonte: Autoria do grupo (2019)

Já dentro da opção em: *Services>Snort>Global Settings*; foi então possível realizar o *download* de todos os pacotes de regras (*rules*) do *Snort* para a utilização e posteriormente realizarmos todas as configurações, bastando-nos apenas selecionarmos a caixa de seleção, esta conforme foi claramente demonstrada na figura 26 a seguir.

Snort Subscriber Rules	
Enable Snort VRT	☒ Click to enable download of Snort free Registered User or paid Subscriber rules
Sign Up for a free Registered User Rules Account Sign Up for paid Snort Subscriber Rule Set (by Talos)	
Snort Oinkmaster Code	 Obtain a snort.org Oinkmaster code and paste it here. (Paste the code only and not the URL!)
Snort GPLv2 Community Rules	
Enable Snort GPLv2	☒ Click to enable download of Snort GPLv2 Community rules
The Snort Community Ruleset is a GPLv2 Talos certified ruleset that is distributed free of charge without any Snort Subscriber License restrictions. This ruleset is updated daily and is a subset of the subscriber ruleset.	
Emerging Threats (ET) Rules	
Enable ET Open	☒ Click to enable download of Emerging Threats Open rules
ETOpen is an open source set of Snort rules whose coverage is more limited than ETPro.	
Enable ET Pro	☐ Click to enable download of Emerging Threats Pro rules
Sign Up for an ETPro Account ETPro for Snort offers daily updates and extensive coverage of current malware threats.	
Sourcefire OpenAppID Detectors	

Figura 26: Configuração Snort
 Fonte: Autoria do grupo (2019)

Nesta mesma tela de seleção, na parte inferior, conforme figura 27, modificamos o intervalo de atualização das regras para que elas fossem atualizadas de acordo com o tempo de nossas necessidades:

OpenAppID Version	N/A (Not Downloaded)	
Enable RULES OpenAppID	☐ Click to enable download of APPID Open rules Note - the AppID Open Rules file is maintained by a volunteer contributor and hosted by the pfSense team. The URL for the file is http://files.pfsense.org/openappid/appid_rules.tar.gz .	
Rules Update Settings		
Update Interval	NEVER 6 HOURS 12 HOURS 1 DAY 4 DAYS 7 DAYS 28 DAYS	disables auto-updates. Default is 00:05. Rules will update at the interval chosen above starting at the time specified here. For example, using the default start time of 00:05 and choosing 12 Hours for the interval, the rules will update at 00:05 and 12:05 each day.
Update Start Time		
Hide Deprecated Rules Categories	☐ Click to hide deprecated rules categories in the GUI and remove them from the configuration. Default is not checked.	
Disable SSL Peer Verification	☐ Click to disable verification of SSL peers during rules updates. This is commonly needed only for self-signed certificates. Default is not checked.	
General Settings		
Remove Blocked Hosts Interval	NEVER Please select the amount of time you would like hosts to be blocked. In most cases, one hour is a good choice.	

Figura 27: Configuração Snort
 Fonte: Autoria do grupo (2019)

Foi necessário inserir também, conforme figura 28, uma interface para ser monitorada pelo *Snort*, esta em *Services>Snort>Snort Interfaces*, clicando no botão verde para adicioná-la:

Services / Snort / Edit Interface / None

Snort Interfaces Global Settings Updates Alerts Blocked Pass Lists Suppress IP Lists SID Mgmt Log Mgmt Sync

None Settings None Categories None Rules None Variables None Preprocs None Barnyard2 None IP Rep None Logs

General Settings

Enable ☒ Enable interface

Interface WAN (em0) Choose the interface where this Snort instance will inspect traffic.

Description WAN Enter a meaningful description here for your reference.

Snap Length 1518

Figura 28: Configuração Snort (Portas)

Fonte: Autoria do grupo (2019)

Já estando na tela inicial de configuração da interface, averiguou-se algumas informações importantes a serem adicionadas, como por exemplo, a escolha da interface utilizada e monitorada pelo *Snort*, informando ao *Pfsense* qual seriam os *IPs* bloqueados das detecções, além de outras opções.

Observamos que a opção de bloqueio de *IP* foi muito mais indicada para a interface *WAN* do que para a *LAN*, onde podemos bloquear várias tentativas suspeitas de conexão através da própria rede externa ao invés de bloquearmos os próprios *hosts* de nossa rede, porém como foi bem configurada esta opção, tivemos também a opção de ser utilizada na interface *LAN*, o que não foi o nosso caso.

Para as configurações da interface foi necessário escolher quais pacotes de regras iriam atuar na interface e que, conforme demonstrada na figura 29, esta estava localizada na aba *WAN Categories* da interface, onde foi selecionado todos os pacotes de regras que iríamos utilizar em nosso projeto.

Services / Snort / Categories / WAN

Snort Interfaces Global Settings Updates Alerts Blocked Pass Lists Suppress IP Lists SID Mgmt Log Mgmt Sync

WAN Settings WAN Categories WAN Rules WAN Variables WAN Preprocs WAN Barnyard2 WAN IP Rep WAN Logs

Snort Rulesets (Categories) Selection

Figura 29: Configuração Snort

Fonte: Autoria do grupo (2019)

Para ativarmos o *Snort* na interface, conforme figura 30, clicamos no botão azul na aba de *Snort Interfaces*.

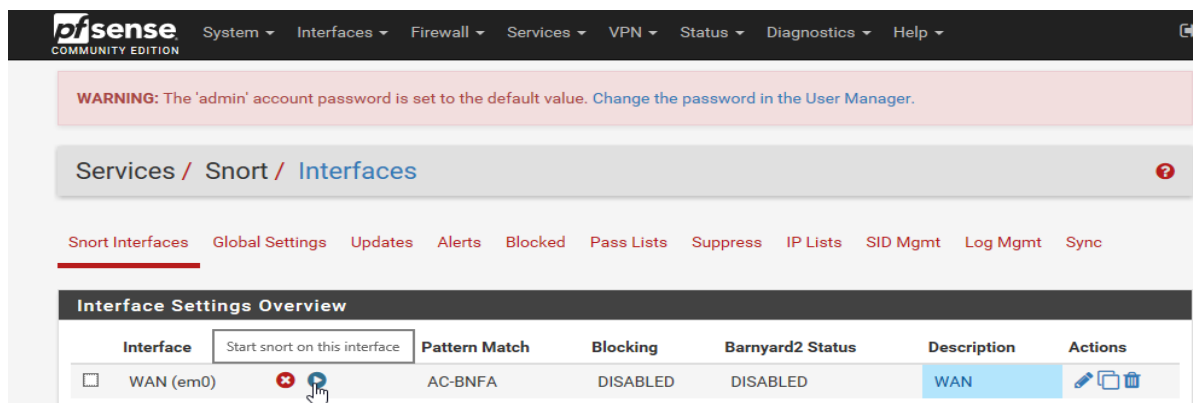


Figura 30: Ativação do Snort

Fonte: Autoria do grupo (2019)

Após alguns instantes, o ícone verde nos indicou que o *Snort* foi ativado com sucesso na interface, porém muitos falsos-positivos foram detectados através das regras selecionadas por nós, o que ocasionou em vários alarmes falsos e automaticamente fomos induzidos a desativarmos algumas regras individuais, estas localizadas na aba do *Snort>Rules*.

Para verificarmos os aplicativos e podermos bloqueá-los, certificamos também que ativamos as regras do *OpenAppId* (na aba de instalação dos pacotes de regras para o *Snort*), que conforme figura 31, as ativamos nas categorias da interface mesmo, sendo este local na aba de *Preprocs* da interface, conforme demonstramos a seguir.

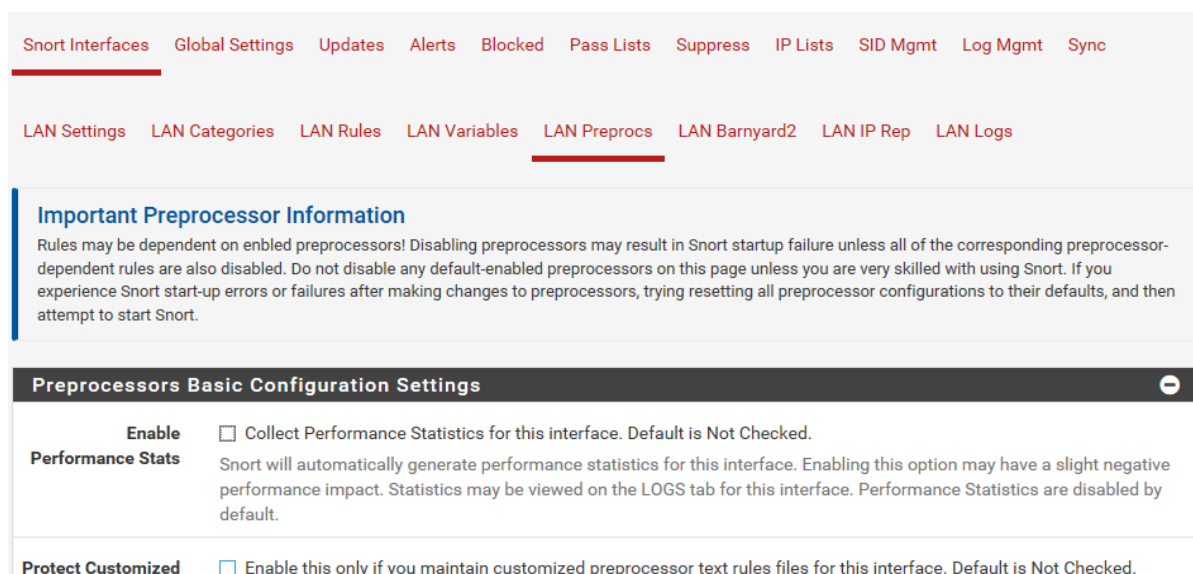
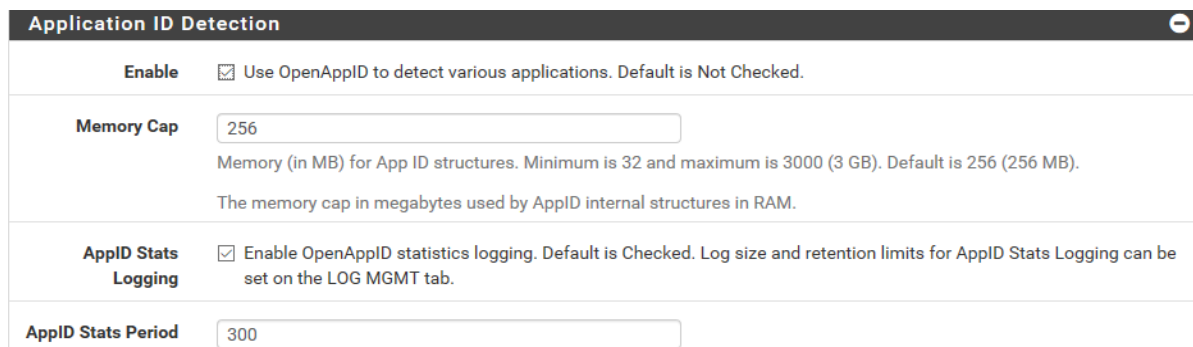


Figura 31: Configuração Snort LAN

Fonte: Autoria do grupo (2019)

Essas regras de detecção foram ativadas especificamente em: *Application ID Detection* conforme figura 32 e que, foi ativado tanto regras de detecção, quanto a opção *AppID Stats Logging*, conforme demonstramos a seguir.



Application ID Detection

Enable ☒ Use OpenAppID to detect various applications. Default is Not Checked.

Memory Cap
 Memory (in MB) for App ID structures. Minimum is 32 and maximum is 3000 (3 GB). Default is 256 (256 MB).
 The memory cap in megabytes used by AppID internal structures in RAM.

AppID Stats Logging ☒ Enable OpenAppID statistics logging. Default is Checked. Log size and retention limits for AppID Stats Logging can be set on the LOG MGMT tab.

AppID Stats Period

Figura 32: Regras de Detecção
 Fonte: Autoria do grupo (2019)

Conforme demonstra na figura 33, as aplicações detectadas puderam ser visualizadas na aba de alertas do *Snort* durante este processo de detecção.

2017-11-16 20:15:18	3	TCP	Misc activity	192.168.20.20 Q ⊕	62641	31.13.71.1 Q ⊕	443	1:70439 ⊕ ✖	facebook
2017-11-16 20:09:28	3	TCP	Misc activity	192.168.20.9 Q ⊕	59412	45.57.45.159 Q ⊕	80	1:70542 ⊕ ✖	netflix
2017-11-16 20:11:14	3	TCP	Misc activity	192.168.20.9 Q ⊕	34759	52.94.212.133 Q ⊕	443	1:70012 ⊕ ✖	Amazon Webservices
2017-11-16 20:21:03	3	TCP	Misc activity	192.168.20.20 Q ⊕	62654	104.244.46.71 Q ⊕	443	1:70656 ⊕ ✖	twitter

Figura 33: Aplicações Detectadas
 Fonte: Autoria do grupo (2019)

4.7.3 Instalação do *Zabbix*

A instalação do *Zabbix* gerente foi realizada em uma máquina virtual *Debian 8 - 64 bits*.

A instalação foi iniciada a partir do *download* do pacote compactado do *Zabbix 3.4* e a partir do repositório oficial do próprio *Zabbix*, conforme demonstrado com os comandos a seguir:

```
# wget http://repo.zabbix.com/zabbix/3.4/debian/pool/main/z/zabbix-  
release/zabbix-release_3.4-1+jessie_all.deb
```

Após a realização do respectivo *download*, fizemos a instalação do pacote de acordo com comando a seguir:

```
# Dpkg -i zabbix-release_3.4-1+jessie_all.deb
```

Para instalarmos os pacotes necessários para o *Zabbix Server* funcionar adequadamente, utilizamos o comando a seguir:

```
#Apt install -f -y zabbix-server-mysql zabbix-frontend-php zabbix-agent
```

Conforme a figura 34, os pacotes como *apache2* e *Mysql* não estavam instalados, então o *Linux* automaticamente instalou tais dependências.

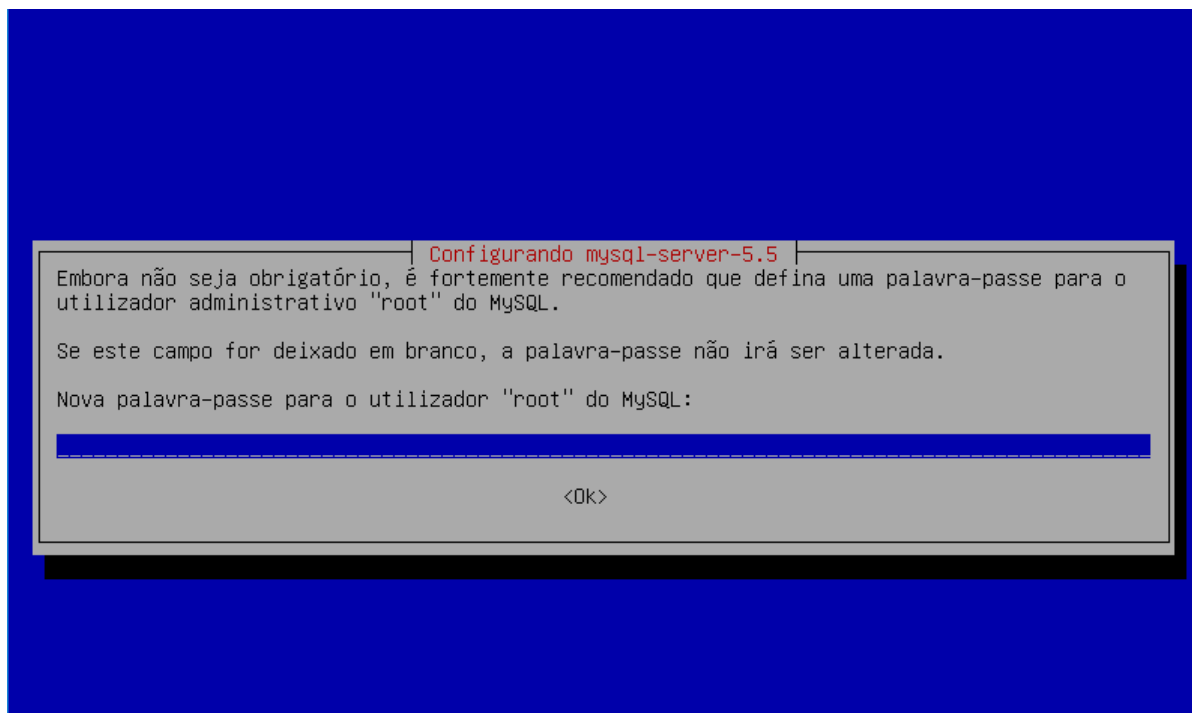


Figura 34: Configuração *Mysql-server*

Fonte: Autoria do grupo (2019)

Foi criado um usuário e um grupo para controlar o acesso às pastas e aos arquivos do *Zabbix* de acordo com os comandos a seguir:

```
# useradd -m -s /bin/bash zabbix // adiciona user zabbix e usa shell bash.
# groupadd grpzabbix // cria grupo grpzabbix
# gpasswd -a zabbix grpzabbix // adiciona usuário zabbix ao grupo
grpzabbix
# gpasswd -a www-data grpzabbix // adiciona usuário www-data ao grupo
grpzabbix
```

Ao criarmos um novo banco de dados *Mysql* que foi utilizado para o *login*, a mesma senha informada na instalação foi utilizada conforme o comando a seguir:

```
# mysql -u root -p -h localhost
Password: <informe sua senha>
```

Criado então a *database zabbixdb* com o seguinte comando:

```
mysql> create database zabbixdb;
```

Foi criado também o usuário "*zabbixuser*" no sistema gerenciador de banco de dados *Mysql* com o seguinte comando:

```
mysql> GRANT ALL PRIVILEGES ON zabbixdb. * TO 'zabbixuser'@'localhost'
IDENTIFIED BY 'password' WITH GRANT OPTION;
Query OK, 0 rows affected (0.03 sec)
```

Foi trocado a palavra *password* pela senha do usuário *zabbixuser* que foi utilizada pelo banco de dados.

Já logado no *Mysql* e novamente utilizando o usuário criado, *zabbixuser*, utilizamos a senha informada no comando *GRANT ALL PRIVILEGES*, e executamos os comandos para verificar se a *database zabbixdb* e o usuário estavam devidamente criados, utilizando os respectivos comandos a seguir:

```
mysql> use zabbixdb
mysql> show tables;
```

Por não haver tabelas na *database*, então adicionamos as respectivas tabelas à nova *database*, de acordo com o comando a seguir:

```
#zcat /usr/share/doc/zabbix-server-mysql/create.sql.gz | mysql -u
zabbixuser -p zabbixdb
```

Devido alguns arquivos não existirem, utilizamos os arquivos *schema.sql.gz*, *images.sql.gz* e *data.sql.gz* que estão no diretório */usr/share/zabbix-server-mysql/*

Já estando no diretório */etc/zabbix*, conseguimos encontrar todos estes arquivos mencionados, sendo um deles, o arquivo *zabbix_agentd.conf*, que se refere ao *daemon* do agente *Zabbix*; e o *zabbix_server.conf* que se refere ao gerente *Zabbix*. Abaixo segue os exemplos do que encontramos:

Exemplo *zabbix_agentd.conf*:

```
PidFile=/tmp/zabbix_agentd.pid
LogFile=/var/log/zabbix/zabbix_agentd.log
LogFileSize=1
DebugLevel=3
EnableRemoteCommands=1
LogRemoteCommands=1
Server=127.0.0.1      // IP do gerente autorizado a solicitar informações.
ListenPort=10050
Hostname=Zabbix Server  // Nome do servidor - ajuste conforme
necessidade.
```

Exemplo *zabbix_server.conf*:

```
ListenPort=10051
LogFile=/var/log/zabbix/zabbix_server.log
LogFileSize=2
PidFile=/tmp/zabbix_server.pid
DBHost=localhost
DBName=zabbixdb
DBUser=zabbixuser
DBPassword=password
StartIPMIPollers=1
StartDiscoverers=5
Timeout=3
```

Como a interface *web* é compilada em *PHP*, foi necessário então descomentarmos e modificarmos a linha do arquivo */etc/zabbix/apache.conf* relacionada à zona temporal conforme demonstra o comando a seguir:

```
php_value date.timezone Europe/Riga
```

Os serviços não estavam iniciando com o sistema, então realizamos os seguintes comandos:

```
#systemctl enable zabbix-server.service zabbix-agent.service
```

Utilizando um navegador, usamos o endereço de *IP* do servidor *Zabbix* para adentrar na interface de configuração, utilizando o comando:

```
<ip do servidor zabbix>/zabbix
```

Verificado o correto funcionamento, conforme a figura 35, então informamos à *database Zabbix* do *Mysql* que foi criado junto com o usuário e senha do *Mysql*.

The screenshot shows the Zabbix web interface during the database configuration step. On the left is a sidebar with the ZABBIX logo and a list of steps: Welcome, Check of pre-requisites, Configure DB connection (highlighted), Zabbix server details, Pre-installation summary, and Install. The main content area is titled 'Configure DB connection' and contains the following fields:

- Database type: MySQL (dropdown menu)
- Database host: localhost
- Database port: 0 (with a note '0 - use default port')
- Database name: zabbixdb
- User: zabbixuser
- Password: masked with dots and a toggle icon

Below the form, there is a note: 'Please create database manually, and set the configuration parameters for connection to this database. Press "Next step" button when done.'

Figura 35: Database Zabbix do Mysql

Fonte: Autoria do grupo (2019)

Tudo configurado, atestamos o procedimento pela tela de *login* a seguir, conforme demonstra a figura 36, indicando que a configuração realizada estava sim devidamente correta, até mesmo pela averiguação que foi realizada, que foi através de um procedimento simples, este feito na própria interface gráfica, onde inserimos o respectivo caminho na mesma, para constataremos a adequação da configuração realizada, dada pelo comando: *<ip server zabbix>/zabbix*

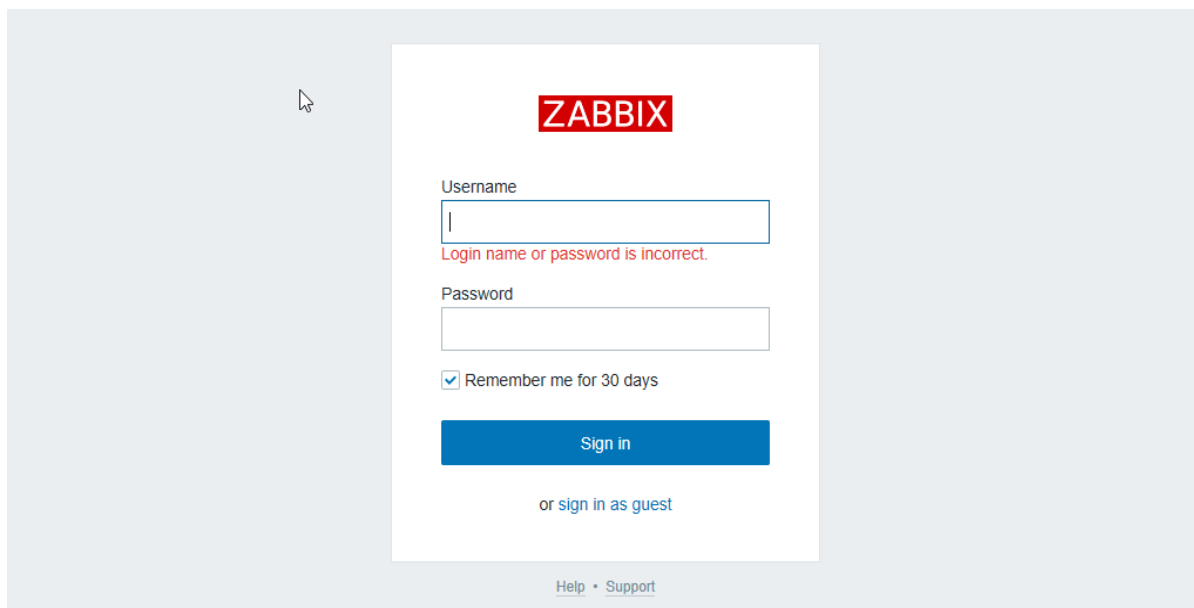


Figura 36: Interface Web Zabbix

Fonte: Autoria do grupo (2019)

Os padrões de usuário e senha utilizados foram:

Usuário: admin

Senha: Zabbix

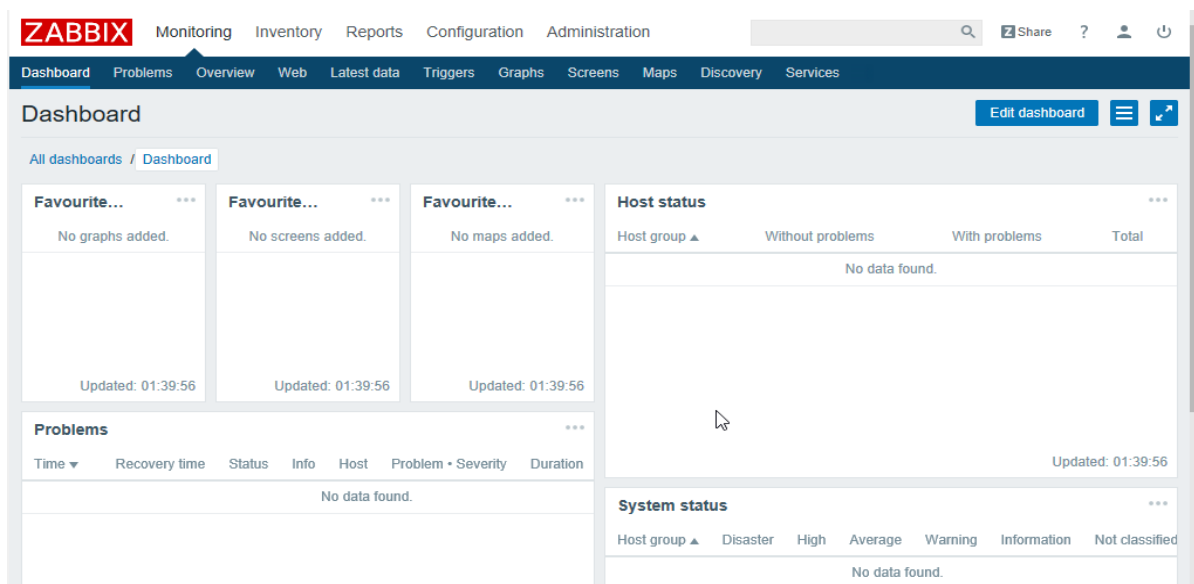


Figura 37: Interface Web Zabbix

Fonte: Autoria do grupo (2019)

Em seguida foi realizado então a instalação dos agentes *Zabbix* nas máquinas a serem monitoradas através do comando *apt*, instalando o pacote *agent-zabbix*, que quando solicitado, envia as informações da máquina para o gerente.

Após as referidas instalações, fomos até ao arquivo de configuração de cada agente, este arquivo localizado no diretório `/etc/zabbix/zabbix-agentd.conf`, e alteramos a linha que indica o endereço *IP* do servidor conforme demonstrado na figura 38, alterando também para o *zabbix-server*.

```
# Mandatory: no
# Default:
# LogRemoteCommands=0

##### Passive checks related

### Option: Server
# List of comma delimited IP addresses (or hostnames) of Zabbix servers.
# Incoming connections will be accepted only from the hosts listed here.
# If IPv6 support is enabled then '127.0.0.1', '::127.0.0.1', '::ffff:127.0.0.1' are treated as
#
# Mandatory: no
# Default:
# Server=

Server=192.168.42.7_

### Option: ListenPort
# Agent will listen on this port for connections from the server.
#
# Mandatory: no
# Range: 1024-32767
# Default:
ListenPort=10050

### Option: ListenIP
```

Figura 38: Endereço *IP* do servidor

Fonte: Autoria do grupo (2019)

No *Pfsense*, conforme a figura 39, assim como o *Snort*, a instalação do *Zabbix* também é simples e foi realizada através do mesmo procedimento, ou seja, sendo ela realizada dentro da própria interface de configuração do *Pfsense*, localizada na aba *Services>Zabbix Agent*, onde indicamos o *IP* e o nome do servidor *Zabbix*.

Package / Services: Zabbix Agent LTS / Agent

Agent

Zabbix Agent LTS Settings

Enable	☒ Enable Zabbix Agent LTS service.
Server	10.0.0.2 List of comma delimited IP addresses (or hostnames) of ZABBIX servers.
Server Active	 List of comma delimited IP:port (or hostname:port) pairs of Zabbix servers for active checks.

Figura 39: Servidor *Zabbix*

Fonte: Autoria do grupo (2019)

As telas a seguir, conforme demonstradas nas figuras 40 e 41, são os locais que nos permitiu o monitoramento de todos os *hosts*, onde os adicionamos manualmente no *Zabbix server* para que, posteriormente conseguíssemos visualizar este monitoramento através da interface gráfica disponibilizada no próprio navegador do sistema.

Figura 40: Host Zabbix Server
Fonte: Autoria do grupo (2019)

Figura 41: Configuração>hosts
Fonte: Autoria do grupo (2019)

Conforme a figura 42, foi possível editarmos os *hosts* e adicionarmos novos na aba *Templates*, esta localizada na parte superior esquerda, onde adicionamos o *template* relacionado com qual OS estava sendo monitorado.

Figura 42: OS monitorado
Fonte: Autoria do grupo (2019)

Após criados, foi necessário indicarmos quais itens seriam monitorados e que, com o intuito de auxiliar na adição de novos itens monitorados pelo *Zabbix*, estes localizados na aba de inventário do *host*, foi necessário então colocarmos em modo automático para que o próprio *Zabbix* procurasse e adicionasse itens básicos ao monitoramento.

Na própria seção de configuração onde se encontram os *hosts*, conforme demonstra na figura 43, foi possível então adicionar e modificar os itens monitorados, clicando em itens e no *host* que desejássemos modificar.

Figura 43: Modificação de *hosts*
Fonte: Autoria do grupo (2019)

Configurado em modo automático, conforme as figuras 44 e 45, o *Zabbix* incluiu uma grande quantidade de itens na lista, bastando-nos apenas procurarmos o que necessitaríamos monitorar, ativar ou desativar caso necessário, podendo nós modificarmos apenas as especificações no item.

Itens Criar item

Todos os hosts / debian 8 - 1 Ativo ZBX SNMP JMX IPMI Aplicações 10 Itens 32 Triggers 15 Gráficos 5 Regras de descoberta 2 Cenários web

Filtrar ▲

Grupo de hosts Selecionar Tipo todos ▼ Tipo de informação todos ▼

Host debian 8 - 1 ✕ Selecionar Intervalo de atualização Histórico

Aplicação Selecionar Estatísticas

Nome

Chave

Aplicar Limpar

Subfiltro afeta somente a área com filtro

APLICAÇÕES
CPU 13 General 5 Memory 5 OS 8 Performance 13 Processes 2 Security 2 Zabbix agent 3

TIPO DE INFORMAÇÃO
Carácter 4 Numérico (fracionário) 12 Numérico (inteiro sem sinal) 16

COM TRIGGERS
Com triggers 15 Sem triggers 17

INTERVALO
1m 21 10m 2 1h 9

☐	Assistente	Nome ▲	Triggers	Chave	Intervalo	Histórico	Estatísticas	Tipo	Aplicações	Status	Informação
☐	...	Template App Zabbix Agent: Agent ping	Triggers 1	agent.ping	1m	1w	365d	Agente Zabbix	Zabbix agent	Ativo	
☐	...	Template OS Linux: Available memory	Triggers 1	vm.memory.size[available]	1m	1w	365d	Agente Zabbix	Memory	Ativo	
☐	...	Template OS Linux: Checksum of /etc/passwd	Triggers 1	vfs.file.cksum[/etc/passwd]	1h	1w	365d	Agente Zabbix	Security	Ativo	
☐	...	Template OS Linux: Context switches per second		system.cpu.switches	1m	1w	365d	Agente Zabbix	CPU, Performance	Ativo	

Figura 44: Ativação de Triggers
Fonte: Autoria do grupo (2019)

☐	Assistente	Nome ▲	Triggers	Chave	Intervalo	Histórico	Estatísticas	Tipo	Aplicações	Status	Informação
☒	...	Template App Zabbix Agent: Agent ping	Triggers 1	agent.ping	1m	1w	365d	Agente Zabbix	Zabbix agent	Ativo	
☒	...	Template OS Linux: Available memory	Triggers 1	vm.memory.size[available]	1m	1w	365d	Agente Zabbix	Memory	Ativo	
☐	...	Template OS Linux: Checksum of /etc/passwd	Triggers 1	vfs.file.cksum[/etc/passwd]	1h	1w	365d	Agente Zabbix	Security	Ativo	
☐	...	Template OS Linux: Context switches per second		system.cpu.switches	1m	1w	365d	Agente Zabbix	CPU, Performance	Ativo	

Figura 45: Ativação de Triggers
Fonte: Autoria do grupo (2019)

Para fins de demonstração apenas, conforme demonstram as figuras 46 e 47, realizamos previamente a configuração, adicionando um novo item, o qual realizou o monitoramento efetivo de toda a carga da *CPU*.

Item **Pré-processamento**

Nome

Tipo

Chave

Interface do host

Tipo de informação

Unidades

Intervalo de atualização

Intervalo customizado

Tipo	Intervalo	Período	Ação
☒ Flexível	50s	1-7,00:00-24:00	Remover
☐ Agendamento			

[Adicionar](#)

Período de retenção do histórico

Período de retenção das estatísticas

Mostrar valor [mostrar mapeamento de valores](#)

Nova aplicação

Aplicações

- Nenhum-
- CPU
- Filesystems
- General
- Memory
- Network interfaces
- OS
- Performance
- Processes
- Security

Preencha o campo do inventário do host

Descrição

Figura 46: Configuração
 Fonte: Autoria do grupo (2019)

Grupos de hosts Templates Hosts Manutenção Ações Correlacionamento de eventos Descoberta Serviços				
Gráficos		Grupo todos	Host debian 8 - 1	Criar gráfico
Todos os hosts / debian 8 - 1 Ativo ZBX SNMP JMX IPMI Aplicações 10 Itens 33 Triggers 15 Gráficos 5 Regras de descoberta 2 Cenários web				
☐ Nome ▲		Largura	Altura	Tipo do gráfico
☐ Template OS Linux: CPU jumps		900	200	Normal
☒ Template OS Linux: CPU load		900	200	Normal
☐ Template OS Linux: CPU utilization		900	200	Pilha
☐ Template OS Linux: Memory usage		900	200	Normal

Figura 47: CPU Gráficos
 Fonte: Autoria do grupo (2019)

Conforme demonstrado na figura 48, aqui relacionamos os gráficos já adicionados com os novos que pudéssemos adicionar ao item customizado que fora criado.

Gráfico Visualizar

Nome

Largura

Altura

Tipo do gráfico

Mostrar legenda ☒

Mostrar horário comercial ☒

Mostrar triggers ☒

Linha percentual (esquerda) ☐

Linha percentual (direita) ☐

Valor MÍNIMO no eixo Y

Valor MÁXIMO no eixo Y

Nome	Função	Estilo	Lado do eixo Y	Cor	Ação
1: debian 8 - 1: Carga do Sistema	méd	Linha	Esquerda	1A7C11	Remover

[Adicionar](#)

Figura 48: Gráficos adicionados
 Fonte: Autoria do grupo (2019)

Um outro importante recurso do *Zabbix* utilizado pelo grupo, foi a capacidade de criarmos *triggers* e gatilhos, estes com finalidades para que quando viesse a ocorrer uma determinada condição no sistema e, mediante a criação de uma resultante pré-configurada ante a esta mesma condição, esta então deveria ocorrer de forma simples e automatizada.

Conforme demonstramos na figura 49, criamos propositalmente uma *trigger* indo até a aba de configuração>*hosts* da interface do *Zabbix* e posteriormente inserimos esta mesma *trigger* no *host* desejado.

ZABBIX Monitoramento Inventário Relatórios Configuração Administração

Grupos de hosts Templates **Hosts** Manutenção Ações Correlacionamento de eventos Descoberta Serviços

Hosts Grupo: todos

Filtrar ▲

Nome DNS IP Porta

	Nome ▲	Aplicações	Itens	Triggers	Gráficos	Descoberta	Web	Interface	Templates	Status	Disponibilidade	Criptografia do agente	Informação
☐	debian 8 - 1	Aplicações 10	Itens 33	Triggers 15	Gráficos 6	Descoberta 2	Web	192.168.42.6:10050	Template OS Linux (Template App Zabbix Agent)	Ativo	ZBX SNMP JMX IPMI	NENHUM	

Figura 49: Trigger
 Fonte: Autoria do grupo (2019)

Como exemplificado na figura 50, criamos uma nova *trigger*, e a configuramos como segue a situação abaixo:

Nome:

Severidade: Não classificada Informação Atenção Média **Alta** Desastre

Expressão: Adicionar

Construtor de expressão

Evento de eventos OK: Expressão Expressão de recuperação Nenhum

Eventos de INCIDENTE: Simple Múltiplo

Eventos de eventos OK: **Todos os incidentes** Todos os incidentes que o valor da etiqueta combine

Etiquetas: Remover

Adicionar

Fechamento manual: ☐

URL:

Descrição:

Ativo: ☒

Adicionar Cancelar

Figura 50: Nova *Trigger*
 Fonte: Autoria do grupo (2019)

Esta *trigger* foi vinculada ao item criado antecipadamente e a sua expressão verificou o valor da carga de sistema e comparou-o com o valor 1.5. Caso este mesmo valor fosse maior, a *trigger* então seria acionada e seria mostrado em vermelho e descrito como incidente na tela de monitoramento.

Foi possível também criarmos ações mediante o acionamento desta mesma *trigger* para quando ela for acionada, pois todas essas configurações estão demonstradas e detalhadas logo abaixo nas figuras 51, 52 e 53, onde fomos até a aba de Administração>Usuários e adicionarmos o *e-mail* de envio de alertas, destinados ao administrador do sistema para que caso algum evento ou anomalia ocorra de forma sistêmica e proposital em toda a rede, o mesmo venha ser prontamente alertado para a tomada de posteriores ações.

Usuários

Usuário Mídia Permissões

Mídia	Tipo	Enviar para	Ativo quando	Usar se severidade	Status	Ação
	Adicionar					

Atualizar Excluir Cancelar

Figura 51: Configuração de e-mail do administrador
 Fonte: Autoria do grupo (2019)

Nova condição

Trigger = debian 8 - 1: Carga de Sistema alta x

informe aqui o argumento para pesquisa

Selecionar

Adicionar

Figura 52: Trigger desejada
 Fonte: Autoria do grupo (2019)

Ação Operações Operações de recuperação Operações de reconhecimento

Duração padrão do passo da operação 1h

Assunto padrão Problem: {TRIGGER.NAME}

Mensagem padrão Problem started at {EVENT.TIME} on {EVENT.DATE}
 Problem name: {TRIGGER.NAME}
 Host: {HOST.NAME}
 Severity: {TRIGGER.SEVERITY}
 Original problem ID: {EVENT.ID}
 {TRIGGER.URL}

Pausar operações enquanto estiver em manutenção ☒

Operações	Passos	Detalhes	Iniciar em	Duração	Ação
Detalhes da operação	Passos 1 - 1 (0 - infinitamente)	Duração do passo 0 (0 - usar a ação padrão)	Tipo da operação Enviar mensagem	Enviar para grupos de usuários Grupo de usuários Ação Adicionar	Enviar para usuários Usuário Admin (Zabbix Administrator) Ação Remover Adicionar
	Enviar apenas para Email	Mensagem padrão ☒	Condições Texto Nome Ação Nova		

Adicionar Cancelar

Figura 53: Detalhes da operação
 Fonte: Autoria do grupo (2019)

4.7.4 Instalação do *HoneyD*

O *HoneyD* é a quarta solução escolhida e utilizada no projeto pelo grupo, onde ele foi previamente instalado e exaustivamente testado em uma máquina virtual contendo o sistema operacional *Linux Debian 10*.

Realizamos o *download* do arquivo denominado como *master* do *HoneyD* através do *GitHub* conforme o comando utilizado a seguir:

```
# wget https://github.com/DataSoft/Honeyd/archive/master.zip
```

Com um descompactador de arquivos previamente instalado em nossa máquina, realizamos então a descompactação deste arquivo conforme o comando abaixo:

```
# unzip master.zip
```

Várias dependências foram necessárias para que o *HoneyD* funcionasse como o almejado pelo grupo, portanto fora necessário instalá-lo juntamente com os pacotes que nos permitiriam a compilação adequada e correta do *software* e que, tais dependências foram instaladas em conformidade e através dos comandos utilizados abaixo:

```
# apt-get install libevent-dev libdumbnet-dev libpcap-dev libpcrc3-dev  
libedit-dev bison flex libtool automake zlibc zliblg zliblg-dev
```

Após adentrarmos na pasta descompactada do *HoneyD*, copiamos então os seguintes arquivos para a pasta de configuração do *HoneyD*, conforme se arrasta a seguir com os comandos utilizados:

```
# cp /usr/share/automake-1.14/config.sub .  
# cp /usr/share/automake-1.14/config.guess .
```

Quando já dentro da pasta do *HoneyD*, instalamos-lo com os seguintes comandos:

```
# ./autogen.sh  
# ./configure  
# make  
# make install
```

Observamos que os arquivos de configuração do *HoneyD* foram instalados no diretório */usr/share/honeyd/*, bem como os arquivos de configurações de máquinas *honeypot*, com todas as suas informações e de seus eventuais comportamentos na rede, constatando por fim que todos são inseridos também neste mesmo diretório.

As personalidades que cada *honeypot* deteve ao longo dos testes, puderam serem inseridas utilizando o nome exato delas que se encontram no diretório `/usr/bin/nmap`. Conforme figura 54, exemplificamos a criação de um arquivo de configuração que possui 4 *honeypots*, cada um com suas particularidades, personalidades e comportamentos distintos.

```
create WindowsXP
set WindowsXP personality "Microsoft Windows XP Professional SP1"
set WindowsXP default tcp action reset
add WindowsXP tcp port 25 "scripts/smtp.pl"
add WindowsXP tcp port 80 "scripts/web.sh"
add WindowsXP tcp port 443 "scripts/web.sh"

create Windows2003Server
set Windows2003Server personality "Microsoft Windows Server 2003"
set Windows2003Server default tcp action reset
add Windows2003Server tcp port 80 "scripts/web.sh"
add Windows2003Server tcp port 25 "scripts/smtp.pl"
add Windows2003Server tcp port 443 "scripts/web.sh"

create Linux
set Linux personality "Linux Kernel 2.4.3 SMP (RedHat)"
set Linux default tcp action reset
add Linux tcp port 25 "scripts/smtp.pl"
add Linux tcp port 80 "scripts/web.sh"
add Linux tcp port 443 "scripts/web.sh"
add Linux tcp port 22 "scripts/test.sh"

create Cisco
set Cisco personality "Cisco router running IOS 12.1(5)-12.2(7a)"
set Cisco default tcp action reset
add Cisco tcp port 23 "scripts/router-telnet.pl"
```

Figura 54: Honeypots

Fonte: Autoria do grupo (2019)

Conforme a figura 55 abaixo, atribuímos também um *IP* estático aos respectivos *honeypots* e utilizamos as seguintes numerações:

```
bind 192.168.0.205 WindowsXP
bind 192.168.0.206 Windows2003Server
bind 192.168.0.207 Linux
bind 192.168.0.208 Cisco
```

Figura 55: IP no honeypot

Fonte: Autoria do grupo (2019)

Para que as máquinas *honeypot* utilizassem adequadamente o *DHCP*, foi necessário atribuímos um *MAC Address* para todas as placas de rede das máquinas *honeypot* utilizadas, para que somente depois e posteriormente pudéssemos ativarmos o *DHCP* conforme o exemplo e comandos abaixo:

```
set WindowsXP ethernet "00:00:24:ab:8c:12"
dhcp windows on eth0
```


Para simularmos as máquinas *honeypot* em nosso estudo e projeto, utilizamos o seguinte comando:

```
# honeyd -f <nome do arquivo de configuração> -l <caminho para arquivo de log>
```

O *HoneyD* detém vários *scripts* já inclusos em sua compilação e observamos que os mesmos ficam instalados no diretório */usr/share/honeyd/scripts*.

Para iniciarmos os testes e também para exemplificarmos aqui, o grupo fez o uso de alguns desses *scripts*, utilizando o seguinte comando:

```
add WindowsXP tcp port 23 "scripts/router-telnet.pl"
```

Notamos uma outra opção interessante para que pudéssemos utilizar, que foi um *script* que detém o objetivo de atrasar a conexão em determinadas situações e que, para executá-lo, utilizamos o seguinte comando abaixo:

```
set WindowsXP default tcp action tarpit open
```

O exemplo abaixo de configuração de um roteador foi o encontrado durante as configurações e testes realizados pelo grupo, que são:

```
### Cisco Router
create router
set router personality "Cisco IOS 11.3 - 12.0(11)"
set router default tcp action reset
set router default udp action reset
add router tcp port 23 "/usr/bin/perl scripts/router-telnet.pl"
set router uptime 1327650
bind 10.0.0.100 router
bind 10.0.1.100 router
bind 10.1.0.100 router
bind 10.0.0.200 router
bind 10.2.0.100 router
```

Foi possível também com este mesmo roteador observarmos e identificarmos qual rede estava atrelada e conectada a ele, conforme o exemplo:

```
route 10.0.1.100 link 10.1.0.0/16
```

Para inserirmos um roteador no ponto de entrada da rede, realizamos o comando *route entry* com o endereço *IP* do roteador e observamos também qual era a rede acessível através dele conforme o exemplo abaixo:

```
route entry 10.0.0.100 network 10.0.1.0/24
```

Além também de criarmos redes inacessíveis conforme exemplificamos logo abaixo:

```
route 16.23.166.1 unreachable 32.0.0.0/3
```

Notamos uma outra funcionalidade útil e interessante durante a etapa de configuração, que é a capacidade de adicionar uma máquina física externa à topologia virtual do *HoneyD*. E também durante esta mesma etapa de configuração,

deparamo-nos com uma máquina física com número de *IP* 10.1.1.3, este que foi agregado à rede virtual pela placa *eth0*, conforme exemplificado abaixo:

```
bind 10.1.1.53 to eth0
```

4.8 A TOPOLOGIA DO AMBIENTE DE REDES SIMULADO E SEGURO

Simple, bem desenhada e também de fácil compreensão, a topologia de nosso projeto foi definida de acordo com as necessidades e complexidade apresentadas pela ambientação de redes simulada e testada exaustivamente através dos ataques realizados juntamente com a proposta do respectivo trabalho, que é e foi totalmente atribuída às ferramentas e serviços de proteção e monitoramento, bem como toda a segurança implementada e apresentada nos títulos anteriores, podendo os membros deste grupo de trabalho descrevê-la como uma topologia a nível de rede local e ou corporativa de pequeno porte apenas, sendo e findando conclusivamente tal topologia, incumbindo-nos de denotarmos-la veemente de estar a um nível de segurança virtual elevadíssimo. Portanto, apenas para a apresentação e fácil compreensão de nossa proposta de trabalho, foi então projetado e desenvolvido em *softwares* diversos, um croqui da topologia, este atrelado e de acordo com a figura 56 demonstrado abaixo:

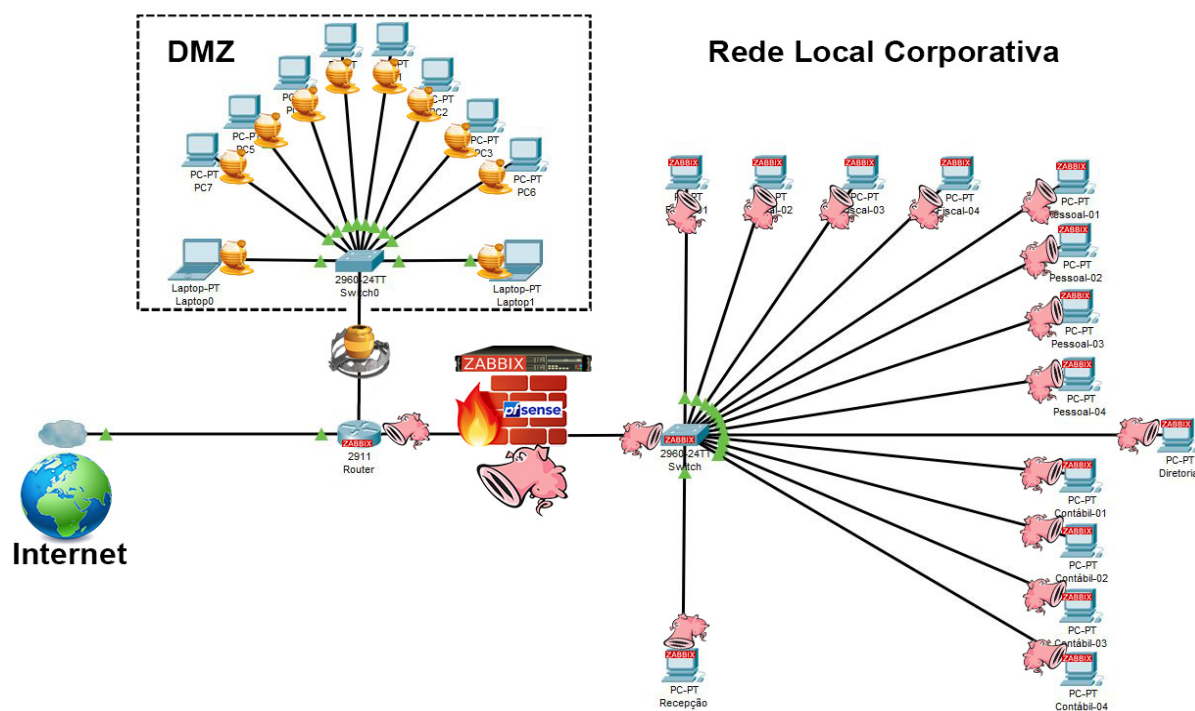


Figura 56: Topologia do ambiente de redes simulado e seguro
Fonte: Autoria do grupo (2019)

4.9 A APRESENTAÇÃO DO PROJETO CONCLUÍDO

4.9.1 A definição dos critérios

Na fase inicial de *brainstorming* e planejamento do nosso trabalho e projeto, por unanimidade do grupo, chegamos ao consenso final de que nossa proposta de trabalho, para se ter uma clara e evidente aceitação por parte dos docentes, dos colegas e de todos que iriam participar, apreciar e comprar a nossa ideia após a fase conclusiva de nosso projeto, em específico na fase de apresentação final do trabalho de conclusão de curso, decidimos então como equipe, para que conseguíssemos demonstrar o projeto concluído, sendo ele executado na prática e apresentado ao mesmo, firmamos na prerrogativa então de que este projeto deveria ser evidenciado de uma forma que houvesse uma excelente aceitação, com bons olhos por quem o visse e de preferência com unanimidade por quem viesse a tomar ciência do mesmo.

Definimos com veemência então, os critérios para que tal aceitação do projeto concluído ocorresse de forma focada e satisfatória, onde o mesmo iria ser executado e demonstrado da maneira mais simples possível, com excelente compreensão da proposta de trabalho, com excelente visualização, audível ao extremo e que, por último o primordial, que fosse extremamente criativo aos olhos de nossos familiares, convidados, colegas e docentes clientes.

4.9.2 As ferramentas disponíveis

À nossa inteira disposição, temos uma excelente escola, com ótimos recursos que nos fez chegar até esta fase conclusiva do curso, pois bem, mas devido à algumas limitações de recursos técnicos da escola, onde a maioria delas sendo e estando até indisponíveis para as necessidades que o grupo detinha, então vislumbramos que não seria possível aproveitarmos e tirarmos ao máximo de nossa capacidade técnica e criativa em relação sobre o quê, como e onde iríamos utilizar tais recursos, fator negativo este responsável que gerou-nos frustração e não foi possível realizarmos nosso intento até aquele exato momento.

Para a imensa sorte e felicidade do grupo, um dos integrantes, além de deter tais recursos técnicos, detêm também todo o conhecimento técnico necessário para que os critérios pré-definidos da apresentação de nossas iniciais

vingasse com a devida excelência almejada em nossa apresentação final do projeto.

Tais recursos técnicos quando enfatizados aqui pelo grupo, estamos falando e denotando sobre os recursos audiovisuais, que são as várias ferramentas e *softwares* à nossa inteira disposição, que são:

- Equipamentos de gravação de vídeo: Câmeras cropadas e semiprofissionais, detendo qualidade em gravação de vídeo e imagem em *full HD (1080p)*;
- Lentes semiprofissionais: Lentes variadas, com cropagem, com focagem de imagem específica para cada situação e ambiente, tornando-se indispensáveis na fase de gravação de qualquer projeto almejado;
- Sistema de áudio: Microfones específicos, sendo eles de modelos canhão, *shotgun* e lapelas, detendo uma excelente captação de voz e som com qualidade digital;
- Sistema de Iluminação: Iluminação adequada, composta por um eficiente número de lâmpadas de modelo *LEDs*, sendo o suficiente e o necessário para termos qualidade extrema para o pós-produção;
- Ambiente adequado: Estúdio semiprofissional, preparado adequadamente e propositalmente para os pós-produções, com espaço suficiente para a realização de curtas tomadas de gravação de filmagens de uma ou duas pessoas;
- Equipamentos de informática para edição: Computador com excelente configuração, detendo periféricos específicos para a utilização de efeitos e gráficos, bem como para a utilização de uma excelente pós-produção, tendo como principal atributo um excelente processamento de vídeo e imagens para a realização da renderização final de qualquer projeto;
- Softwares de edição de áudio, vídeo e imagens: Pacote *Adobe*, *Sound Forge* e *MS Office*, *softwares* estes sendo os responsáveis e indispensáveis pelas “mágicas” realizadas no pós-produção, aplicando efeitos variados, criativos e interativos.

Ferramentas estas que definimos e julgamos serem indispensáveis e de suma importância para a utilização na criação e em toda pós-produção de ideias criativas que o grupo viria a deter ao longo de nosso projeto.

Nossa crença nesta estratégia, de termos tais ferramentas à nossa disposição, estava e está atrelada até mesmo à experiência de apresentações de trabalhos vivenciadas no passado e que, felizmente obtivemos êxito e grande aceitação quando apresentados por nós, pois tais experiências citadas foram inclusive obtidas por todos os integrantes do grupo quando em realização de atividades curriculares nos semestres passados.

4.9.3 A proposta da apresentação

Definidos os critérios e detendo as ferramentas necessárias e adequadas para utilizarmos, em comum e unânime consenso, decidimos então realizarmos a gravação da tela do desktop de todos os ataques bem-sucedidos, bem como todos os ataques frustrados pela nossa implementação já concluída, sendo ataque e defesa apresentada em duas máquinas virtuais, uma do lado da outra em todo o perímetro do quadrado retangular do desktop enquanto gravando-as em ação.

Após as referidas gravações do desktop, definimos também que gravaríamos pequenas tomadas de gravação em vídeo, onde cada integrante do grupo apresenta e fala à frente de uma câmera, toda a nossa proposta de trabalho. Os assuntos, os ataques e toda nossa implementação fora previamente discutida, planejada e dividida igualitariamente para todos nós, equalizando assim a carga de trabalho ao grupo, obtendo com isso o material adequado, necessário e bem distribuído para o pós-produção na edição de vídeo de nosso projeto.

Com material de gravações conclusos e em mãos, tanto das gravações do desktop, quanto de nossas próprias tomadas de gravações, partimos então para a realização de toda a edição dos vídeos, imagens e sons utilizados em toda a extensão da mídia que contém nosso projeto já concluso e finalizado.

O diferencial que julgamos ter, fora a criatividade difundida dentro de nosso tema, pois atribuir essa competência comportamental e ao mesmo tempo utilizá-la no vídeo de nossa proposta de trabalho na apresentação final, foi primordial, pois decidimos pela inserção de um quarto integrante ao grupo, este meramente fictício, mas de suma e fundamental importância e que, este personagem é muito conhecido na rede mundial de computadores, devido à sua ascensão em toda a *web*, que também é conhecido pelo pseudônimo de *Anonymous*, onde o mesmo representa os milhares de *hackers* e *crackers* que por

deterem profundos conhecimentos em códigos maliciosos e programações em geral, guerreiam entre si no mundo inteiro quando em frente à uma tela de computador, sempre intentando o mal com corriqueiras invasões cibernéticas e muitas vezes chegam a ser extremamente prejudiciais e danoso a vários usuários da *web*.

Em nosso vídeo do projeto finalizado, este personagem foi amplamente utilizado para representar e apresentar previamente e rapidamente todos os ataques estudados, pesquisados e executados por nós mesmo, pois o nosso foco é e foi a implementação da segurança, onde detalhamo-las em nossas tomadas de gravações, todos os procedimentos e ferramentas da implementação de segurança utilizadas nas ocorrências de todos os ataques deferidos contra uma rede simulada de computadores, simulando inclusive serem quatro partidas do bem contra o mal, do *Anonymous* contra nós, os técnicos de redes de computadores da escola SENAI Santos Dumont, atribuindo à estas “partidas”, pitadas de criação com as respectivas edições de vídeos, com uma variedade de efeitos visuais e sonoros, além de propor um pós-produção eficiente e eficaz, tornando o projeto finalizado um atrativo apresentável a quem assiste, fixando totalmente a atenção do telespectador, fazendo-o realmente compreender de forma clara e simples, toda a nossa proposta de trabalho e projeto em segurança da informação, que é a implementação de um ambiente seguro em redes de computadores.

4.9.4 Projeto e vídeo apresentado em âmbito escolar

No dia 05 de dezembro de 2019, aproximadamente as 8h20 minutos, tivemos o imenso prazer de apresentarmos no auditório da escola SENAI Santos Dumont, todo o nosso estudo e pesquisas realizadas em âmbito escolar acerca do tema escolhido por nós, bem como a apresentação de nossa proposta de trabalho, esta que fora toda idealizada em um vídeo com 14 minutos de duração aproximadamente.

O *feedback* por parte dos docentes e colegas que apreciaram a nossa apresentação, foram extremamente positivos, fator este que nos fez termos uma serena reflexão e pensarmos de que valeu muito a pena todo o esforço e empenho aplicado em nosso projeto, que fora muito bem planejado e idealizado por todos os integrantes do grupo, que inclusive rendeu-nos a validação positiva e em público de

todo o nosso trabalho de conclusão de curso por nossos professores presentes na banca, os senhores Paulo Eduardo Galvão Alves e Kleber Gelli.

5 RESULTADOS E DISCUSSÃO

Os resultados inicialmente encontrados acerca da topologia do projeto e do objeto de trabalho proposto, foram extremamente exaustivos e frustrantes para todo o grupo em geral, em específico aos responsáveis pela implementação do projeto e pela complexa edição de todo o trabalho em si.

À parte que compete à implementação do projeto, todo o dispêndio do grupo se deu por conta da configuração e integração dos quatro *softwares*, sendo ambos serviços distintos atuando em tempo real em uma rede local, esta propositalmente preparada para a realização dos testes, onde todos eles foram incumbidos de realizar seus objetivos com um único foco, que era o de proteger e monitorar toda a rede local de computadores. Porém, a inexperiência do grupo em relação a ataques cibernéticos, a códigos maliciosos e ao fomento dos testes de penetração, até então eram praticamente um conhecimento técnico desconhecido e inimaginável para todos os integrantes do grupo e que, encontramos muitos obstáculos, problemas e empecilhos diversos para que antes conseguíssemos realizarmos a execução dos mesmos e pudéssemos demonstrar contemporaneamente a eficiência apregoada e vendida inicialmente no planejamento deste trabalho. À edição de todo o trabalho em si, esta citada no fim do primeiro parágrafo, compete a ela toda a parte de planejamento, introdução, desenvolvimento, execução, conclusão e principalmente da apresentação do projeto implementado, pois houveram árduas atividades e tarefas distintas com uma complexidade imensurável para superarmos e avançarmos para a fase subsequente, conclusiva e final de todo o nosso trabalho.

Sobre o cerne de nossa proposta de trabalho, é de grande relevância ressaltarmos sim, que houve inúmeras e exaustivas tentativas acerca dos ataques praticados juntamente e ao mesmo tempo com a instalação, bem como a complexa configuração de toda a proteção da rede simulada, onde em determinados momentos chegamos até mesmo a discutirmos entre nós, pensarmos e a duvidarmos que os objetivos específicos de nosso projeto fora excedidos e não seriam atingidos de forma satisfatória e ou até mesmo, podemos dizer também e com propriedade que chegamos ao cume de nosso controle emocional com uma

pressão desenfreada sobre nós alunos, pensando que talvez tivéssemos nos exacerbado em relação ao desafio aceito nas iniciais das aulas de projetos, que é e foi o compromisso assumido pelo grupo acerca do tema do trabalho escolhido pelos mesmos.

Todo o trabalho e tempo despendido acerca do projeto, podemos dizer com propriedade que não foi de maneira alguma em vão, pois há o que se ressaltar e inclusive dar mérito sim aos excelentes resultados obtidos no decorrer das pesquisas, dos estudos e durante todo o desenvolvimento, até findar na conclusão, pois a busca incessante e deliberada pelo desconhecido, bem como as novas experiências adquiridas, nos ensejou experimentarmos assuntos de cunho técnico diferenciados do aprendizado do dia a dia quando em curso no SENAI, chegando a sentirmos na pele uma competência técnica aprofundada em assuntos relacionados à políticas de segurança da informação, inclusive ajudando-nos a nos educarmos como meros usuários de computadores quando acessando a rede mundial e ou até mesmo com simples atitudes e bons comportamentos quando a frente e responsável por um sistema e rede qualquer.

Como já dito, os resultados foram todos satisfatórios, pois alcançamos e atingimos os objetivos gerais e específicos do trabalho, dando-nos inclusive uma sensação de missão cumprida, uma sensação de que fizemos sim toda a nossa lição de casa no decorrer desses dois anos de curso e que, além de gratificante foi também satisfatório e prazeroso visualizarmos uma etapa e ou um grande desafio por assim dizer, este vencido, onde no final se vislumbra um protótipo de segurança robusto e eficaz, funcionando e atuando no mesmo nível das necessidades que o mercado de trabalho, em específico a área de segurança da informação almeja e enseja de um profissional pleno de tecnologia da informação.

Independente das dificuldades e limitações em particular de cada integrante do grupo, achamos sim de fundamental importância, relatar, registrar e destacar o imenso comprometimento dos membros e da equipe de trabalho para com o projeto em si, pois foi extremamente relevante tal compromisso, bem como as competências técnicas e comportamentais demonstradas e apresentadas por cada integrante ao longo do trabalho, principalmente ao depararmos-nos com diversos desafios, pois as características técnicas distintas de cada integrante, fizeram toda a diferença na execução e na apresentação deste projeto, que para

nós em particular, além de ser uma vitória, foi e é um imenso sucesso o resultado obtido no fim deste trabalho.

6 CONCLUSÃO

Atestamos com veracidade e na prática, todo o intento do projeto e trabalho proposto, pois seria redundante não demonstrarmos algo tão robusto em segurança da informação e ou até mesmo se não houvesse a materialização daquilo que validamos com veemência e perspicácia nas iniciais do projeto, que é constatar em tempo real, a simulação de uma rede local e ou corporativa e principalmente instalar, configurar e demonstrar a eficiência e a eficácia dos quatro *softwares* atuando juntos, sendo o primeiro um excelente *firewall* como o *Pfsense* com todos os seus serviços, *plug-ins* e pacotes disponibilizados para utilização e ou até mesmo constatar também a pertinência positiva e efetiva que é um *IDS/IPS* atuante na rede, este denominado como *Snort*, um produto disponibilizado gratuitamente pela empresa e desenvolvedora *CISCO*, nos proporcionando excelentes resultados de alertas e bloqueios ao realizarmos os testes de penetração. Mas mais abrangente é, que além de detectar, analisar e excluir ameaças e intrusões cibernéticas, é extremamente interessante deter o poder de monitoramento sob todos os pacotes que circulam na rede, expondo uma excelente e amigável interface gráfica para conduzir e relatar todo o tráfego, gerando gráficos, relatórios e alertas aos administradores, além também de obter um monitoramento detalhado de uma rede, detivemos várias ferramentas e ações para se utilizar. *Software* este denominado *Zabbix*, onde o mesmo, embora não tenha características técnicas de proteção em segurança, mas sim de monitoramento, este não deixou a desejar e em todos os ataques, o mesmo detectou anomalias e tráfegos de dados inconsistentes na rede local testada, sempre monitorando e alertando o administrador de redes.

De lambuja, aos atacantes mal-intencionados, ou melhor dizendo, aos testes de penetração realizados na rede, foi instalado e disponibilizado em uma *DMZ*, um *software/serviço* de grande avalia para os excelentes resultados obtidos, pois além de simular com veemência uma rede extensa de computadores com variedade de portas abertas, contendo inclusive falsos arquivos nas mesmas e principalmente com o intuito de dispersar e enganar o atacante, o *HoneyD* demonstrou ser de grande utilidade até mesmo na identificação falsa-positiva do

inimigo atacante, alertando inclusive seu parceiro *Zabbix* que monitora toda a rede, gerando *logs* e alertando incisivamente o administrador da rede na sua interface gráfica, onde ambos demonstraram durante os testes, sempre agirem integrados um ao outro quando na realização de todos os testes efetivados por nós, bem como a integração do potente *firewall Pfsense* atuando consistentemente com um de seus pacotes de serviços, este denominado *Snort*, que foi instalado e pré-configurado com todas as *rules* (regras) necessárias ao nosso projeto, atuando fortemente na interface gráfica que o *Pfsense* disponibiliza e demonstra ao mesmo tempo, extrema eficácia na detecção e bloqueios de intrusos e ou ameaças em uma rede de computadores.

Ao findarmos o projeto na fase de execução, consideramos a implementação de um ambiente seguro em redes de computadores concluída com grande êxito, inclusive reforçamos nossa ideia, de ser uma implementação fortemente atuante em segurança cibernética quando integrada a vários *softwares* e serviços em uma única rede local, pois ao obtermos e depararmos-nos com os excelentes resultados obtidos, compreendemos que todo o dispêndio nos estudos e nas pesquisas envolvendo os integrantes do grupo, consideramos que nada não foi em vão e sim conclusivo e efetivo no quesito experiência para nós, esta extremamente importante para nossas carreiras profissionais na área de tecnologia da informação, em específico à área de segurança da informação, onde neste projeto tivemos a excelente oportunidade de demonstrar e apresentar um projeto muito bem planejado, exaustivamente executado, testado, demonstrado e com muita persistência obtivemos os melhores resultados possíveis, motivos estes sendo em particular para cada um de nós, mais uma vez, um imenso sucesso.

7 RECOMENDAÇÕES

As recomendações que deixamos a quem quer que leia este documento acerca do nosso trabalho, é que mesmo com grandes dificuldades nas etapas de um projeto qualquer na área de segurança, enfatizamos e recomendamos pela persistência no intento dos respectivos testes e que, pesquisem sim, pesquisem muito mesmo, pois sem estudos na área e sem a devida comprovação empírica da fase executória do projeto, não há como obter-se bons resultados, sendo e ficando até inviável investir em um tema tão complexo.

O grupo testou, instalou e configurou os quatros *softwares* em um servidor que não era um servidor físico, mas sim um outro computador de um integrante do grupo, onde simulamos nesta máquina ela ser um servidor físico, fazendo o uso do *software/servidor XenServer* e obtivemos resultados satisfatórios, mas não tão compensatórios, pois além de não determos *hardwares* ideais, suficientes e compensatórios, também não era o foco de nossos estudos e objetivos acerca deste extenso trabalho e projeto, até porque não homologamos na etapa do desenvolvimento deste documento a utilização do mesmo por nós, mas fazemos jus sim e recomendamos portanto, que alguém ou algum grupo em um breve futuro, possa utilizar nosso projeto para efeito de estudos, pesquisas e quem sabe utilizá-lo para outros e maiores projetos, como por exemplo a implementação deste ambiente de segurança em um servidor físico e real, com uma gama real de computadores na rede, podendo assim obter outros tipos de resultados e quem sabe até mesmo, melhores do que os encontrados por nós.

REFERÊNCIAS

ABNT NBR ISO/IEC 17799. **Tecnologia da informação - Técnicas de segurança - Código de prática para a gestão da segurança da informação**, Segunda Edição, 2005.

AHMAD, David R. Mirza e RUSSEL, Ryan. **REDE SEGURA NETWORK**. Alta Books, 2002, p. 46.

ALBUQUERQUE, Ricardo e RIBEIRO, Bruno. **Segurança no Desenvolvimento de Software – Como desenvolver sistemas seguros e avaliar a segurança de aplicações desenvolvidas com base na ISO 15.408**. Editora Campus. Rio de Janeiro, 2002, p. 25 - 33.

ALVARENGA, Roni Peterson Cunha de. **Monitoramento e Segurança: uma abordagem sobre como o Zabbix pode contribuir com relação à segurança e a gestão de suas melhores práticas em Tecnologia da Informação**. 2015. Monografia (Especialização) - Curso de Tecnologias e Sistemas de Informação, Universidade Federal do Abc, Santo André, 2015, p. 91.

AMOROSO, E.G. **Intrusion Detection: An Introduction to Internet Surveillance, Correlation, Traps, Trace-Back and Response**. Intrusion.Net Books, 1999, p. 25 - 39.

ARAÚJO, Tiago Emílio de Sousa; MATOS, Fernando Menezes; MOREIRA, Josilene Aires. Intrusion detection systems' performance for distributed denial-of-service attack. In: **Electrical, Electronics Engineering, Information and Communication Technologies (CHILECON), 2017 CHILEAN Conference on**. IEEE, 2017, p. 1 - 6.

ASSUNÇÃO, M.F. **Valhala Honeypot - Free Open Source Honeypot for the Windows system**. Disponível em <<http://valhalahoneypot.sourceforge.net/>>. Acessado em 28/10/2019.

ASSUNÇÃO, Marcos F. **Honeypots e Honeynets**. Florianópolis: Visual Books, 2009, p. 3 - 109.
ASSUNÇÃO, Marcos F. **Segredos do Hacker Ético**. 5 ed. Florianópolis: Visual Books, p. 2014, 27 - 65.

AZEVEDO, R. P. **Deteccão de ataques de negação de serviço em redes de computadores através da transformada Wavelet 2D**. Dissertação de mestrado. UFSM, Centro de Tecnologia, Programa de Pós-Graduação em Informática, RS. 2012, p. 18 - 32.

BERTHOLDO, Leandro Márcio; ANDREOLI, Andrey Vedana; TAROUCO, Liane M. R. **Gerência de Segurança Através do Uso de Netflow**. Disponível em: <http://www.rnp.br/_arquivo/wrnp2/2003/gsaun01a.pdf>. Acesso em: 18/11/2019.

BEZERRA, Adonel. **Evitando Hackers: Controle seus sistemas computacionais antes que alguém o faça**. Rio de Janeiro, Ciência Moderna. 2012, p. 06 - 33.

BLACK, T. L. **Comparação de ferramentas de gerenciamento de redes**. Porto Alegre: Universidade Federal do Rio Grande do Sul, 2008, p. 34 - 46.

BORAN, Sean. **IT SECURITY COOKBOOK**, 1996. Disponível em <http://www.boran.com/security/>. Disponível em: 17/10/2019.

BORGES; Ciro Fernando Preto, BENTO; Paulo Diego Nogueira; **Segurança de Redes utilizando Honeypot**. Belem. 2006, p. 44 - 69.

BROWN, S; LAM, R; PRASAD, S; RAMASUBRAMANIAN, S; SLAUSON, J. **Honeypots in the Cloud**. Disponível em < http://www.joshslauson.com/pdf/cs642_project.pdf >. Acessado em 17/10/2019.

CAMPOS, André. **Sistema de Segurança da Informação**. 3 ed. Florianópolis: Visual Books, 2014, p. 05 - 28.

CARVALHO, Alan Henrique Pardo. Segurança de aplicações web e os dez anos do relatório OWASP Top Ten: o que mudou? **Fasci-Tech – Periódico Eletrônico da FATEC-São Caetano do Sul**, São Caetano do Sul, v.1, n. 8, Mar./set. 2014, p. 6 - 18.

CASWELL, BRIAN et al. Snort 2 – Sistema de Detecção de Intruso - Open Source. Rio de Janeiro: Alta Books, 2003, p. 8 - 24.

CERT.br. **Recomendações para Melhorar o Cenário de Ataques Distribuídos de Negação de Serviço**. 2016. Disponível em: <<https://www.cert.br/docs/whitepapers/ddos/#6.3>>. Acesso 12/11/2019.

CERT.br: **Cartilha de segurança para Internet**, versão 4.0 /– São Paulo: Comitê Gestor da Internet no Brasil, 2012, p. 21 - 26.

CETIC. **CENTRO DE ESTUDOS SOBRE AS TECNOLOGIAS DA INFORMAÇÃO E DA COMUNICAÇÃO**. 2013. Disponível em: < <http://www.cetic.br/>>. Acessado em: 17/10/2019.

CHIN, Liou Kuo. **Rede Privada Virtual - VPN**, 1998. Boletim bimestral sobre tecnologia de redes. RNP – Rede Nacional de Ensino e Pesquisa, 1998. Vol. 2, Nº 8. Disponível em: <http://www.rnp.br/newsgen/9811/vpn.html>. Acessado em: 17/10/2019.

CISCO, **Cisco. Netranger documentation**. Disponível em: <<http://www.cisco.com/univercd/cc/td/doc/product/iaabu/netranger/>> Acesso em: 25/10/2019.

COSTA, D.G. **Administração de Redes com scripts: Bash Script, Python e VBScript**. Rio de Janeiro: Brasport, 2010, p. 32 - 48.

CRESPO DE CARVALHO, J., ENCANTADO, L. **Logística e Negócio Eletrônico**. PORTO: SPI – Sociedade Portuguesa de Inovação. 2006, p. 14 - 31.

CROSBY, Philip B. **Qualidade é Investimento**. José Olympio Editora. 5ª Edição, Rio de Janeiro, 1992. SANDHU, Ravi S. e SAMARATI, Pierangela. **Authentication, Access Control, and Intrusion Detection**. IEEE Communications, 1994, p. 6 - 11.

CRUZ, Frank. **Entendendo um sistema de detecção de invasões**. Disponível em: <http://www.timaster.com.br/revista/artigos/main_artigo.asp?codigo=100>. Acesso em: 25/10/2019.

DA SILVA, Pilar; RANGHETTI, Denise; STEIN, Lilian Milnitsky. Segurança da Informação: uma reflexão sobre o componente humano. *Ciências & Cognição*, v. 10 2007, p. 06 - 36.

DUARTE Júnior, Antonio Marcos. **A importância do Gerenciamento de Riscos Corporativos**. UNIBANCO – Global Risk Management. Disponível em: <http://www.risktech.com.br/>. Acessado em: 25/10/2019.

GIL, Antonio Carlos. **Como elaborar projetos de pesquisa**. 5. ed. São Paulo: Atlas, 2010, p. 14 - 35.

HANDLEY. M. Internet Denial-of-Service Considerations **RFC 4732**. Disponível em: < <http://tools.ietf.org/html/rfc4732>>. Acesso em: 25/10/2019.

ISO 1 7799. **ABNT NBR ISO/IEC 1 7799:2005 – Tecnologia da Informação – Técnicas de segurança – Código de prática para a gestão da segurança da informação**. Associação Brasileira de Normas Técnicas – Rio de Janeiro: ABNT, 2005.

KUROSE, James F.; ROSS, Keith W. **Redes de Computadores: uma abordagem top-down**. 5. ed. São Paulo: Pearson Education do Brasil. Tradução de: Opportunity Translations, 2010, p. 614.

KUROSE, James; ROSS, Keith. **Redes de Computadores e a Internet: uma abordagem top-down**. 3. ed. São Paulo: Pearson, 2005, p. 06 - 41.

LAKATOS, Eva M.; MARCONI, Marina A. **Fundamentos de metodologia científica**. 5. ed. São Paulo: Atlas. 2003, p. 8 - 14.

LARI, PAULO AUGUSTO MODA; AMARAL, DINO MACEDO Snort, MySQL, Apache e ACID. Rio de Janeiro: Brasport, 2004, p. 5 - 12.

LASALA, Kenneth P. **Human Performance Reliability: A Historical Perspective**. IEEE Transactions on Reliability, vol 47, 1998, p. 06.

LAUFER; Rafael P. **Introdução a sistemas de detecção de intrusão**. Rio de Janeiro, 2003. Disponível em: http://www.gta.ufrj.br/grad/03_1/sdi/index.htm. Acesso em: 18/11/2019.

LEVESON, Nancy G. et al. **Analyzing Software Specifications for Mode Confusion Potential**. Workshop on Human Error and System Development, 1997, p. 03 - 10.

LIMA, F. A. **Estudo do sistema de detecção de intrusão**. Curso de Especialização em Redes e Segurança de Sistemas Universidade Católica do Paraná. 2012, p. 11 - 38.

LIMA, João P. **Administração de Redes Linux**. Goiânia: Terra, 2003, p. 4 - 29.

LINDA, Ondrej; VOLLMER, Todd; MILOS. **Manic Neural network-based intrusion detection system for critical infrastructures**, em 'IJCNN'09, Int. Joint INNS-IEEE Conf. on Neural Networks', Atlanta, Georgia, USA, 2009, p. 1827–1834.

LINHARES, A. G.; GONÇALVES, P. A. D. S. Uma Análise dos Mecanismos de Segurança de Redes IEEE 802.11: WEP, WPA, WPA2 e IEEE 802.11w *. 2006, Recife. In: **PROCEEDINGS OF I CONGRESSO DE INICIAÇÃO CIENTÍFICA DA UNIBRATEC**, 2006, p. 1 - 17.

LOPES, Raquel V.; SAUVÉ, Jacques P.; NICOLLETTI, Pedro S. **Melhores Práticas Para Gerência de Redes de Computadores**. 1. ed. Editora Campus, 2002, p. 12 - 44.

MACÊDO, Márcio A.; QUEIROZ, Ricardo G.; DAMASCENO, Júlio C. jShield: **Uma Solução Open Source para Segurança de Aplicações Web**. Universidade Federal de Pernambuco. 2010, p. 09 - 23.

MAGENTO. **About US**. Disponível em: < <http://www.magentocommerce.com/company/> > Acessado em: 17/10/2019.

MARCIANO, João Luiz; LIMA-MARQUES, Mamede. O enfoque social da segurança da informação. Ci. Inf., Brasília, v. 35, 2006, p. 9 - 15.

MARTINELO, Clériston Aparecido Gomes; BELLEZI, Marcos Augusto. **Análise de Vulnerabilidades com OpenVAS e Nessus**. T.I.S. São Carlos, v. 3, n. 1, jan-abr 2014, p. 34 - 44.

MATTOS, Diogo Menezes Ferrazani; DUARTE, Otto Carlos Muniz Bandeira. **AuthFlow: Um Mecanismo de Autenticação e Controle de Acesso para Redes Definidas por Software**. Rio de Janeiro –RJ –Brasil 2014, p. 26 - 38.

MAURO, Douglas. R. & SCHMIDT, Kevin J. **Essentials SNMP**. Sebastopol: O' Reilly, 2001, 5 - 20. MCCLURE, Stuart; SCAMBRAY, Joel; KURTZ, George. **Hackers Expostos**. 7 ed. São Paulo: Bookman. 2014, p. 15 - 36.

MCKEOWN, Nick et al. Open Flow: enabling innovation in campus networks. **ACM SIGCOMM Computer Communication Review**, v. 38, n. 2 – 2008, p. 69 - 74.

MIRANDA, M. **Segurança virtual não é mais diferencial, é obrigação**. Revista E-Commerce Brasil. 6 Ed. 2011, p. 03 - 12.

MOHR, Rodrigo Fraga. **Análise de Ferramentas de Monitoração de Código Aberto**. 2012. TCC (Graduação) - Curso de Ciência da Computação, Instituto de Informática, Universidade Federal do Rio Grande do Sul, Porto Alegre, 2012, p. 69.

MORAIS, Guilherme Filipe Zorego Rodrigues. **Análise e Implementação de Sistemas IDS e IPS**. Tese de Doutorado. 2011, p. 22 – 50.

MOURA, Alex. **Gerenciamento de Redes com Software Livre**, out. 2005. Disponível em: <https://memoria.rnp.br/_arquivo/sci/2005/moura-alex_gerenciamento-redes.pdf>. Acesso em: 12/11/2019.

MURINI, C. T. **Análise de Sistemas de Detecção de Intrusão em Redes de Computadores**. JAI UFSM (Jornada Acadêmica Integrada da Universidade Federal de Santa Maria). 2013, p. 9 - 44.

NAKAMURA, E. **Segurança em de redes em cooperativos**. São Paulo: Novatec, 2007, 8 - 55.

NED, Frank. **Ferramentas de IDS**. Disponível em: <<http://www.rnp.br/newsgen/9909/ids.html>>. Acesso em: 25/10/2019.

NOBRE, JCA. Ameaças e Ataques aos Sistemas de Informação: Prevenir e antecipar. Cadernos UniFOA, Volta Redonda, ano 2. n. 5, dez. 2007. 2013, p. 04 - 48.

OPENSIMS. **Integrate Snort, Nmap, Nagios and Nessus**. Disponível em: <<http://opensims.sourceforge.net/#>>. Acesso em: 25/10/2019.

OWASP. **10 Top Web Vulnerabilities**. Disponível em <www.owasp.org/index.php/Top_10_2013>. Acesso em 17/10/2019.

PAULI, Josh. **Introdução ao WebHacking**. São Paulo: Novatec, 2014, p. 08 - 39.

PESSOA, Márcio. **Segurança em PHP**. São Paulo: Novatec, 2007, p. 15 - 17.

PFSENSE. **PfSense**. Disponível em: < www.pfsense.org.br >. Acesso em: 21/10/2019.

PIRES, A. S. **Tutorial de instalação do agente Zabbix**. João Pessoa. Out. 2010. Disponível em: <http://zabbixbrasil.org/files/Tutorial_de_instalacao_do_agente_Zabbix.pdf>. Acesso em 10/11/2019.

PIRES, Aécio. **Zabbix: Uma ferramenta para Gerenciamento de ambientes de T.I**. Natal: Expotec, 2015, p. 42.

POLETO, Olavo F. **Gerenciamento e Monitoramento de Redes II: Análise de Desempenho**, Jan. 2012. Disponível em: <http://www.teleco.com.br/tutoriais/tutorialgmredes2/pagina_2.asp>. Acesso em: 12/11/2019.

PORRAS, P. Stat. **A state transation analysis tool for intrusion detection**. Tese de mestrado, University of California at Santa Barbara, 1992, p. 15 - 38.

PROVOS, N. **Developments of the Honeyd Virtual Honeypot**. Disponível em: < <http://www.honeyd.org> > Acesso em: 17/10/2019.

REHMAN, U. R. (2013). **Intrusion Detection Systems with Snort: Advanced IDS Techniques with Snort, Apache, MySQL, PHP, and ACID**. Disponível em: <<http://ptgmedia.pearsoncmg.com/images/0131407333/downloads/0131407333.pdf>> Acesso em: 18/11/2019.

REZENDE, D. A.; ABREU, A. F. de. Tecnologia da Informação, ~ao Aplicada a Sistemas de Informação Empresariais. São Paulo: Atlas S.A, 2003, p. 24.

RIBEIRO, B. **Detecção de intrusos usando o snort**. Monografia de Pós-graduação. Universidade Federal de Lavras – MS 2005, p. 44.

ROESCH, M. **Snort - lightweight intrusion detection for networks**. In: LISA '99: Proceedings of the 13th USENIX conference on System administration, Berkeley, CA, USA. USENIX Association, 1999, p. 229 - 238.

ROSE, Marshall T.; McCLOGHRIE, Keith. **How to Manage Your Network using SNMP: The Network Management Practicum**. New Jersey: Prentice Hall, 1995. p. 6 - 11.

RUSSELL, Deborah; GANGEMI, G. T. Computer security basics. " O'Reilly Media, Inc.", 1991, 05.
SANTOS, Mauro Tapajós. **Gerência de Redes de Computadores**. Brasília: Escola Superior de Redes, 2011, p. 153.

SAUVÉ, J. P; LOPES, R.V; NICOLLETTI, P.S. **Melhores práticas para a gerência de redes de computadores**. 1. ed. Rio de Janeiro: Campus, p. 06 - 27.

Scarfone, K.; Mel, P. **Guide to Intrusion Detection and Prevention Systems (IDPS)**. National Institute of Standards and Technology. 2007, p. 15 - 44.

SÊMOLA, Marcos. **Gestão da Segurança da Informação – Uma visão Executiva**. Editora Campus. Rio de Janeiro, 2003, p. 08 - 34.

SEZER, Sakir et al. Are we ready for SDN? Implementation challenges for software-defined networks. **IEEE Communications Magazine**, v.51, n. 7, 2013, p. 36 - 43.

SHIREY, R. RFC 2828 – Internet Security Glossary. The Internet Society. 2000. Disponível em: <<http://www.ietf.org/rfc/rfc2828.txt?number=2828>>. Acesso em: 11/11/2019.

SILVA COSTA, Guilherme. **Detecção de Intrusão em Redes de Computadores: Algoritmo Imunoinspirado Baseado na Teoria do Perigo e Células Dendríticas**. Universidade Federal de Minas Gerais, Dissertação de mestrado. 2009, p. 15 - 44.

SILVA M. P.; SAMPAIO N. S. **Estudos de sistemas de detecção e prevenção de intrusões uma abordagem open Source**. 2006, p. 05 - 32.

SNORT. **Snort Uses Manual**. [S. l.]. 2010. Disponível em: <<http://www.snort.org/>>. Acesso em: 25/10/2019.

SPITZNER, L. **Honeypots: Definitions and Value of Honeypots**. 29 de maio de 2003. Disponível em: < <http://www.tracking-hackers.com/papers/honeypots.html>>. Acesso em: 17/10/2019.

SPITZNER, L. **Honeypots: Tracking Hackers**. Addison Wesley. 2002, p. 06 - 11.

STALLINGS, William, **SNMP, SNMPv2, SNMPv3 and RMON 1 and 2**. 3 ed. Boston: Addison-Wesley, 1998, p. 15 - 24.

STALLINGS, William. **Criptografia e segurança em redes**. 4. ed. São Paulo: Person Prentice Hall, 2008, p. 04 - 59.

STALLINGS, William. **Redes e sistemas de comunicação de dados: teorias e aplicações corporativas**. 5 ed. Rio de Janeiro: Elsevier, 2005, p. 07 - 22.

STALLINGS, William; BROWN, Lawrie. **SEGURANÇA DE COMPUTADORES: PRINCÍPIOS E PRÁTICAS**. 2. ed. Rio de Janeiro: Elsevier, Tradução de: Arlete Simille Marques, 2014, p. 726.

STATO FILHO, André. **Linux Controle de Redes**. 1ª ed. Florianópolis: Editores Visuais Books, 2009, p. 15 -33.

STONEBURNER, Gary. **Underlying Technical Models for Information Technology Security**. NIST Special Publication 800-33, 2001, p. 15 - 19.

SUNDARAM, Aurobindo. **An introduction to intrusion detection**. Crossroads: The ACM Student Magazine, 2 ed. Abril, 1996, p. 08 - 25.

SURICATA **Suricata-Vs_Bufo**. Disponível em: <<http://www.aldeid.com/wiki/Suricata-vs-snort>>. Acesso em: 25/10/2019.

TANEMBAUM, Andrew S; WETHERALL, D. **Redes de computadores**. Tradução Daniel Vieira. São Paulo: Pearson Prentice Hall. 5 ed 2011, p. 18 - 29.

TANEMBAUM, Andrew S; WOODHULL, Albert S. **Sistemas operacionais: Projeto e implementação**. Tradução: Edson Furmankiewicz.2. Ed. Porto Alegre-RS: Bookman, 2000, p. 06 - 44.

TANENBAUM, A. S. **Computer networks**. 4. Ed. Pearson, 2011, p. 18 - 21.

TANENBAUM, A. S. **Sistemas Operacionais Modernos**. 3 ed. São Paulo: Person Prentice Hall, 2009, p. 638.

TURBAN, E; LEIDNER, D; MCLEAN, E; WETHERBE, J. **Tecnologia da informação para gestão – transformando os negócios na economia digital**. 6.ed. Porto alegre: Bookman, 2009, p. 24 - 33.

VLADISHEV, Alexei, **Manual Zabbix**, Vs 3.0, 2016. Disponível em: <<https://www.zabbix.com/documentation/3.0/pt/manual>>. Acesso em: 21/10/2019.

WANG, J. **Computer network security: theory and practice**. Higher Education Press, 2009, p. 33.
WEIDMAN, Georgia. **Testes de Invasão: Uma introdução prática ao hacking**. Novatec Editora, 2014, p. 06 - 60.

ZABBIX BRASIL, **Comunidade Brasileira de Usuários do Zabbix**, 2016. Disponível em: <<http://www.zabbixbrasil.org/wiki/tiki-index.php?page=Implementando+Zabbix+Proxy>>. Acesso em: 21/10/2019.

ZABBIX, **Zabbix LLC**, 2016. Disponível em: <<http://www.zabbix.com/screenshots.php>>. Acesso em: 21/10/2019.

TERMO DE COMPROMISSO ÉTICO

ESCOLA SENAI SANTOS DUMONT

DECLARAÇÃO DE COMPROMISSO ÉTICO COM A ORIGINALIDADE CIENTÍFICO-INTELLECTUAL

Responsabilizamos-nos pela redação do trabalho de projeto de pesquisa e estudo, sob título “Implementação de um Ambiente Seguro em Redes de Computadores”, atestando que todos os trechos que tenham sido transcritos de outros documentos, bem como as imagens, gráficos e ou tabelas utilizadas (publicados ou não), que não sejam de nossa exclusiva autoria, estão todos (as) devidamente referenciados (as) ou somente indicados (as) por fonte e ano, conforme normas e padrões vigentes estabelecidas pela ABNT.

Declaramos, ainda, termos pleno conhecimento de que podemos ser responsabilizados legalmente caso infringjamos tais disposições.

São José dos Campos, 16 de dezembro de 2019.

Aluno: **Alessandro Roberto dos Reis Santos**
Número R.A.: 18137531

Aluno: **Bruno Souza dos Santos**
Número R.A.: 18137298

Aluno: **Giovani Hardt Filho**
Número R.A.: 18137531

Professor Orientador: **Paulo Eduardo Galvão Alves**

Orientando: Alessandro Roberto dos Reis Santos			
Orientando: Bruno Souza dos Santos			
Orientando: Giovani Hardt Filho			
Matrículas: 18137531 18137298 18137531	Turma: 4RMA 4RMA 4RMA	Data de Entrega: 16/12/2019 16/12/2019 16/12/2019	Telefones Celular: (12) 98867-8750 (12) 98121-4347 (12) 98183-8205
e-mail principal: <i>alessandro.coringa@yahoo.com.br</i>		e-mail alternativo: <i>alessandroreis5s@icloud.com</i>	
Está matriculado na Disciplina TCC? (x) Sim Se NÃO está matriculado, pretende cursá-la quando? _____º semestre de 20____		Previsão de término do TCC : 2º semestre de 2019 Prazo Final para conclusão do curso : 2º semestre de 2019	
TEMA			
TÍTULO DO TCC: IMPLEMENTAÇÃO DE UM AMBIENTE SEGURO EM REDES DE COMPUTADORES			
OBJETIVOS GERAIS E ESPECÍFICOS – DESCRIÇÃO SUCINTA (Se necessário anexar folhas complementares)			
Objetivos gerais Prover e simular um ambiente local, destacando e utilizando um sistema de segurança integrado e eficaz através de <i>softwares opensource</i> disponíveis e atuais. Objetivos específicos • Implementar um <i>firewall</i> customizado (<i>PfSense</i>); • Implementar um <i>IDS/IPS</i> (<i>Snort</i>); • Implementar um serviço de monitoramento de dados e pacotes (<i>Zabbix</i>); • Implementar um <i>Honeypot</i> (<i>HoneyD</i>); • Apresentar e simular uma rede vulnerável e uma rede segura e protegida.			
PROFESSOR ORIENTADOR: PAULO EDUARDO GALVÃO ALVES			
USO EXCLUSIVO – ORIENTADOR			
ORIENTAÇÃO: () SIM () NÃO DESISTÊNCIA DA ORIENTAÇÃO: () SIM () NÃO TCC é: () Monografia () Projeto Justificativa/Comentários: _____ _____ _____ _____			
_____ Aceite assinado pelo professor orientador São José dos Campos, 16 de dezembro de 2019.			